

Chapter 4 – LangGraph

Part I – LangGraph Fundamentals, Graph Architecture, State, Nodes & Edges

1. Why Do We Need LangGraph?



In previous chapters, we built the basic agent loop manually:

```
while True:

    response = llm.invoke(state)

    if response.tool_calls:

        result = execute_tool(
            response.tool_calls
        )

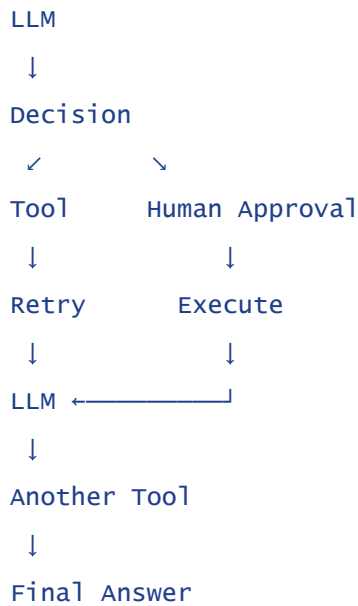
        update_state(result)
```

```
else:
```

```
    break
```

This works for simple agents.

But real agent workflows can become more complex:



Managing this using:

```
while loops
```

```
if/else statements
```

```
manual state updates
```

```
manual persistence
```

can become difficult.

LangGraph helps us represent this workflow explicitly as a:

Graph

2. What is LangGraph?



LangGraph is a framework for building stateful, multi-step LLM applications and agents using graph-based workflows.

Instead of thinking only in terms of:

```
Prompt
↓
LLM
↓
Response
```

we think in terms of:

```
State
↓
Node
↓
Edge
↓
Node
↓
Conditional Edge
↓
Node
```

3. Core LangGraph Mental Model

Remember four concepts:

STATE

→ Data flowing through the workflow

NODE

→ A function that performs some work

EDGE

→ Connects one node to another

CONDITIONAL EDGE

→ Chooses the next node dynamically

Together:

Graph

=

State

+

Nodes

+

Edges

+

Routing

4. Basic Graph

Suppose we want:

START



Agent



Tool

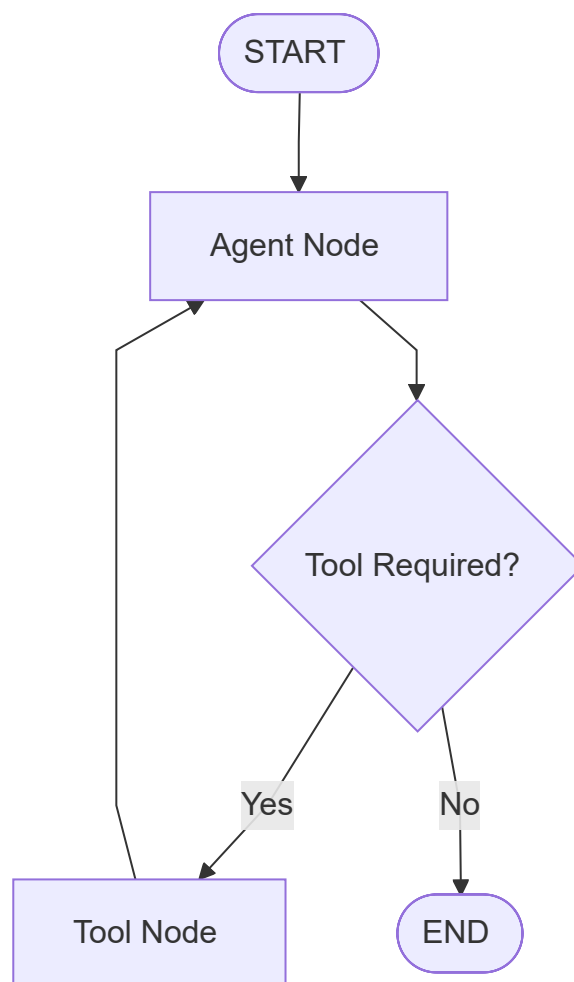


Agent



END

Graphically:



This already looks like the agent loop we created manually.

5. LangGraph vs LangChain



They are related but solve different problems.

LangChain

Provides useful abstractions for:

LLMs

Prompts

Tools

Messages

Retrievers

Structured Output

RAG Components

LangGraph

Focuses on:

Workflow orchestration

State management

Nodes

Edges

Loops

Branching

Persistence

Human-in-the-loop

A LangGraph application can use LangChain components inside its nodes.

Mental model:

LangChain

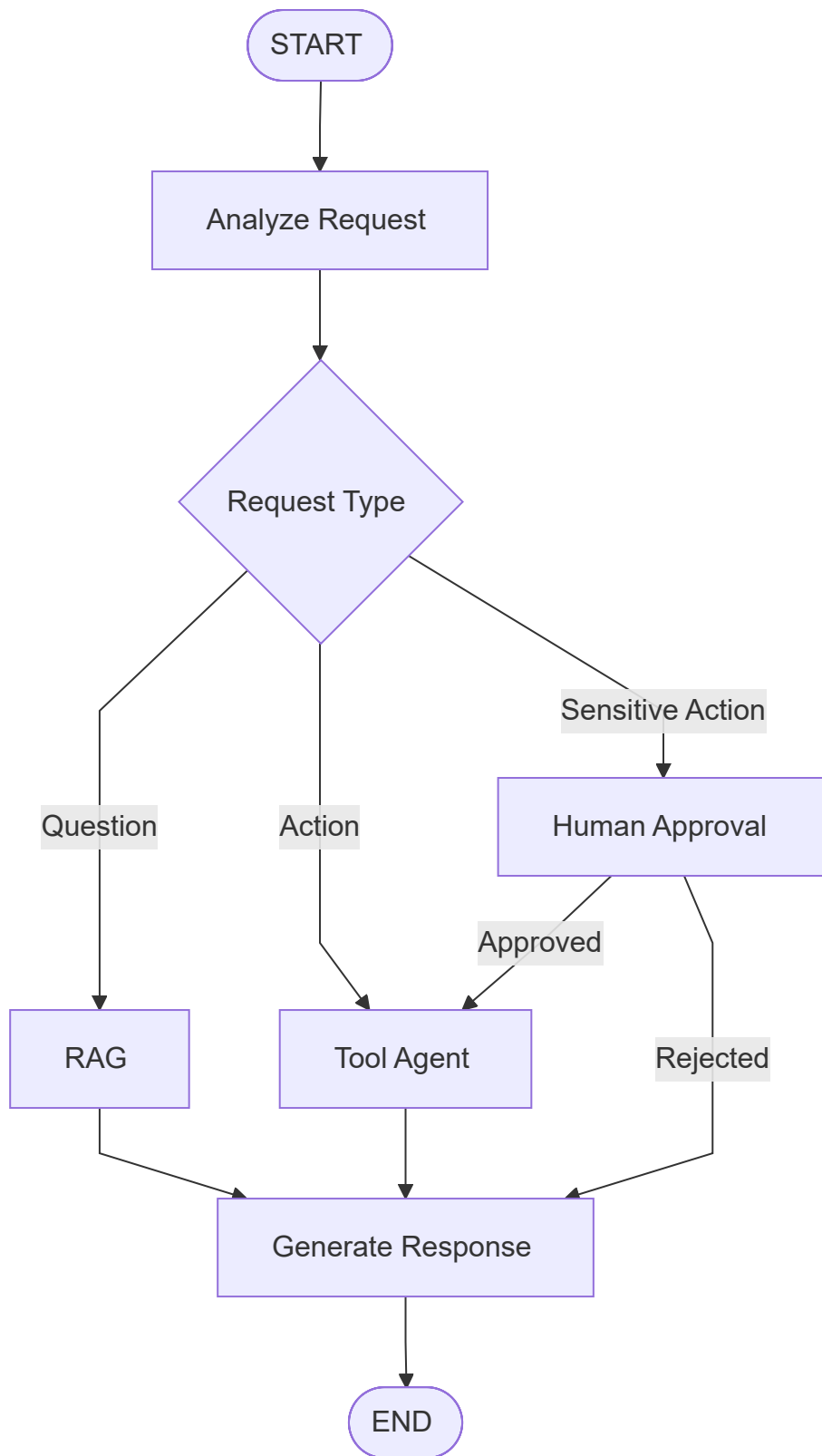
→ Components

LangGraph

→ Orchestration

6. Why Graphs?

Consider this workflow:



Representing this explicitly makes it easier to understand:

what can happen?

where can execution go?

where can it loop?

where is human approval required?

what state is passed between steps?

7. StateGraph



A common LangGraph abstraction is:

```
StateGraph
```

Conceptually:

```
from langgraph.graph import StateGraph
```

You create a graph around a state definition:

```
builder = StateGraph(  
    AgentState  
)
```

Then add:

Nodes

Edges

Conditional Edges

and compile the graph.

8. LangGraph State

State is the shared data passed between nodes.

Example:

```
class AgentState(TypedDict):  
  
    query: str  
  
    response: str
```

Then a workflow may receive:

```
{  
    "query":  
        "What is crop rotation?",  
  
    "response":  
        ""  
}
```

Each node can:

Read State

Perform Work

Return State Updates

9. Why TypedDict?

A common Python approach is:

```
from typing import TypedDict
```

Example:

```
class AgentState(TypedDict):  
  
    query: str  
  
    response: str
```

This gives a clear expected structure:

AgentState

query
→ string

response
→ string

It improves:

Readability

Type checking

Maintainability

10. State Example

```
from typing import TypedDict

class AgentState(TypedDict):

    query: str

    category: str

    response: str
```

Initial state:

```
{
    "query":
        "Tell me about wheat",

    "category":
        "",

    "response":
        ""
}
```

During execution:

Node 1

↓

category = "crop_information"

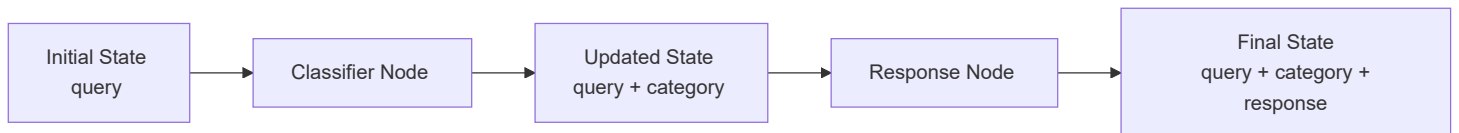
Later:

Node 2

↓

response = "wheat is..."

11. State Flow



The same logical state evolves as nodes execute.

12. What is a Node?



A:

Node

is a function representing one step in the graph.

Example:

```
def classify_query(state):  
  
    query = state["query"]  
  
    category = classify(query)  
  
    return {  
        "category":  
            category  
    }
```

This node:

Reads:
query

Performs:
classification

Updates:
category

13. Nodes Return Updates

An important idea:

A node does not necessarily need to return the complete state.

Suppose state contains:

```
{  
  "query": "...",  
  "category": "",  
  "response": ""  
}
```

The node can return only:

```
{  
  "category":  
    "crop_information"  
}
```

LangGraph applies this update to the graph state according to the state definition/reducers.

Conceptually:

```
Existing State  
+  
Node Update  
=  
Updated State
```

14. Node Example

```
def generate_response(state):  
  
    query = state["query"]  
  
    answer = llm.invoke(  
        query  
    )  
  
    return {  
        "response":  
            answer.content  
    }
```

Flow:

```
State  
↓  
generate_response()  
↓  
LLM  
↓  
Return response update  
↓  
Updated State
```

15. Adding Nodes

Conceptually:


```
builder.add_node(  
    "classifier",  
    classify_query  
)
```

Another node:

```
builder.add_node(  
    "response_generator",  
    generate_response  
)
```

Now the graph knows about:

```
classifier  
  
response_generator
```

But they are not connected yet.

16. What is an Edge?



An edge defines:

```
which node executes next?
```

Example:

```
Classifier
↓
Response Generator
```

Implementation:

```
builder.add_edge(
    "classifier",
    "response_generator"
)
```

Graph:



17. START and END

LangGraph workflows have conceptual:

```
START

END
```

They represent:

```
START
→ Entry point

END
→ workflow completed
```

Common imports:

```
from langgraph.graph import (  
    START,  
    END  
)
```

Then:

```
builder.add_edge(  
    START,  
    "classifier"  
)
```

and:

```
builder.add_edge(  
    "response_generator",  
    END  
)
```

18. Complete Simple Graph



Implementation:

```
builder.add_edge(  
    START,  
    "classifier"  
)
```

```
builder.add_edge(  
    "classifier",  
    "response_generator"  
)  
  
builder.add_edge(  
    "response_generator",  
    END  
)
```

19. First Complete LangGraph Example



```
from typing import TypedDict  
  
from langgraph.graph import (  
    StateGraph,  
    START,  
    END  
)  
  
class AgentState(TypedDict):  
  
    query: str  
  
    response: str  
  
def generate_response(state):  
  
    query = state["query"]  
  
    response = (  
        f"You asked: {query}"
```

```
)

return {
    "response":
        response
}

builder = StateGraph(
    AgentState
)

builder.add_node(
    "generate_response",
    generate_response
)

builder.add_edge(
    START,
    "generate_response"
)

builder.add_edge(
    "generate_response",
    END
)

graph = builder.compile()
```

We have now created our first LangGraph workflow.

20. Compiling the Graph



After defining:

State

Nodes

Edges

we call:

```
graph = builder.compile()
```

Conceptually:

Graph Definition

↓

Validation / Compilation

↓

Executable Graph

After compilation, the graph can be invoked.

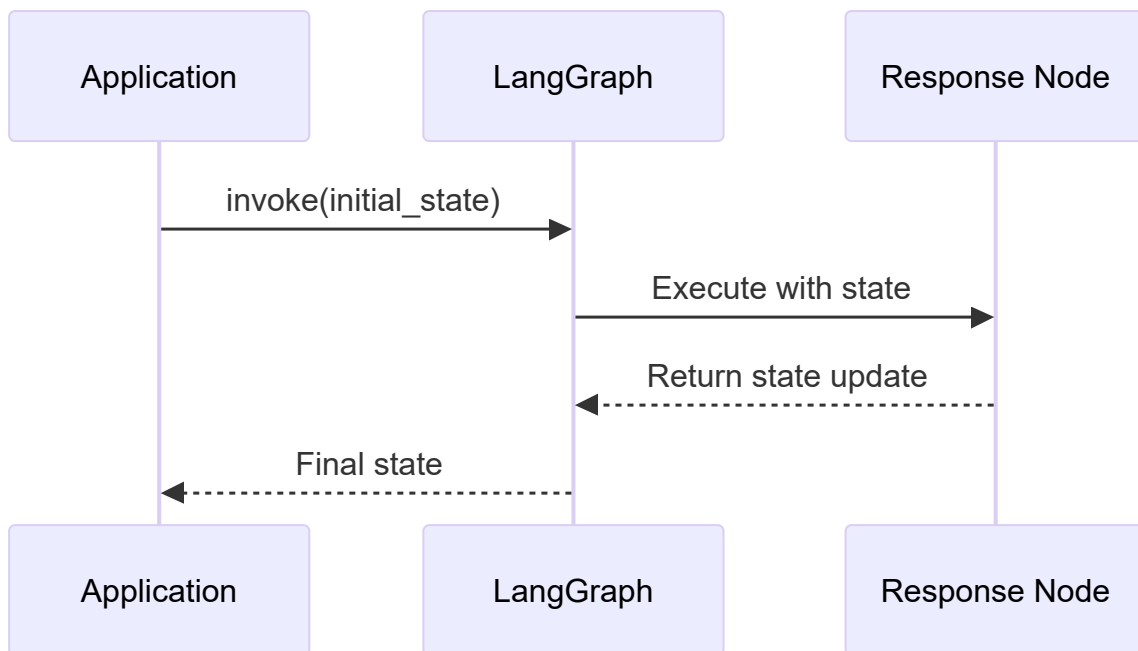
21. Invoking the Graph

```
result = graph.invoke(  
    {  
        "query":  
            "What is crop rotation?",  
  
        "response":  
            ""  
    }  
)
```

Possible output:

```
{  
    "query":  
        "What is crop rotation?",  
  
    "response":  
        "You asked: What is crop rotation?"  
}
```

22. Complete Execution Flow



23. Multiple Nodes

Let's add another node.

Goal:

```
User Query
↓
Normalize Query
↓
Generate Response
```

State:


```
class AgentState(TypedDict):
```

```
    query: str
```

```
    normalized_query: str
```

```
    response: str
```

24. Normalize Node

```
def normalize_query(state):
```

```
    query = state["query"]
```

```
    normalized = (
```

```
        query
```

```
        .strip()
```

```
        .lower()
```

```
)
```

```
    return {
```

```
        "normalized_query":
```

```
            normalized
```

```
    }
```

25. Response Node

```
def generate_response(state):
```

```
    query = state[
```

```
        "normalized_query"
```

```
    ]
```

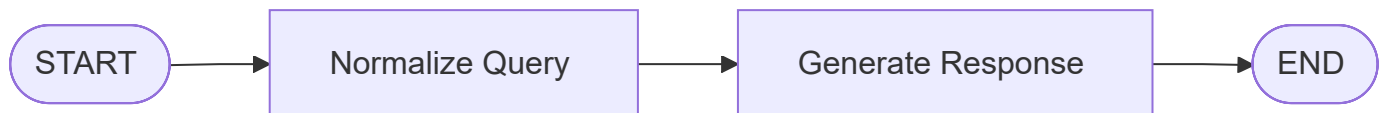
```
response = (  
    f"Processing: {query}"  
)  
  
return {  
    "response":  
        response  
}
```

26. Build Multi-Node Graph

```
builder = StateGraph(  
    AgentState  
)  
  
builder.add_node(  
    "normalize",  
    normalize_query  
)  
  
builder.add_node(  
    "generate",  
    generate_response  
)  
  
builder.add_edge(  
    START,  
    "normalize"  
)  
  
builder.add_edge(  
    "normalize",
```

```
        "generate"  
    )  
  
    builder.add_edge(  
        "generate",  
        END  
    )  
  
    graph = builder.compile()
```

27. Multi-Node Execution



State evolution:

Initial

query



After Normalize

query

normalized_query



After Generate

query

normalized_query

response

28. What is a Conditional Edge?



A normal edge always goes to the same destination.

Example:

```
A
↓
B
```

A conditional edge chooses the destination dynamically.

Example:

```
Classifier
  ↓
Request Type?
  ↙   ↘
Weather Crop
  ↓     ↓
Weather Crop
Node   Node
```

This is crucial for agent workflows.

29. Router Function

Suppose state contains:

```
{  
    "category":  
        "weather"  
}
```

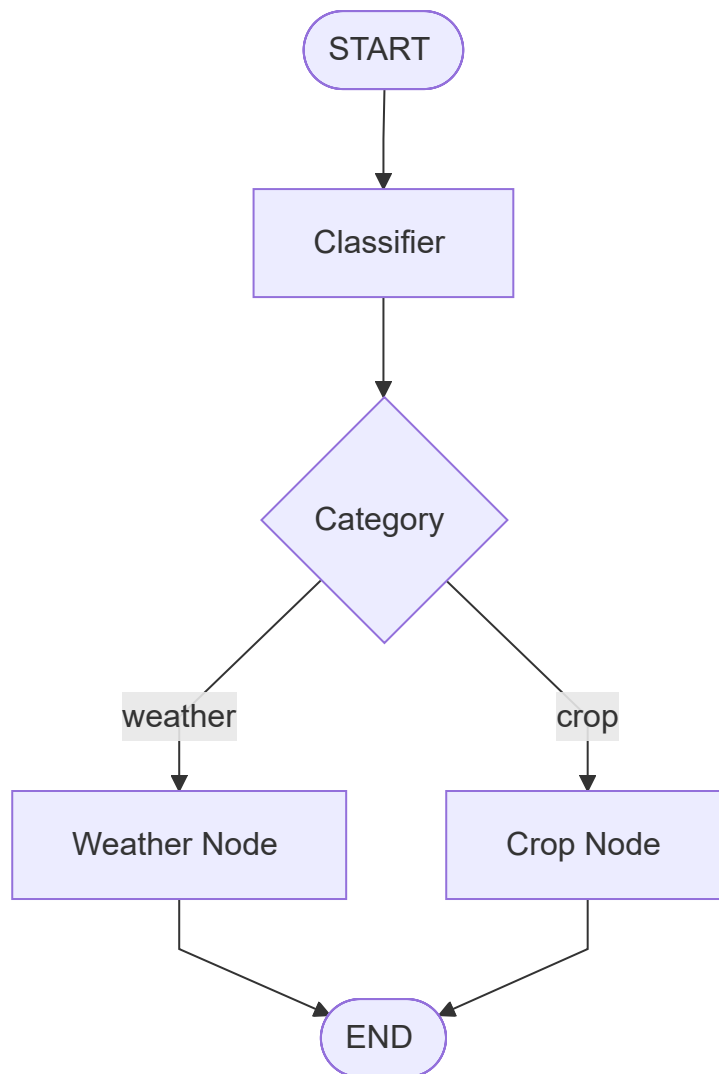
We can define:

```
def route_query(state):  
  
    if state["category"] == "weather":  
  
        return "weather"  
  
    return "crop"
```

The router decides:

which branch should execute?

30. Conditional Edge Architecture



31. Adding Conditional Edges

Conceptually:

```
builder.add_conditional_edges(  
    "classifier",  
    route_query,  
    {  
        "weather":  
            "weather_node",  
  
        "crop":  
            "crop_node"  
    }  
)
```

Meaning:

```
Run:  
route_query(state)  
  
If result = "weather"  
→ weather_node  
  
If result = "crop"  
→ crop_node
```

32. Complete Conditional Example

```
from typing import TypedDict  
  
from langgraph.graph import (  
    StateGraph,  
    START,  
    END  
)  
  
class AgentState(TypedDict):
```

query: str

category: str

response: str

```
def classifier(state):
```

```
    query = state["query"].lower()
```

```
    if "weather" in query:
```

```
        category = "weather"
```

```
    else:
```

```
        category = "crop"
```

```
    return {
```

```
        "category":
```

```
        category
```

```
    }
```

```
def route_query(state):
```

```
    return state["category"]
```

```
def weather_node(state):
```

```
    return {
```

```
        "response":
```

```
        "Weather request detected."
```

```
    }
```

```
def crop_node(state):
```



```
return {  
    "response":  
        "Crop request detected."  
}
```

33. Build Conditional Graph

```
builder = StateGraph(  
    AgentState  
)  
  
builder.add_node(  
    "classifier",  
    classifier  
)  
  
builder.add_node(  
    "weather",  
    weather_node  
)  
  
builder.add_node(  
    "crop",  
    crop_node  
)  
  
builder.add_edge(  
    START,  
    "classifier"  
)  
  
builder.add_conditional_edges(  
    "classifier",  
    lambda state: state["response"],  
    {  
        "Crop request detected.": "crop",  
        "Weather request detected.": "weather",  
        "Other request detected.": "end"  
    })
```

```
        "classifier",
        route_query,
        {
            "weather":
                "weather",

            "crop":
                "crop"
        }
    )

    builder.add_edge(
        "weather",
        END
    )

    builder.add_edge(
        "crop",
        END
    )

    graph = builder.compile()
```

34. Conditional Graph Execution

Input:

```
{  
  "query":  
    "What is today's weather?",  
  
  "category":  
    "",  
  
  "response":  
    ""  
}
```

Flow:

```
START  
↓  
Classifier  
↓  
category = weather  
↓  
Router  
↓  
Weather Node  
↓  
END
```

Another query:

```
Tell me about wheat cultivation.
```

Flow:

```
START
↓
Classifier
↓
category = crop
↓
Router
↓
Crop Node
↓
END
```

35. LLM-Based Routing

Our previous classifier used:

```
if "weather" in query:
```

But an LLM can also classify the request.

Conceptually:

```
def classifier(state):

    query = state["query"]

    result = llm.invoke(
        f"""
        Classify this request as:

        weather
        crop

        Request:
        {query}
```

```
        ""
    )

    return {
        "category":
            result.content
    }
```

Now routing becomes semantic rather than keyword-based.

36. Prefer Structured Routing Output



Do not rely unnecessarily on unpredictable text such as:

```
"I think this is probably a weather request."
```

Prefer structured output such as:

```
{
  "category": "weather"
}
```

Then:

```
category
↓
Conditional Edge
↓
Correct Node
```

This makes routing more reliable.

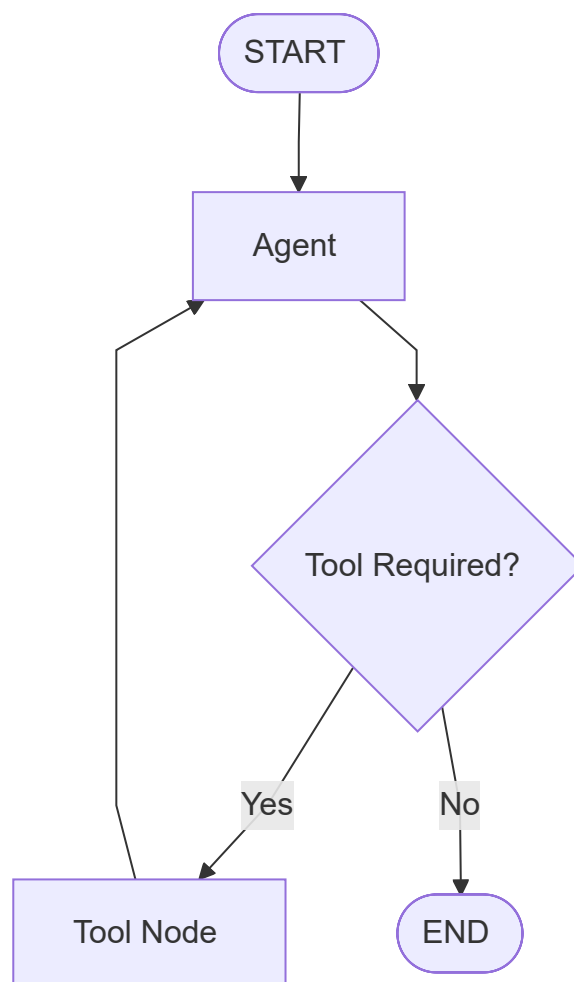
37. Loops in LangGraph



Graphs do not need to move only forward.

A node can route back to an earlier node.

Example agent:



Notice:

Tool
↓
Agent

This creates a:

Cycle / Loop

38. Why Loops Matter

Agent execution naturally behaves like:

Reason
↓
Act
↓
Observe
↓
Reason Again

Graph:

Agent



Tool



Agent



Tool



Agent



END

This is one of the reasons LangGraph is suitable for agents.

39. Traditional DAG vs LangGraph

Many workflow systems use a:

DAG

Directed Acyclic Graph.

Meaning:

No cycles.

Example:

A
↓
B
↓
C

Agents often need:

A
↓
B
↓
A

LangGraph supports cyclic workflows, which are useful for:

Tool loops

Retries

Reflection

Replanning

40. Agent Node

A common agent graph contains an:

Agent Node

Its responsibility may be:

Read messages/state

Call LLM

Generate response or tool call

Update messages

Conceptually:

```
def agent_node(state):  
  
    response = (  
        llm_with_tools.invoke(  
            state["messages"]  
        )  
    )  
  
    return {  
        "messages":  
            [response]  
    }
```

41. Tool Node

A:

Tool Node

executes requested tools and adds tool results to the state.

Conceptually:

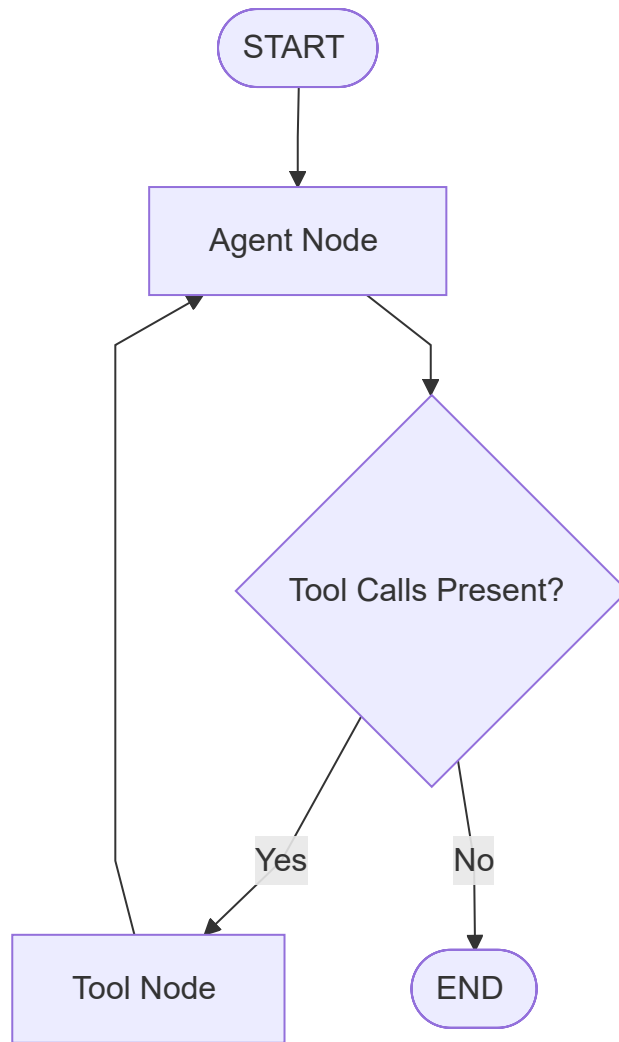
```
Agent Node
↓
Tool Call
↓
Tool Node
↓
Execute Function
↓
Tool Result
↓
State
```

LangGraph provides utilities that can simplify this pattern.

We will implement the complete tool-calling graph in the next part.

42. Agent + Tool Graph

The architecture we are moving toward is:



This replaces our manual:

```
while True:
```

with an explicit graph cycle.

43. State Reducers



There is one important state-management problem.

Suppose:

```
state = {  
  "messages": [  
    message1  
  ]  
}
```

Node returns:

```
{  
  "messages": [  
    message2  
  ]  
}
```

Should LangGraph:

```
Replace message1?
```

or:

```
Append message2?
```

This is where:

Reducers

become important.

44. What is a Reducer?

A reducer defines how new state updates should combine with existing state values.

Conceptually:

```
Old value
+
New value
↓
Reducer
↓
Combined value
```

For messages:

```
Existing Messages
+
New Messages
↓
Message Reducer
↓
Updated Message History
```

45. Simple Reducer Concept

Suppose:

```
old = [  
  "Hello"  
]  
  
new = [  
  "Hi"  
]
```

Replacement:

```
[  
  "Hi"  
]
```

Append reducer:

```
[  
  "Hello",  
  "Hi"  
]
```

For conversation state, appending/merging messages is often what we want.

46. MessagesState



LangGraph provides a convenient state abstraction for message-based applications:

```
MessagesState
```

Conceptually:

```
from langgraph.graph import (  
    MessagesState  
)
```

It provides a message-oriented state structure with appropriate message handling.

Example:

```
class AgentState(  
    MessagesState  
):  
    pass
```

For many simple agents, this can reduce manual state setup.

47. Custom State + Messages

Sometimes we need more than messages.

Example:

```
Messages  
  
User ID  
  
Current Plan  
  
Retrieved Documents  
  
Current Step
```

Then we can define custom state.

Conceptually:

```
class AgentState(TypedDict):  
  
    messages: list  
  
    user_id: str  
  
    current_step: int  
  
    retrieved_docs: list
```

The exact state design should match the workflow.

48. Keep State Minimal



Do not create:

```
state = {  
    "everything_in_the_application":  
        ...  
}
```

State should contain information required by the workflow.

Good:

Messages

Current Plan

Tool Results

Approval Status

Avoid unnecessary:

Huge unrelated objects

Entire database records

Secrets

API keys

49. Never Put Secrets in Agent State

Do not store:

OPENAI_API_KEY

Database passwords

JWT signing secret

Cloud credentials

inside agent state.

Tools should access secrets securely through:

Environment Variables

Secret Managers

Backend Configuration

The LLM should not need to see them.

50. Graph Compilation Mental Model

The complete process is:

1. Define State
2. Create StateGraph
3. Add Nodes
4. Add Edges
5. Add Conditional Edges
6. Compile Graph
7. Invoke Graph

Implementation pattern:

```
builder = StateGraph(  
    AgentState  
)  
  
builder.add_node(  
    ...  
)
```

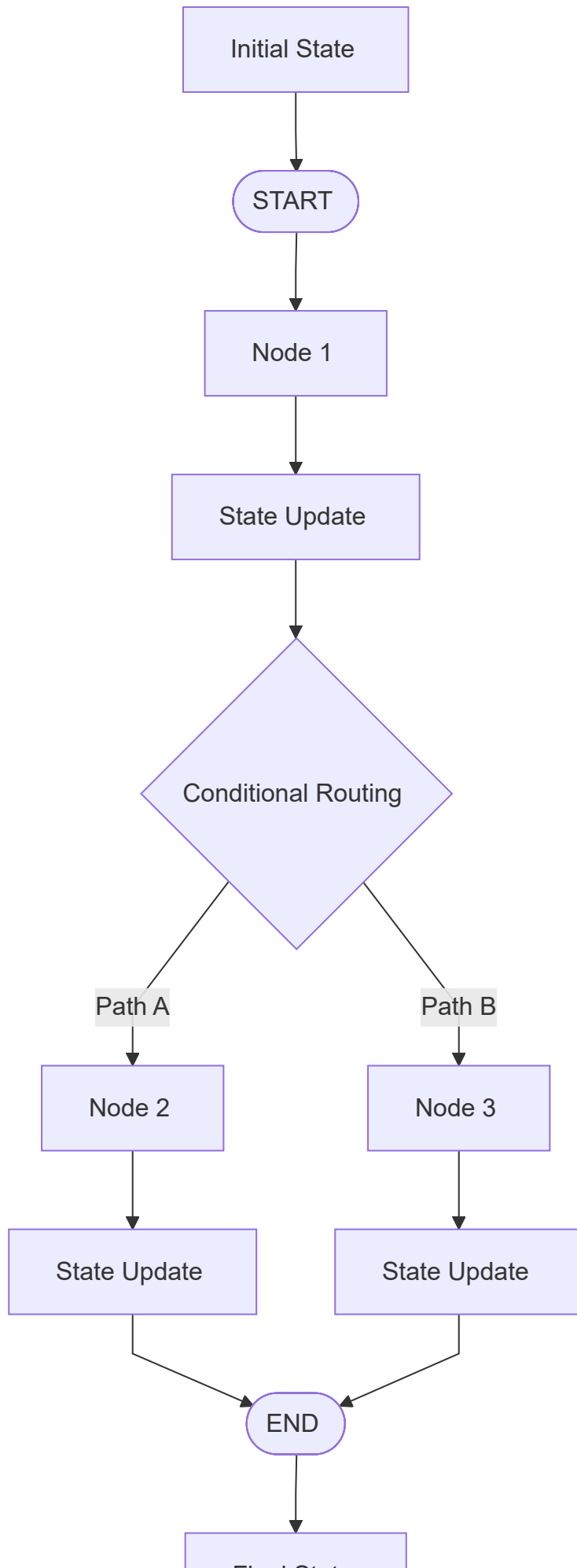
```
builder.add_edge(  
    ...  
)
```

```
builder.add_conditional_edges(  
    ...  
)
```

```
graph = builder.compile()
```

```
result = graph.invoke(  
    initial_state  
)
```

51. Complete LangGraph Mental Model



Remember:

Nodes

→ Do work

Edges

→ Define flow

Conditional Edges

→ Make routing decisions

State

→ Carries information

Reducers

→ Define how updates combine

Compile

→ Creates executable workflow

52. LangGraph and Our Previous Chapters

Everything we studied now connects.

Tool Calling

Tool Node

State

Graph State

Planning

Planner Node

Reflection

Evaluator Node

Replanning

Conditional Edge
→ Planner Again

Human Approval

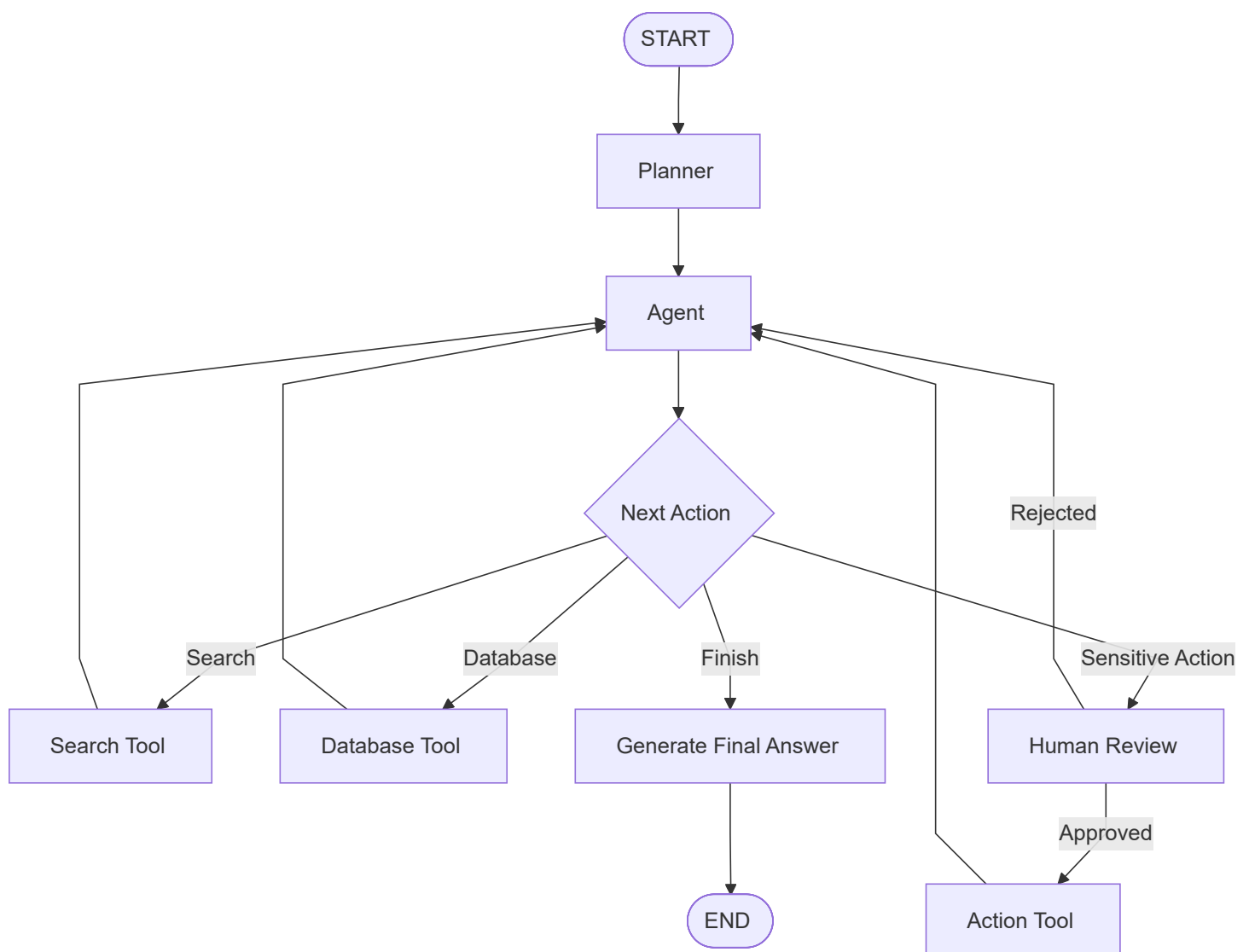
Interrupt / Human Review Step

Memory

Memory retrieval can provide relevant information to graph execution.

LangGraph gives us a structured way to orchestrate these pieces.

53. Example Complex Graph



Instead of hiding this logic inside dozens of:

```
if
elif
while
try
except
```

the workflow becomes explicit.

54. When Should You Use LangGraph?



LangGraph becomes useful when you need:

- Multi-step agent workflows
- Stateful execution
- Loops
- Conditional routing
- Tool-calling agents
- Persistence
- Human-in-the-loop
- Retry/recovery flows
- Complex orchestration

55. When You Probably Don't Need LangGraph

For:

```
Simple LLM call  
  
Basic chatbot  
  
Single RAG request  
  
One deterministic API sequence
```

LangGraph may add unnecessary complexity.

Example:

```
User  
↓  
Retriever  
↓  
LLM  
↓  
Answer
```

If this simple pipeline solves the problem, you may not need a graph framework.

56. Common Mistakes

Mistake 1 – Thinking LangGraph is an LLM

It is not.

LLM

→ Intelligence/model

LangGraph

→ Workflow orchestration

Mistake 2 – Thinking LangGraph Replaces LangChain

They can be used together.

Mistake 3 – Putting Everything in One Node

Bad:

One Node

→ Classification

→ Search

→ Tool Calling

→ Validation

→ Final Response

Then the graph provides little benefit.

Prefer meaningful responsibilities.

Mistake 4 – Creating Too Many Tiny Nodes

Do not create a node for every line of code.

Nodes should represent meaningful workflow steps.

Mistake 5 – Forgetting State Design

Poor state design makes the entire graph harder to maintain.

Mistake 6 – Confusing Edge and Conditional Edge

Edge

→ Fixed next node

Conditional Edge

→ Dynamic next node

Mistake 7 – Assuming Graphs Cannot Loop

Agent graphs often intentionally contain cycles.

57. Important Interview Questions

Q1. What is LangGraph? ★ ★ ★ ★ ★

Answer:

LangGraph is a framework for building stateful, multi-step LLM applications and agents using graph-based workflows consisting of state, nodes, edges and conditional routing.

Q2. Why use LangGraph? ★ ★ ★ ★ ★

Answer:

LangGraph helps explicitly orchestrate complex agent workflows involving state, branching, loops, tool calls, persistence and human-in-the-loop interactions that can become difficult to manage with manual loops and conditional statements.

Q3. What is state in LangGraph?



Answer:

State is the shared data that flows through the graph and is read or updated by nodes during workflow execution.

Q4. What is a node? ★ ★ ★ ★ ★

Answer:

A node is a function or processing step in a LangGraph workflow that reads the current state, performs some work and returns state updates.

Q5. What is an edge? ★ ★ ★ ★ ★

Answer:

An edge defines the execution flow from one node to another.

Q6. Normal edge vs conditional edge?



Answer:

A normal edge always routes execution to a fixed next node, while a conditional edge dynamically selects the next node based on state or routing logic.

Q7. What are START and END?

Answer:

START represents the graph's entry point, while END represents completion of graph execution.

Q8. What does `compile()` do?

Answer:

It converts the graph definition containing state, nodes and edges into an executable graph that can be invoked.

Q9. Can LangGraph contain loops?



Answer:

Yes. LangGraph supports cyclic workflows, which are especially useful for agent patterns where execution repeatedly moves between an LLM and tools until the task is complete.

Q10. LangChain vs LangGraph?

Answer:

LangChain provides components and abstractions such as models, tools, prompts and retrievers, while LangGraph focuses on orchestrating these components in stateful workflows involving branching, loops and persistence.

Q11. What is a reducer in LangGraph?



Answer:

A reducer defines how a new state update should be combined with an existing state value, such as appending new messages instead of replacing the existing message history.

Q12. What is `MessagesState` ?

Answer:

`MessagesState` is a convenient LangGraph state abstraction for message-based applications that provides a `messages` field with suitable message update behavior.

Q13. Why shouldn't every application use LangGraph?

Answer:

Simple LLM calls, straightforward RAG pipelines or deterministic workflows may not need graph orchestration. LangGraph is most useful when stateful branching, loops or complex workflow control are required.

58. Final Mental Model

The five things to remember:

STATE

→ what information is flowing?

NODE

→ what work should happen?

EDGE

→ where should execution go next?

CONDITIONAL EDGE

→ which path should be selected?

REDUCER

→ How should state updates combine?

Basic LangGraph:

START

↓

Node

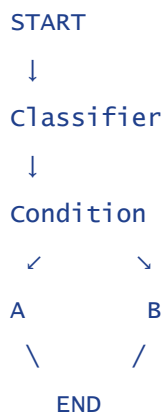
↓

Node

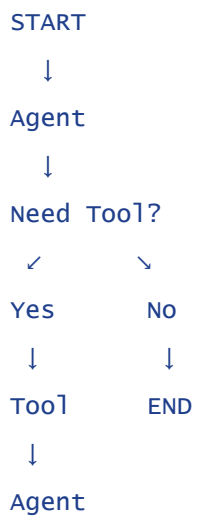
↓

END

Conditional LangGraph:



Agent LangGraph:



And the code pattern:

```
# 1. Define state

class AgentState(TypedDict):
    ...

# 2. Create graph

builder = StateGraph(
    AgentState
)
```

3. Add nodes

```
builder.add_node(  
    "node_name",  
    node_function  
)
```

4. Add edges

```
builder.add_edge(  
    START,  
    "node_name"  
)
```

5. Add conditional routing if needed

```
builder.add_conditional_edges(  
    ...  
)
```

6. End workflow

```
builder.add_edge(  
    "node_name",  
    END  
)
```

7. Compile

```
graph = builder.compile()
```

8. Execute

```
result = graph.invoke(  
    initial_state
```

)

The transition from our previous manual agent to LangGraph is simply:

```
Manual Agent  
  
while loop  
+  
state dictionary  
+  
if/else routing  
+  
tool execution
```

becoming:

```
LangGraph Agent  
  
StateGraph  
+  
Nodes  
+  
Edges  
+  
Conditional Edges  
+  
Cycles
```

LangGraph does not provide the intelligence of the agent. The LLM provides the reasoning, tools provide external capabilities, state stores workflow information, and LangGraph orchestrates how these components interact over time.

Part 2 – Building a Complete Tool-Calling Agent with LangGraph

1. What Are We Building?



In Part 1, we learned the core LangGraph components:

- State
- Nodes
- Edges
- Conditional Edges
- Reducers
- START / END

Now we will combine them with the tool-calling concepts from Chapter 2.

We will build:

- User
 - ↓
- LangGraph
 - ↓
- Agent Node
 - ↓

LLM



Need Tool?



Yes

No



Tool

END

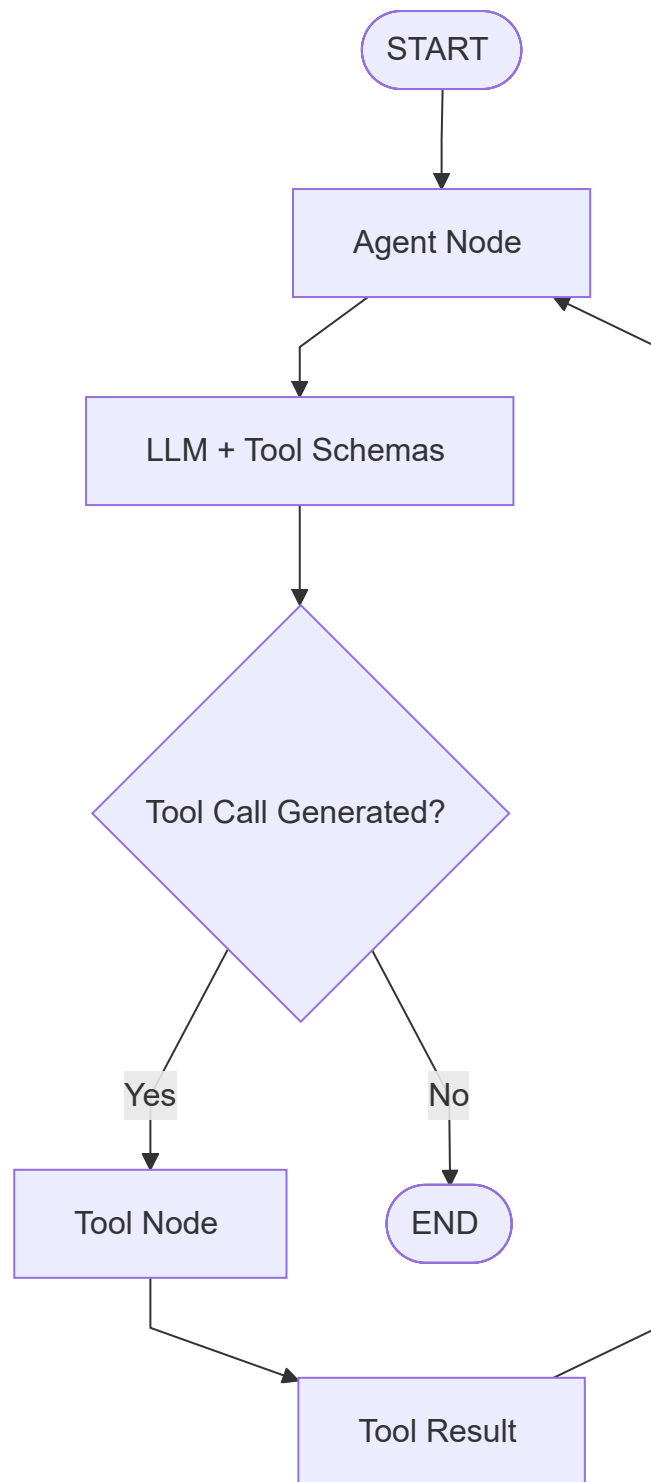


Agent

This is one of the most important LangGraph patterns.

2. Architecture of a Tool-Calling Agent





The important loop is:

Agent
↓
Tool
↓
Agent

It continues until the LLM decides that no more tools are required.

3. Components We Need

Our implementation requires:

1. LLM
2. Tools
3. Tool-aware LLM
4. State
5. Agent Node
6. Tool Node
7. Conditional Routing
8. Graph
9. Invocation

4. Installation

A typical Python setup may require packages such as:

```
pip install langgraph langchain
```

and the integration package for your chosen model provider.

For example, when using a provider integration:

```
pip install langchain-openai
```

or another supported provider package.

The exact package setup can change with library versions.

5. Imports

A typical implementation may start with:

```
from langgraph.graph import (
    StateGraph,
    START,
    END,
    MessagesState
)

from langgraph.prebuilt import (
    ToolNode
)

from langchain_core.tools import tool
```

We will understand each component rather than memorizing imports.

6. Step 1 – Create Tools

Let's create two simple tools.

```
@tool
def get_weather(city: str) -> str:
    """Get weather information for a city."""

    weather_data = {
        "delhi": "31°C and rainy",
        "mumbai": "29°C and cloudy"
    }

    return weather_data.get(
        city.lower(),
        "Weather information unavailable."
    )
```

Calculator:

```
@tool
def multiply(
    a: float,
    b: float
) -> float:
    """Multiply two numbers."""

    return a * b
```

7. Tool List

Now create:

```
tools = [
    get_weather,
    multiply
]
```

Conceptually:

```
Agent
  ↓
Available Tools
  ├── get_weather
  └── multiply
```

The model can choose between them based on the user request.

8. Step 2 – Create the LLM

The exact initialization depends on the model provider.

Conceptually:

```
llm = ChatModel(...)
```

For example, with a LangChain-compatible chat model:

```
llm = ChatOpenAI(
    model="your-model"
)
```

The important concept is simply:

```
LLM Instance
```

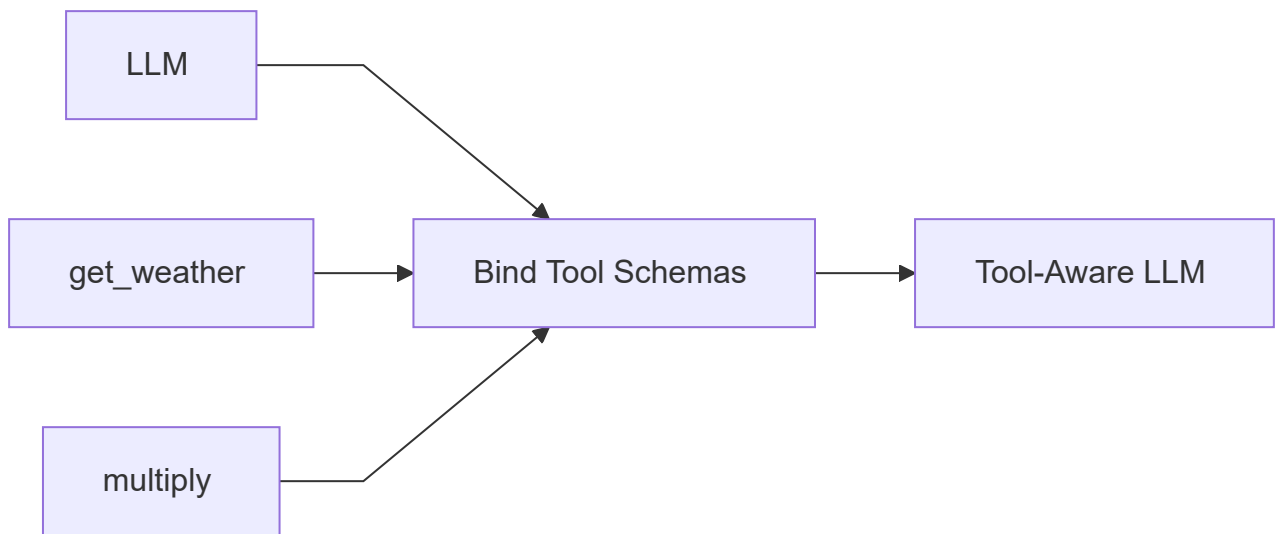
9. Step 3 – Bind Tools



Now make the LLM aware of the available tools:

```
llm_with_tools = (  
    llm.bind_tools(tools)  
)
```

Architecture:



Remember:

```
bind_tools()
```

does **not** execute tools.

It gives the model information about the available tools.

10. What Can the LLM Return?

Suppose the user asks:

```
what is the weather in Delhi?
```

The model may produce a tool call conceptually like:

```
{  
  "name": "get_weather",  
  "args": {  
    "city": "Delhi"  
  }  
}
```

But:

```
Tool Selection  
≠  
Tool Execution
```

Something still needs to execute the function.

That is where the Tool Node comes in.

11. Step 4 – Define State

For a conversational tool-calling agent, we mainly need:

```
messages
```

Therefore we can use:

```
MessagesState
```

Conceptually:

```
class AgentState(  
    MessagesState  
):  
    pass
```

Or use `MessagesState` directly when no extra fields are needed.

12. What Does Message State Contain?

During execution:

```
HumanMessage  
↓  
AIMessage containing tool call  
↓  
ToolMessage containing tool result  
↓  
AIMessage containing final response
```

Example:

Human:

What is the weather in Delhi?

AI:

```
get_weather(city="Delhi")
```

Tool:

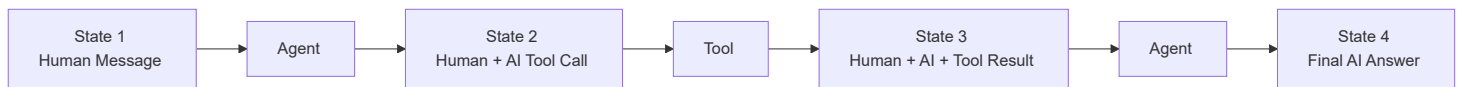
31°C and rainy

AI:

The weather in Delhi is 31°C and rainy.

All of this forms the agent's message state.

13. State Evolution



This is why message reducers are important.

We want new messages to be added rather than losing the previous history.

14. Step 5 – Create Agent Node



The Agent Node calls the LLM.


```
def agent_node(state):  
  
    response = (  
        llm_with_tools.invoke(  
            state["messages"]  
        )  
    )  
  
    return {  
        "messages": [  
            response  
        ]  
    }
```

This is surprisingly small.

15. What Does Agent Node Actually Do?

```
Current State  
↓  
Extract Messages  
↓  
Send Messages to LLM  
↓  
LLM Makes Decision  
↓  
Return AI Message  
↓  
Update State
```

The AI message may contain:

Final Text

or:

Tool Call

16. Agent Node Does Not Execute Tools

Important distinction:

```
response = llm_with_tools.invoke(...)
```

may produce:

```
get_weather("Delhi")
```

but the actual Python function has still not run.

So:

Agent Node

→ Decides

Tool Node

→ Executes

17. Step 6 – Create Tool Node



LangGraph provides:

```
ToolNode
```

We can create one using:

```
tool_node = ToolNode(  
    tools  
)
```

Conceptually:

```
AI Tool Call  
↓  
ToolNode  
↓  
Find Requested Tool  
↓  
Execute Tool  
↓  
Create ToolMessage  
↓  
Update State
```

18. Why ToolNode is Useful

Without `ToolNode`, we would manually write:

```
for tool_call in response.tool_calls:

    tool_name = tool_call["name"]

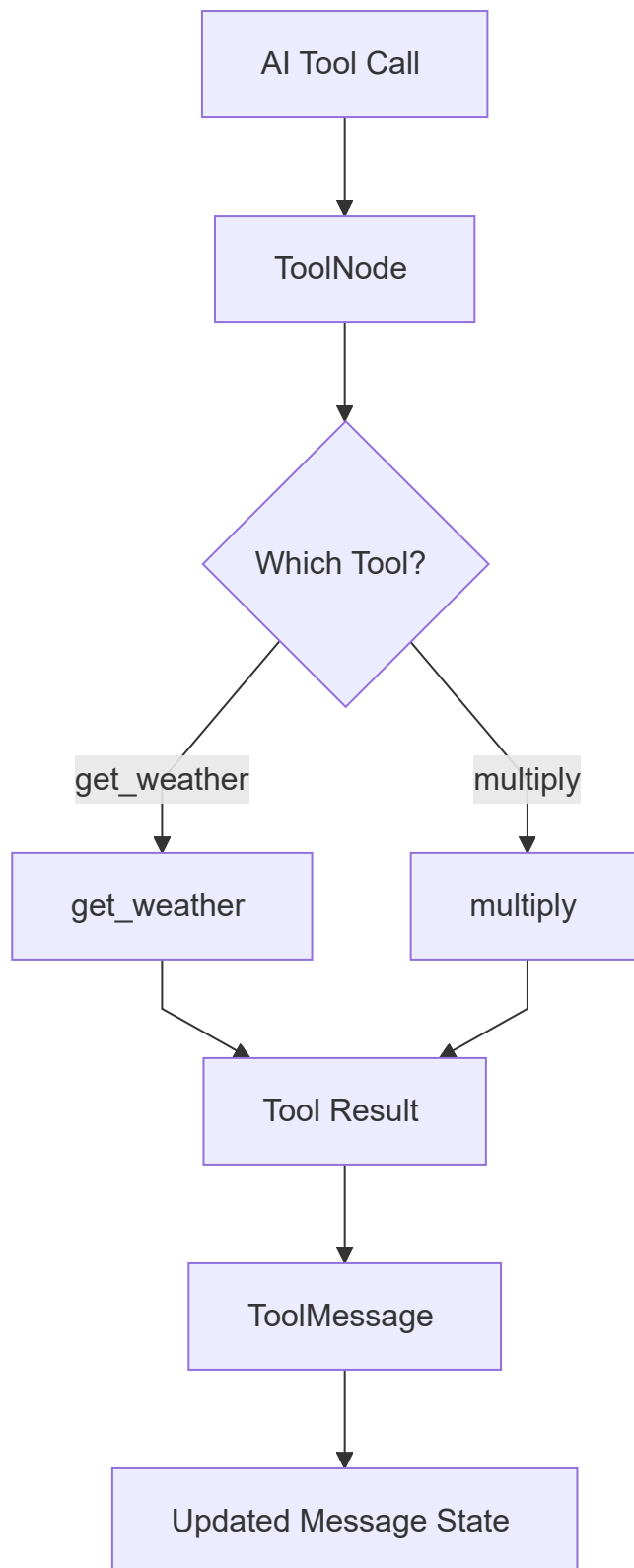
    tool = tools_by_name[
        tool_name
    ]

    result = tool.invoke(
        tool_call["args"]
    )

    messages.append(
        ToolMessage(...)
    )
```

ToolNode *handles much of this common execution pattern.*

19. ToolNode Mental Model



20. Step 7 – Decide Whether to Call Tools

After the Agent Node executes, we need to inspect the latest AI message.

Question:

Does this AI message contain tool calls?

If:

YES
→ Tool Node

If:

NO
→ END

This requires:

Conditional Routing

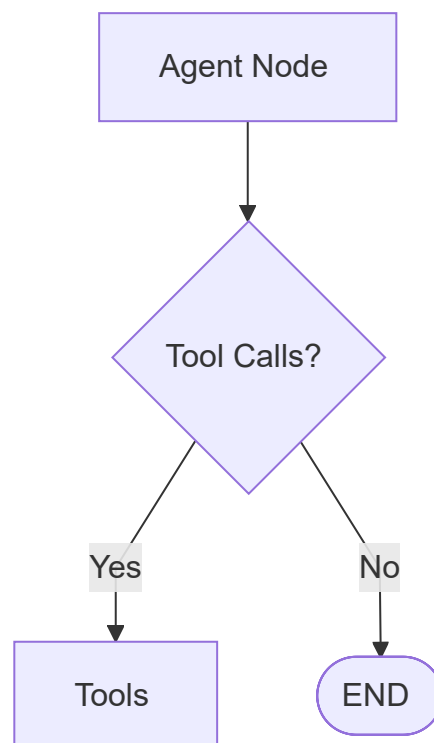
21. Router Function

We can manually create:

```
def should_continue(state):  
  
    last_message = (  
        state["messages"][-1]  
    )  
  
    if last_message.tool_calls:  
  
        return "tools"  
  
    return END
```

The function examines the latest message.

22. Router Architecture



The router itself does not execute the tool.

It only determines:

where should execution go next?

23. Prebuilt Tool Routing

LangGraph also provides utilities for common tool-routing patterns.

You may encounter:

```
tools_condition
```

Conceptually it performs:

```
graph TD
    A[Latest AI Message] --> B{Contains Tool Calls?}
    B -- ✓ --> C[Yes]
    B -- ✗ --> D[No]
    C --> E[Tools]
    D --> F[END]
```

So we do not always need to write the routing function manually.

24. Manual vs Prebuilt Routing

Manual:


```
def should_continue(state):  
  
    last_message = (  
        state["messages"][-1]  
    )  
  
    if last_message.tool_calls:  
        return "tools"  
  
    return END
```

Prebuilt:

```
tools_condition
```

Both represent the same basic idea.

For learning, understand the manual logic first.

25. Step 8 – Create Graph

```
builder = StateGraph(  
    MessagesState  
)
```

Now add our two nodes:

```
builder.add_node(  
    "agent",  
    agent_node  
)  
  
builder.add_node(  
    "tools",  
    tool_node  
)
```

Graph currently contains:

```
Agent Node  
  
Tool Node
```

26. Step 9 – Add START Edge

Execution should begin with the agent:

```
builder.add_edge(  
    START,  
    "agent"  
)
```

Graph:

```
START  
↓  
Agent
```

27. Step 10 – Add Conditional Routing

```
builder.add_conditional_edges(  
    "agent",  
    should_continue,  
    {  
        "tools": "tools",  
        END: END  
    }  
)
```

Conceptually:

```
Agent  
↓  
should_continue()  
↓  
Tool Call?  
✓      ✗  
Yes    No  
↓      ↓  
Tools  END
```

28. Step 11 – Tool Back to Agent



After the tool executes:

```
Tool Result
```

must return to the LLM.

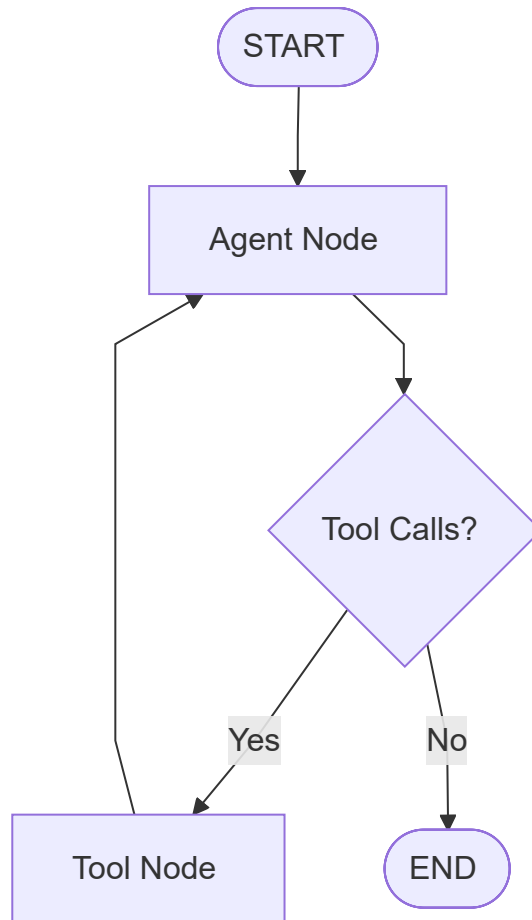
Therefore:

```
builder.add_edge(  
    "tools",  
    "agent"  
)
```

This creates the loop:

```
Agent  
  ↓  
Tools  
  ↓  
Agent
```

29. Complete Graph



This is the core architecture of a LangGraph tool-calling agent.

30. Step 12 – Compile

```
graph = builder.compile()
```

Now the graph becomes executable.

31. Complete Implementation



```
from langchain_core.tools import tool

from langgraph.graph import (
    StateGraph,
    START,
    END,
    MessagesState
)

from langgraph.prebuilt import (
    ToolNode
)

# -----
# Tools
# -----

@tool
def get_weather(city: str) -> str:
    """Get weather information for a city."""

    data = {
        "delhi": "31°C and rainy",
        "mumbai": "29°C and cloudy"
    }

    return data.get(
        city.lower(),
        "Weather information unavailable."
    )

@tool
```

```
def multiply(  
    a: float,  
    b: float  
    ) -> float:  
    """Multiply two numbers."""  
  
    return a * b
```

```
tools = [  
    get_weather,  
    multiply  
]
```

```
# -----  
# LLM  
# -----
```

```
llm_with_tools = (  
    llm.bind_tools(tools)  
)
```

```
# -----  
# Agent Node  
# -----
```

```
def agent_node(state):  
  
    response = (  
        llm_with_tools.invoke(  
            state["messages"]  
        )  
    )  
  
    return {  
        "messages": [  
            response  
        ]  
    }
```

```
# -----  
# Tool Node  
# -----
```

```
tool_node = ToolNode(  
    tools  
)
```

```
# -----  
# Routing  
# -----
```

```
def should_continue(state):  
  
    last_message = (  
        state["messages"][-1]  
    )  
  
    if last_message.tool_calls:  
  
        return "tools"  
  
    return END
```

```
# -----  
# Graph  
# -----
```

```
builder = StateGraph(  
    MessagesState  
)
```

```
builder.add_node(  
    "agent",  
    agent_node  
)
```



```
builder.add_node(  
    "tools",  
    tool_node  
)
```

```
builder.add_edge(  
    START,  
    "agent"  
)
```

```
builder.add_conditional_edges(  
    "agent",  
    should_continue,  
    {  
        "tools": "tools",  
        END: END  
    }  
)
```

```
builder.add_edge(  
    "tools",  
    "agent"  
)
```

```
graph = builder.compile()
```

32. Invoke the Agent

```
result = graph.invoke(  
    {  
        "messages": [  
            {  
                "role": "user",  
                "content":  
                    "What is the weather in Delhi?"  
            }  
        ]  
    }  
)
```

Conceptually:

START

↓ Agent

LLM:

Need weather.

↓ Tool Call

```
get_weather(  
    city="Delhi"  
)
```

↓ ToolNode

31°C and rainy

↓ Agent

LLM:

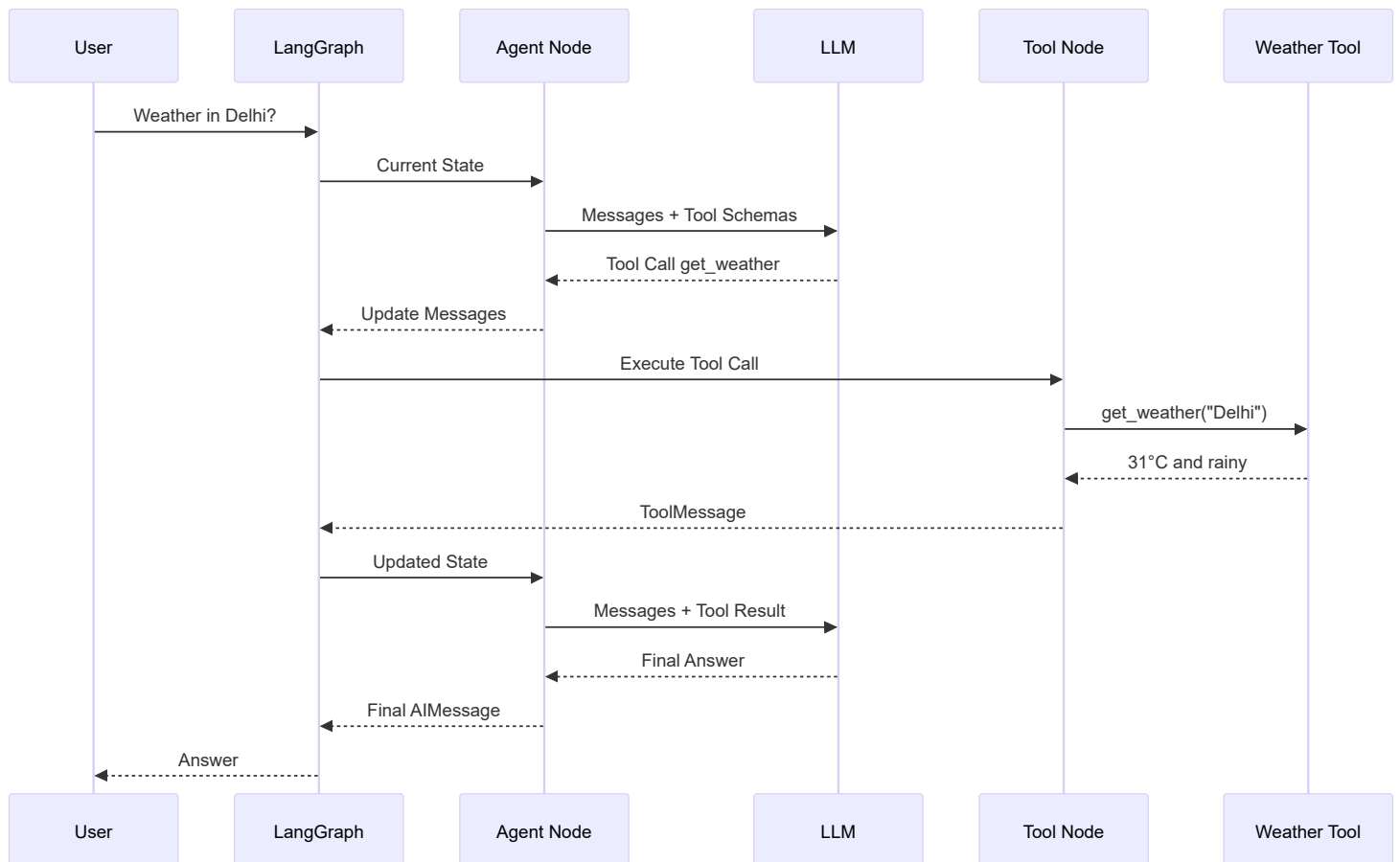
The weather in Delhi is

31°C and rainy.

↓ No Tool Call

END

33. Execution Sequence



34. Multiple Tool Calls

Suppose the user asks:

What is the weather in Delhi and Mumbai?

The LLM may request:

```
get_weather("Delhi")  
  
get_weather("Mumbai")
```

The Tool Node can process the requested tool calls and return their observations.

Then the graph returns to:

Agent Node

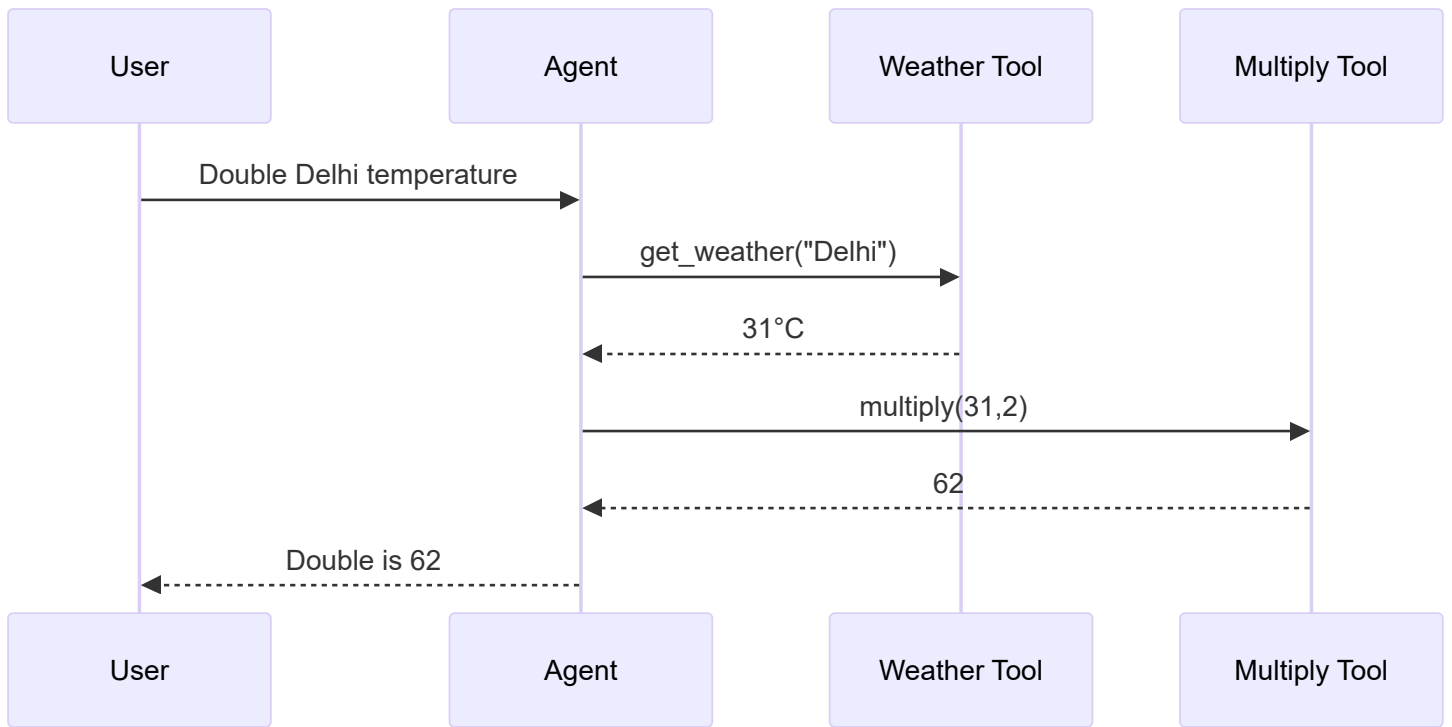
where the model combines the results.

35. Sequential Multi-Tool Reasoning

Now consider:

```
Double the current temperature in Delhi.
```

Possible flow:



Notice that the graph can loop multiple times:

```
Agent
↓
Weather Tool
↓
Agent
↓
Multiply Tool
↓
Agent
↓
END
```

36. Why the Agent Must Run Again After a Tool



A tool result is usually not the final user-facing response.

Example:

```
Tool Result:

{
  "temperature": 31,
  "condition": "rainy"
}
```

The LLM may need to:

```
Interpret it

Combine it with other observations

Decide whether another tool is needed

Generate a natural-language response
```

Therefore:

```
Tool
↓
Agent
```

is essential.

37. Tool Results Become State

After tool execution:

ToolMessage

is added to:

messages

Therefore the next Agent Node sees:

User Request

Previous AI Tool Call

Tool Result

and can continue reasoning.

38. Custom State

Our current state only contains:

messages

But real applications may need more.

Example:

```
class AgentState(  
    MessagesState  
):  
  
    user_id: str  
  
    current_plan: list  
  
    retry_count: int
```

Then the graph state might contain:

```
{  
    "messages": [...],  
  
    "user_id":  
        "user_123",  
  
    "current_plan":  
        [...],  
  
    "retry_count":  
        0  
}
```

39. Custom State Example

Suppose we want to track tool calls:


```
class AgentState(  
    MessagesState  
):  
  
    tool_count: int
```

Then a node could update:

```
return {  
    "tool_count":  
        state["tool_count"] + 1  
}
```

This information can later be used for:

- Limits
- Logging
- Routing
- Monitoring

40. Limiting Agent Execution



Remember our manual agent:

```
MAX_STEPS = 10
```

Agent graphs should also avoid uncontrolled loops.

Possible protections include:

Recursion / step limits

Retry limits

Tool-call limits

Timeouts

Human escalation

Conceptually:

Agent

↓

Step Count

↓

Limit Reached?

↙

No

↓

Continue

↘

Yes

↓

Stop

41. Why Infinite Loops Can Happen

Example:

Agent:

Call search.

Tool:

No result.

Agent:

```
Call search.
```

```
Tool:
```

```
No result.
```

```
Agent:
```

```
Call search.
```

```
...
```

Or:

```
Agent A
```

```
↓
```

```
Agent B
```

```
↓
```

```
Agent A
```

```
↓
```

```
Agent B
```

```
...
```

Therefore production workflows need termination conditions.

42. Tool Error Handling

Suppose:

```
get_weather("UnknownCity")
```

fails.

The tool should preferably return or raise an error that the execution layer can handle appropriately.

Conceptually:

Tool Call



Execute



Error



Structured Tool Error



Agent



Decide Next Step

The agent may:

Try corrected arguments

Use another tool

Ask the user

Stop gracefully

43. Tool Errors Should Return to the Agent

Consider:

Search API unavailable.

If the graph simply crashes:

Workflow Ends Unexpectedly

But if appropriate error information reaches the agent:

Tool Error

↓

Agent

↓

Alternative Action

the workflow may recover.

Not every exception should be exposed directly to the model, especially if it contains sensitive internal information.

44. Agent Routing Based on State

Routing does not have to depend only on:

tool_calls

It can use any relevant state.

Example:

```
def route(state):  
  
    if state["retry_count"] >= 3:  
  
        return "fallback"  
  
    if state["messages"][-1].tool_calls:  
  
        return "tools"  
  
    return END
```

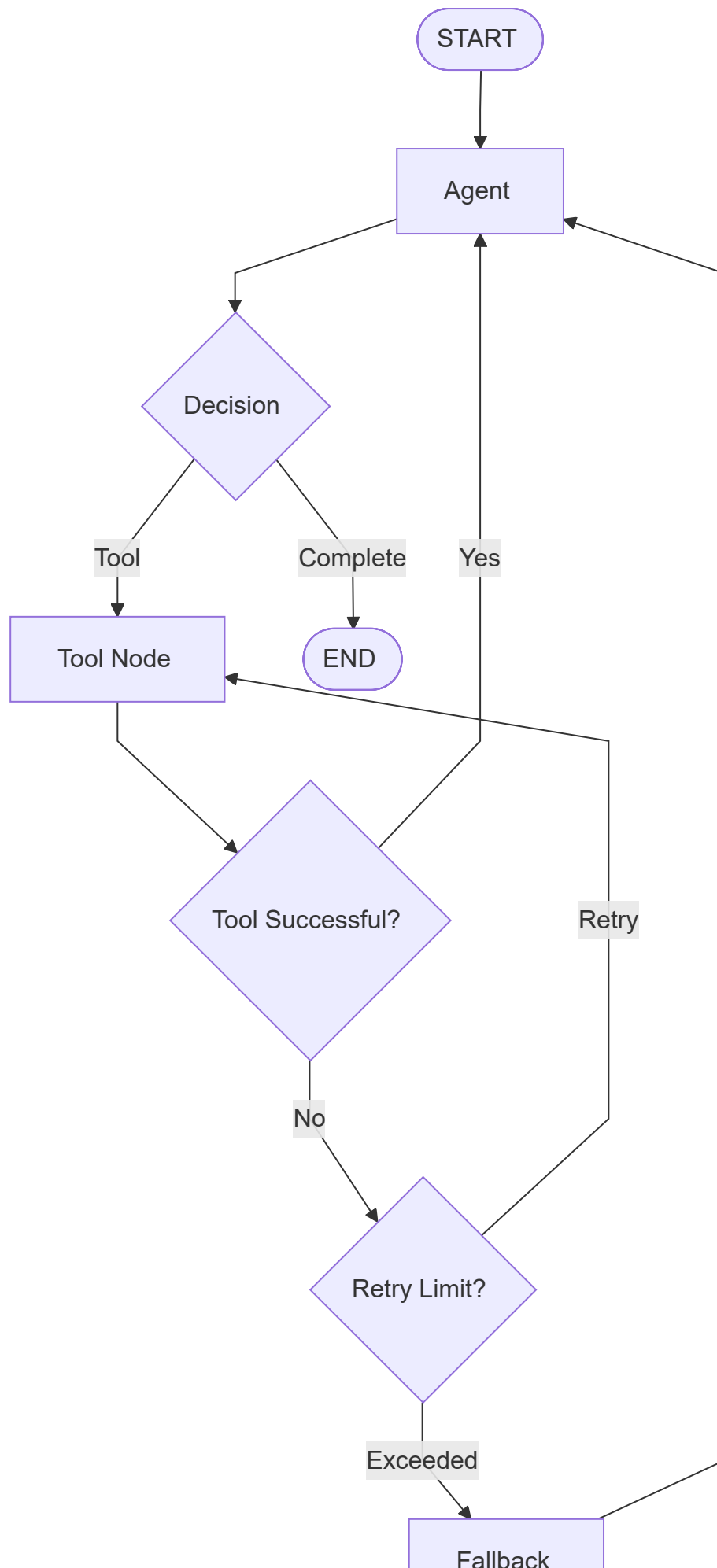
Now graph behavior depends on:

Tool Call

+

Retry Count

45. More Complex Agent Graph



This is where LangGraph becomes much more useful than a simple manual loop.

46. Adding a System Message

A real agent normally needs instructions.

Example:

```
SYSTEM_PROMPT = """
You are a helpful assistant.

Use tools when required.

Do not invent tool results.

If required information is missing,
ask the user.
"""
```

The Agent Node can include this instruction when calling the model.

Conceptually:

```
def agent_node(state):

    messages = [
        SystemMessage(
            content=SYSTEM_PROMPT
        )
    ] + state["messages"]

    response = (
        llm_with_tools.invoke(
```

```
        messages
    )
)

return {
    "messages": [response]
}
```

47. System Prompt vs Graph Logic



Do not put every rule only in the prompt.

Example:

```
"Never execute more than 5 tools."
```

Prompt-only enforcement is weaker.

Better:

```
System Prompt
→ Tell model expected behavior

Graph/Application Logic
→ Enforce actual limit
```

Similarly:

Prompt:

Do not perform unauthorized actions.

Backend:

Actually check authorization.

48. Deterministic Control Around LLM

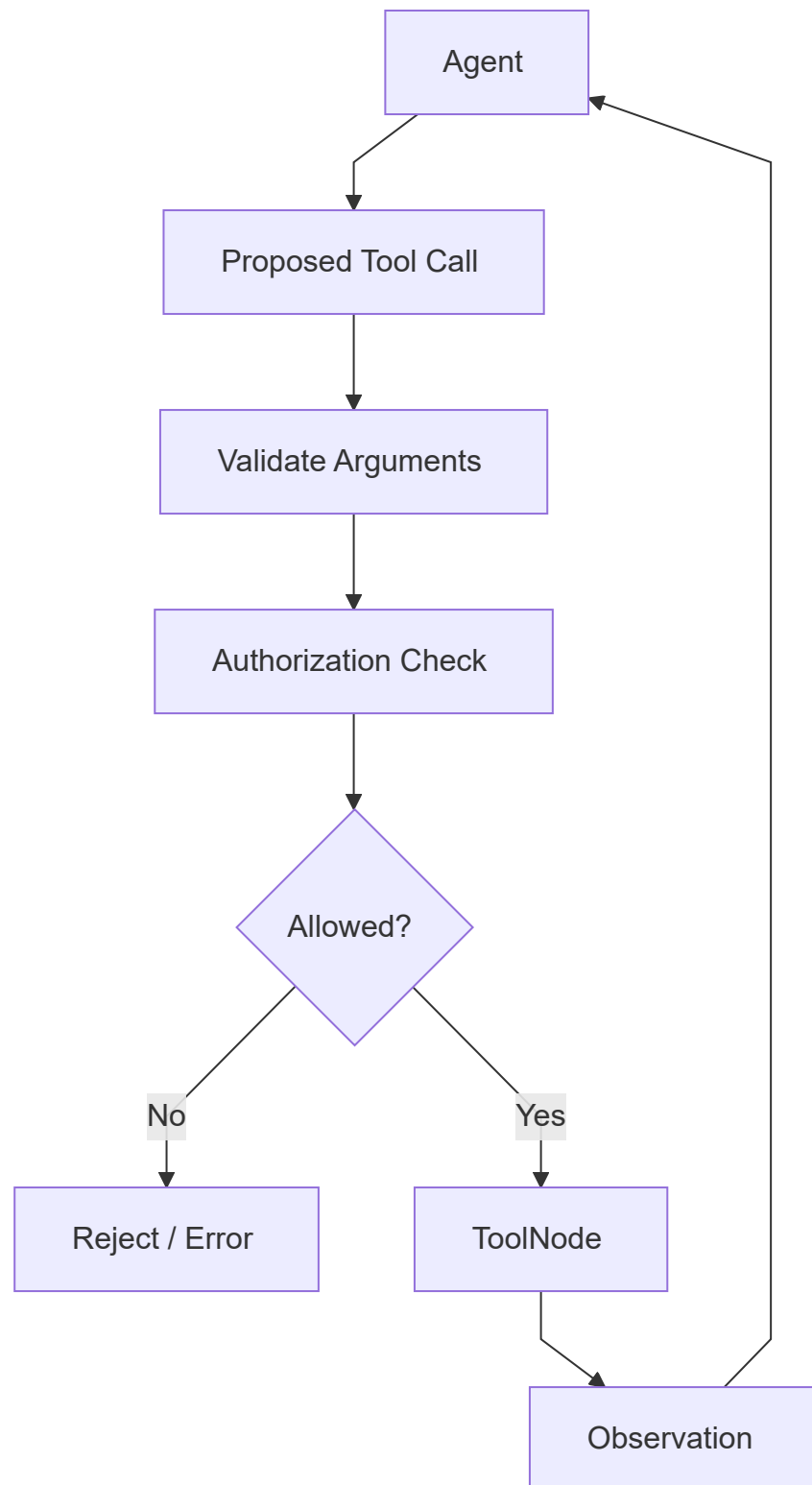
A strong architecture separates:

LLM Decisions

from:

Application Enforcement

Example:



The LLM proposes actions.

The application controls whether they are actually allowed.

49. Tool Calling Agent with Database Tools

A real agent could have:

```
get_weather  
  
search_crops  
  
get_market_price  
  
get_user_history
```

Then:

```
tools = [  
    get_weather,  
    search_crops,  
    get_market_price,  
    get_user_history  
]
```

The same graph architecture still works:

```
Agent  
↓  
Choose Any Tool  
↓  
ToolNode  
↓  
Agent
```

You do not need a separate graph architecture for every tool.

50. Example: Agriculture Agent

Suppose an agricultural assistant has:

Weather Tool

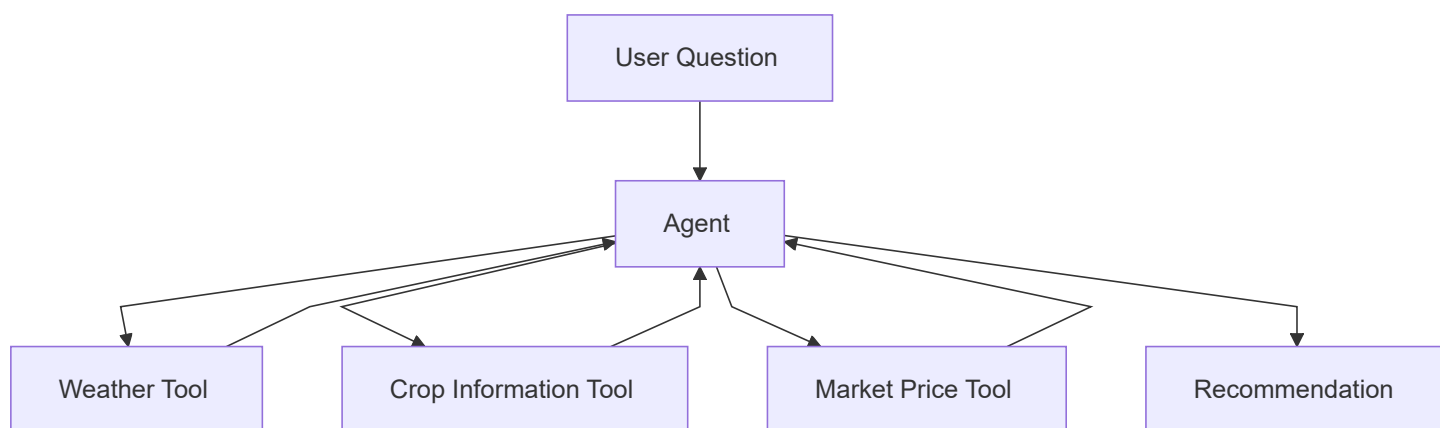
Crop Information Tool

Market Price Tool

User asks:

should I grow wheat in my district?

Possible execution:



The agent dynamically decides what information it needs.

51. Agent vs Hard-Coded Workflow

Suppose every crop recommendation always requires:

```
Weather
↓
Soil
↓
Market Price
↓
Prediction
```

Then a deterministic workflow may be better.

But if:

```
Different questions require different tools

Some tools are optional

Tool sequence changes

Agent must adapt based on observations
```

then an agentic graph becomes more useful.

52. Very Important Design Principle



Do not convert every backend API into an AI tool.

Ask:

Does the LLM need to decide
when this capability should be used?

If:

No

normal application code may be better.

Example:

Password hashing
JWT verification
Database connection
Input sanitization

should not become LLM tools.

53. Tools Should Represent Meaningful Capabilities

Good:


```
get_weather(city)

search_crop_information(query)

get_market_price(crop, district)
```

Poor tool design:

```
open_database_connection()

parse_json()

convert_string_to_lowercase()

hash_password()
```

The LLM should work with meaningful application capabilities.

54. Tool Granularity

Tools should not be:

```
Too broad
```

Example:

```
do_everything()
```

because the model has little control.

But also not:

Too tiny

such as:

`add_one_character()`

`lowercase_word()`

Prefer meaningful business-level operations.

55. Observability



Agent systems can be difficult to debug because execution paths change.

You should be able to inspect:

which node executed?

which tool was called?

what arguments were used?

what result came back?

How many iterations occurred?

where did failure happen?

Conceptually:

User Request



Trace ID



Agent



Tool



Agent



Final Answer

Each step should be observable.

56. Streaming

Agents may take multiple steps before completion.

Instead of waiting for the entire graph:

Start

...

...

...

Final Answer

applications may stream:

Intermediate events

Messages

Node updates

Final response

LangGraph supports streaming patterns.

Conceptually:

```
for event in graph.stream(  
    initial_state  
):  
  
    print(event)
```

The exact event structure depends on the streaming mode and library version.

57. Why Streaming Helps

Without streaming:

```
Agent takes 10 seconds  
↓  
User sees nothing
```

With streaming:

Processing request...

Checking information...

Generating response...

or token-level output can improve perceived responsiveness.

Be careful not to expose hidden reasoning or sensitive internal information.

58. Persistence



So far:

```
graph.invoke(...)
```

runs the workflow.

But suppose we need:

Conversation continues later

Application restarts

Human approval pauses workflow

Agent resumes later

Then we need:

Persistence / Checkpointing

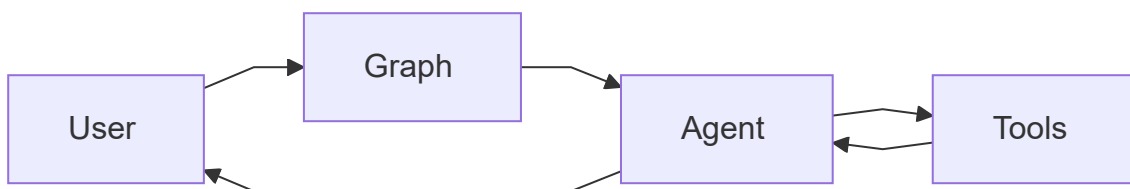
Conceptually:

```
graph TD;
  A[Graph State] --> B[Checkpointer];
  B --> C[Persistent Storage];
```

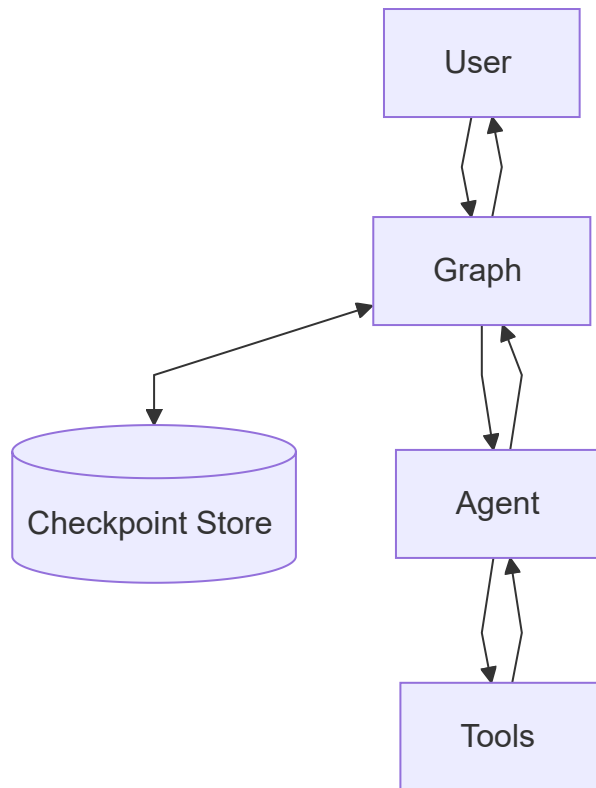
We will study this in the next part.

59. Current Architecture vs Persistent Architecture

Current:



With persistence:



Now workflow state can survive beyond one execution.

60. Common Mistakes

Mistake 1 – Thinking ToolNode Chooses the Tool

It does not.

LLM

→ Chooses/request tool

ToolNode

→ Executes requested tool

Mistake 2 – Forgetting Tool → Agent Edge

Without:

```
builder.add_edge(  
    "tools",  
    "agent"  
)
```

the model cannot naturally continue after receiving the tool result.

Mistake 3 – Executing Tool Inside Agent Node

Possible, but it removes the clean separation:

```
Agent  
→ Decision  
  
ToolNode  
→ Execution
```

Mistake 4 – No Termination Condition

Can create infinite cycles.

Mistake 5 – Trusting LLM Tool Arguments

Validate important arguments.

Mistake 6 – Using Prompt for Security

Security must be enforced by application/backend logic.

Mistake 7 – Exposing Every Backend Function as a Tool

Only expose meaningful capabilities that require model-driven selection.

61. Important Interview Questions

Q1. How do you build a tool-calling agent using LangGraph? ★ ★ ★ ★ ★

Answer:

I define the tools, bind their schemas to the LLM, maintain message state, create an Agent Node that calls the tool-aware LLM, create a Tool Node for executing requested tools, use conditional routing to decide whether tools are required, and connect the Tool Node back to the Agent Node until the model produces a final response.

Q2. What is **ToolNode**? ★ ★ ★ ★ ★

Answer:

ToolNode is a LangGraph utility that executes tool calls requested by the model and returns the resulting tool messages into the graph state.

Q3. Does ToolNode decide which tool should be called?

Answer:

No. The LLM generates the tool call. ToolNode executes the requested tool.

Q4. Why does the Tool Node connect back to the Agent Node? ★ ★ ★ ★ ★

Answer:

The model needs to receive the tool observation so it can interpret the result, decide whether additional tools are needed and eventually generate the final response.

Q5. What does **tools_condition** do?



Answer:

It is a common routing utility that examines the latest model message and routes execution toward the tool node when tool calls are present, otherwise toward completion.

Q6. What state is commonly used for simple tool-calling agents?

Answer:

Message-based state is commonly used because it stores the user messages, AI tool-call messages, tool results and final model responses.

Q7. Can an agent call multiple tools?

Answer:

Yes. A model may request multiple independent tool calls or call tools across multiple agent-tool iterations depending on the task.

Q8. Why do we need conditional edges?



Answer:

Conditional edges allow the graph to dynamically decide whether to execute tools, finish the workflow or route to another node based on the current state.

Q9. How does LangGraph replace the manual agent loop? ★ ★ ★ ★ ★

Answer:

Instead of manually writing a `while` loop and `if/else` logic, LangGraph represents the loop using nodes, edges and conditional routing, such as Agent → Tool → Agent until the completion condition is reached.

Q10. Should every backend API become an agent tool?

Answer:

No. Only meaningful capabilities that benefit from model-driven selection should generally be exposed as tools. Deterministic infrastructure and security operations should remain normal application logic.

Q11. How do you prevent infinite agent loops?

Answer:

Use explicit termination conditions, step or recursion limits, retry limits, timeouts and appropriate fallback or escalation logic.

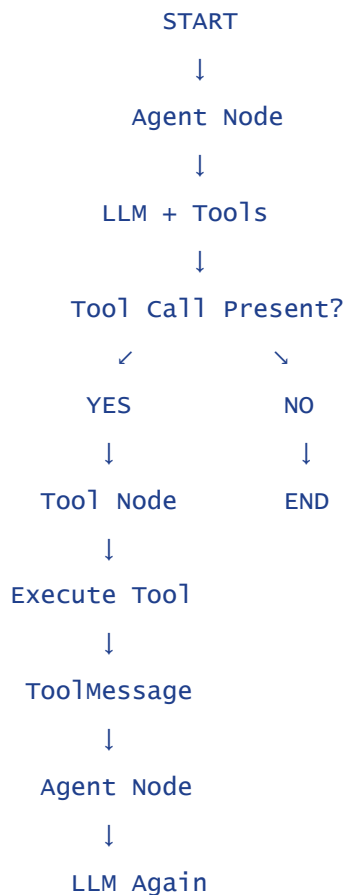
Q12. System prompt vs graph logic?

Answer:

The system prompt guides model behavior, while graph and application logic enforce deterministic workflow rules and limits. Critical restrictions should not rely only on prompts.

62. Final Mental Model

The complete tool-calling agent:



Code structure:

```
# Tools
```

```
tools = [  
    tool1,  
    tool2  
]
```

```
# Bind tools
```

```
llm_with_tools = (  
    llm.bind_tools(tools)  
)
```

```
# Agent
```

```
def agent_node(state):  
  
    response = (  
        llm_with_tools.invoke(  
            state["messages"]  
        )  
    )  
  
    return {  
        "messages": [response]  
    }
```

```
# Tool execution
```

```
tool_node = ToolNode(  
    tools  
)
```

```
# Routing
```

```
def should_continue(state):
```

```
if state["messages"][-1].tool_calls:
```

```
    return "tools"
```

```
return END
```

```
# Graph
```

```
builder = StateGraph(  
    MessagesState  
)
```

```
builder.add_node(  
    "agent",  
    agent_node  
)
```

```
builder.add_node(  
    "tools",  
    tool_node  
)
```

```
builder.add_edge(  
    START,  
    "agent"  
)
```

```
builder.add_conditional_edges(  
    "agent",  
    should_continue,  
    {  
        "tools": "tools",  
        END: END  
    }  
)
```

```
builder.add_edge(  
    "tools",  
    "agent"  
)
```

```
graph = builder.compile()
```

The responsibilities are:

LLM

→ Decide what to do

Tool

→ Provide external capability

ToolNode

→ Execute requested tools

State

→ Preserve messages and workflow data

Conditional Edge

→ Decide where execution goes

LangGraph

→ Orchestrate the complete loop

A LangGraph tool-calling agent is essentially the manual agent loop expressed as an explicit stateful graph: the model decides, the tool node acts, the result updates state, and the graph routes execution back to the model until the task is complete.

Part 3 – Persistence, Checkpointing, Threads & Human-in-the-Loop

1. Why Do We Need Persistence?



Our current LangGraph agent works like:

```
User Request
↓
Graph Starts
↓
Agent
↓
Tool
↓
Agent
↓
Final Answer
↓
Execution Ends
```

But real applications often need to continue across multiple interactions.

Example:

```
User:
Check the weather in Delhi.

Assistant:
Delhi is rainy.

User:
What about Mumbai?
```

The second request may need access to the previous conversation.

We may also need to:

- Pause a workflow
- Resume it later
- Wait for human approval
- Recover after interruption
- Maintain conversation state

This introduces:

Persistence

2. What is Persistence in LangGraph?



Persistence means saving graph state so that it can be retrieved and continued later.

Without persistence:

- Graph State
- ↓
- Process Ends
- ↓
- State Lost

With persistence:

```
Graph State
↓
Checkpoint
↓
Saved State
↓
Later Invocation
↓
Restore State
↓
Continue
```

3. What is a Checkpoint?



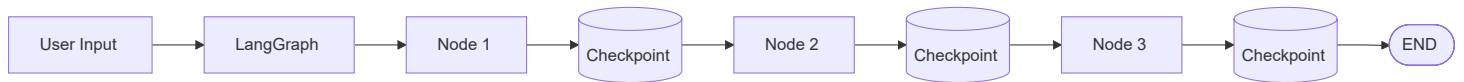
A **checkpoint** is a saved snapshot of graph state at a particular point during execution.

Suppose state contains:

```
{
  "messages": [...],
  "current_step": 3,
  "retry_count": 0
}
```

A checkpoint can preserve this state so execution can later continue from the saved workflow state.

4. Checkpointing Architecture



Conceptually:

Node Executes



State Changes



Checkpoint Saved



Next Node

5. Why Checkpointing Matters

Checkpointing enables important capabilities such as:

Conversation continuity

Workflow recovery

Pause and resume

Human-in-the-loop

State inspection

Long-running workflows

These are major reasons to use LangGraph for complex agents.

6. Checkpointer



A **checkpointer** is the component responsible for saving and retrieving graph checkpoints.

Conceptually:

```
LangGraph
  ↓
Checkpointer
  ↓
Storage
```

Different storage implementations can be used depending on the application.

For learning, we can begin with an in-memory checkpointer.

7. In-Memory Checkpointer

A common development setup is:

```
from langgraph.checkpoint.memory import (
    InMemorySaver
)

checkpointer = InMemorySaver()
```

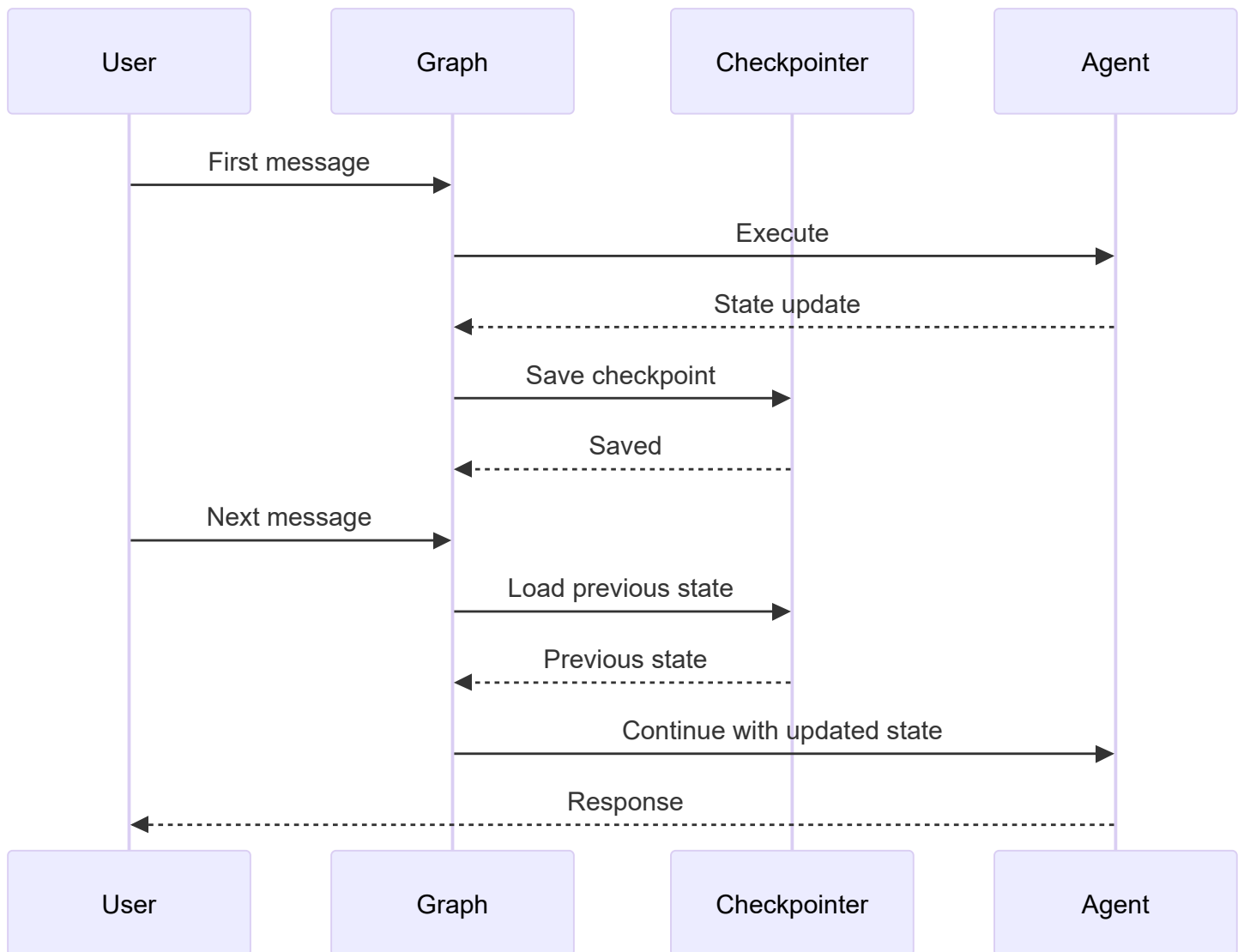
Then compile the graph with it:

```
graph = builder.compile(  
    checkpoint=checkpointer  
)
```

Now the graph can persist state for different threads while that process is alive.

Exact LangGraph APIs can evolve, so always check the version-specific documentation when implementing production code.

8. Persistence Flow



9. Thread ID



If many users or conversations use the same graph, LangGraph must know:

which saved state belongs to which conversation?

This is commonly handled using a:

thread_id

Example:

```
config = {  
  "configurable": {  
    "thread_id": "conversation_1"  
  }  
}
```

*The same **thread_id** identifies the same persisted conversation/workflow.*

10. Thread Mental Model

Think of:

Graph

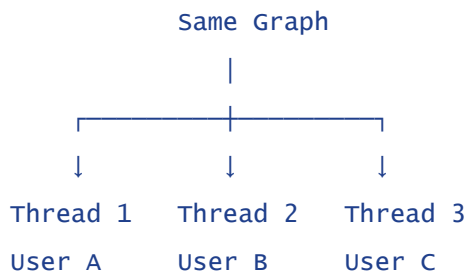
as the workflow definition.

And:

Thread

as one particular execution/conversation history using that graph.

Example:



Each thread can maintain separate state.

II. Why Thread IDs Matter

Suppose:

User A:
Weather in Delhi?

and:

User B:
Tell me about wheat.

Their states must never accidentally mix.

Conceptually:

thread_A
→ State A

thread_B
→ State B

Therefore:

Thread Identity
+
Checkpoint
=
Correct State Retrieval

12. Invoke Graph With Thread ID

Example:

```
config = {  
  "configurable": {  
    "thread_id": "user_123_chat_1"  
  }  
}
```

First interaction:

```
result = graph.invoke(  
  {  
    "messages": [  
      {  
        "role": "user",  
        "content":  
          "My city is Delhi."  
      }  
    ]  
  },  
  config=config  
)
```

Later, use the same config:

```
result = graph.invoke(  
  {  
    "messages": [  
      {  
        "role": "user",  
        "content":  
          "What is the weather there?"  
      }  
    ]  
  },  
  config=config  
)
```

The graph can recover the previous thread state through the checkpointer.

13. Different Thread = Different State

If we use:

```
config2 = {  
  "configurable": {  
    "thread_id": "user_123_chat_2"  
  }  
}
```

this represents another thread.

Conceptually:

chat_1

→ Conversation State 1

chat_2

→ Conversation State 2

This is useful when one user can have multiple conversations.

14. Thread ID Should Come From Application Logic



Do not ask the LLM to invent security-sensitive conversation identity.

Your backend should determine:

Authenticated User

Conversation ID

Thread ID

Example:

```
Frontend
↓
Backend
↓
Authenticate User
↓
Resolve Conversation
↓
Create / Load Thread ID
↓
Invoke LangGraph
```

15. Thread ID is Not Authentication

Very important:

```
thread_id
≠
user authentication
```

Knowing:

```
thread_id = "123"
```

must not automatically grant access to that conversation.

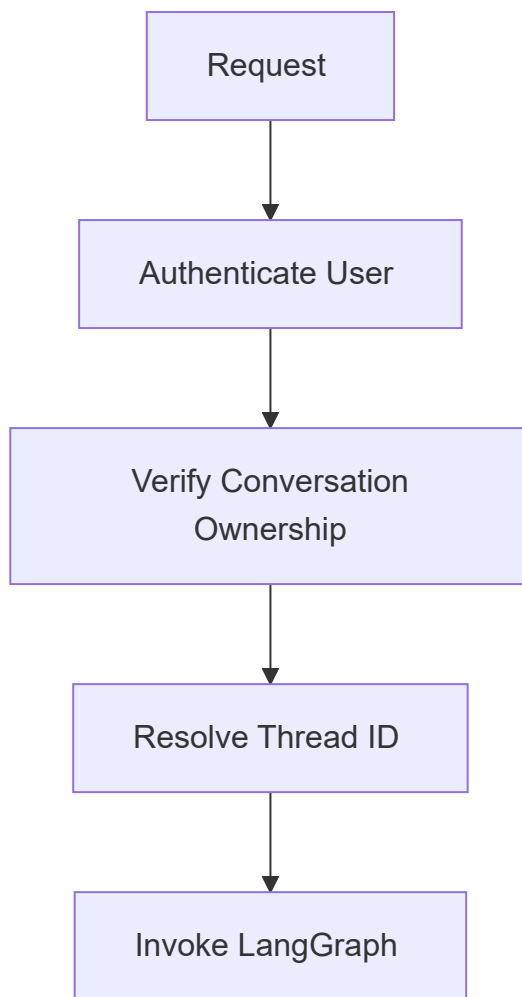
Your backend must still enforce:

```
Authentication

Authorization

Ownership
```

Correct architecture:



16. In-Memory vs Persistent Checkpointer

An in-memory saver is useful for:

Learning

Development

Testing

But:

Server Restarts

↓

Memory Lost

For production, state usually needs durable storage.

Conceptually:

Development

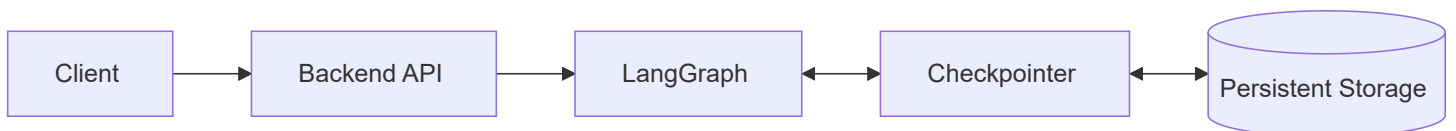
→ Memory

Production

→ Persistent Database / Checkpoint Store

17. Durable Persistence

Production architecture may look like:



Now:

Server restarts

do not necessarily mean:

Conversation state disappears

because checkpoints are stored externally.

18. Checkpointing vs Long-Term Memory



This distinction is extremely important.

Checkpointing

Stores:

workflow state

Example:

Messages

Current node

Current workflow information

Pending execution state

Purpose:

Continue / restore workflow

Long-Term Memory

Stores:

Useful information for future interactions

Example:

Preferred language

Preferred units

Relevant past decisions

Purpose:

Personalization / future retrieval

19. Simple Difference

Remember:

CHECKPOINT

"where was this workflow?"

versus:

MEMORY

"what useful information
should be remembered?"

They can be used together.

20. Conversation History and Checkpoints

Suppose a message-based graph has:

HumanMessage

AIMessage

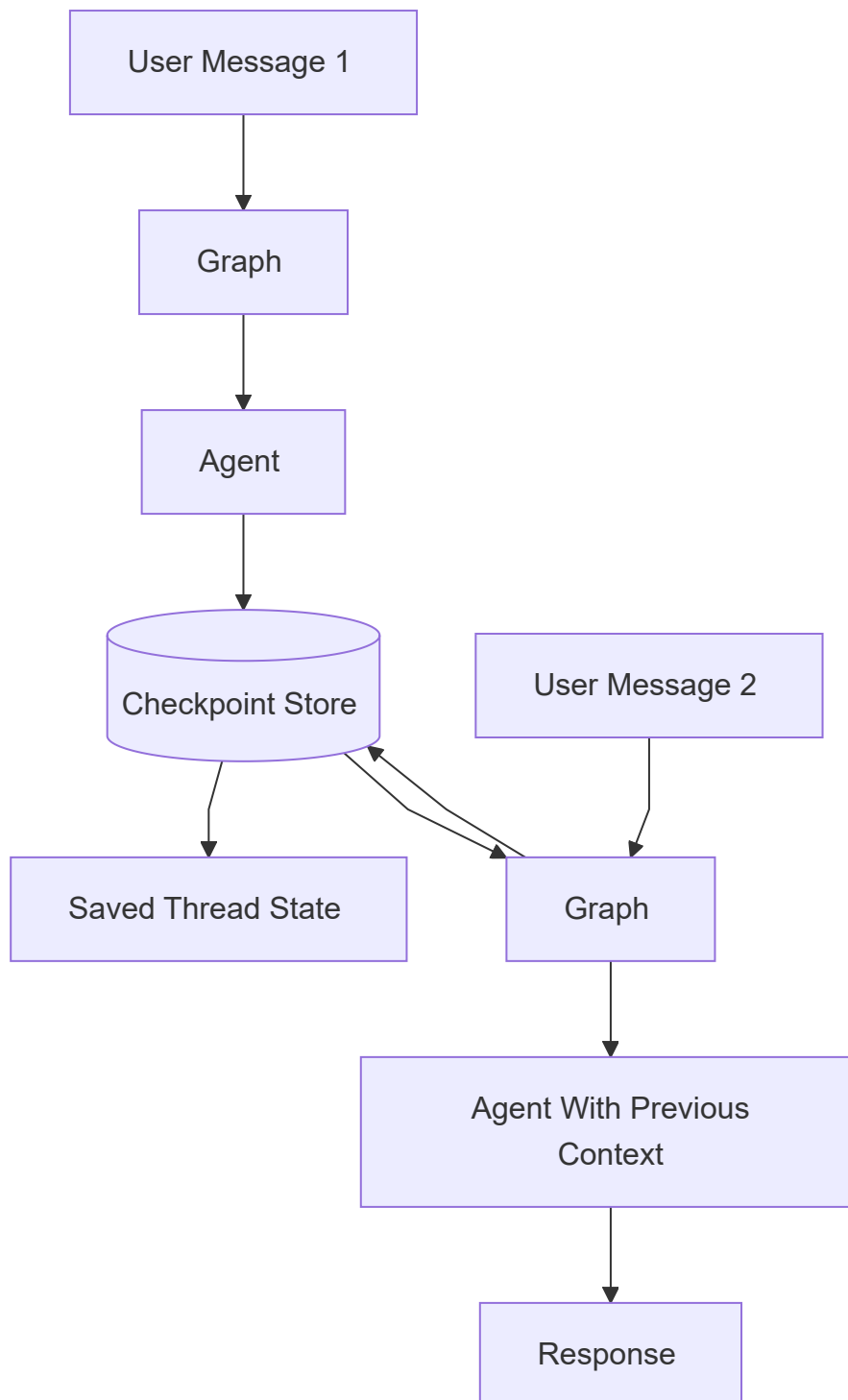
ToolMessage

AIMessage

Because these messages are part of graph state, checkpointing can preserve the conversation state.

Then the next invocation can continue from that state.

21. Persistence Architecture



22. Persistence Does Not Mean Send Everything Forever



A conversation may eventually contain:

Hundreds of messages

Large tool outputs

Documents

Long responses

Even if they are persisted, sending all of them to the model every time may be inefficient.

Remember from Chapter 3:

Persisted State

≠

Model Context

You may need:

Trimming

Summarization

Selective retrieval

Memory extraction

for long-running applications.

23. Human-in-the-Loop



Some agent actions should not execute automatically.

Example:

Agent:

"I should send this email."

Instead of immediately sending:

Agent Decision



Pause workflow



Human Reviews



Approve / Reject



Continue

This pattern is called:

Human-in-the-Loop

24. Why Human-in-the-Loop?

Useful for actions involving:

Sending messages

Deleting data

Financial operations

Changing important records

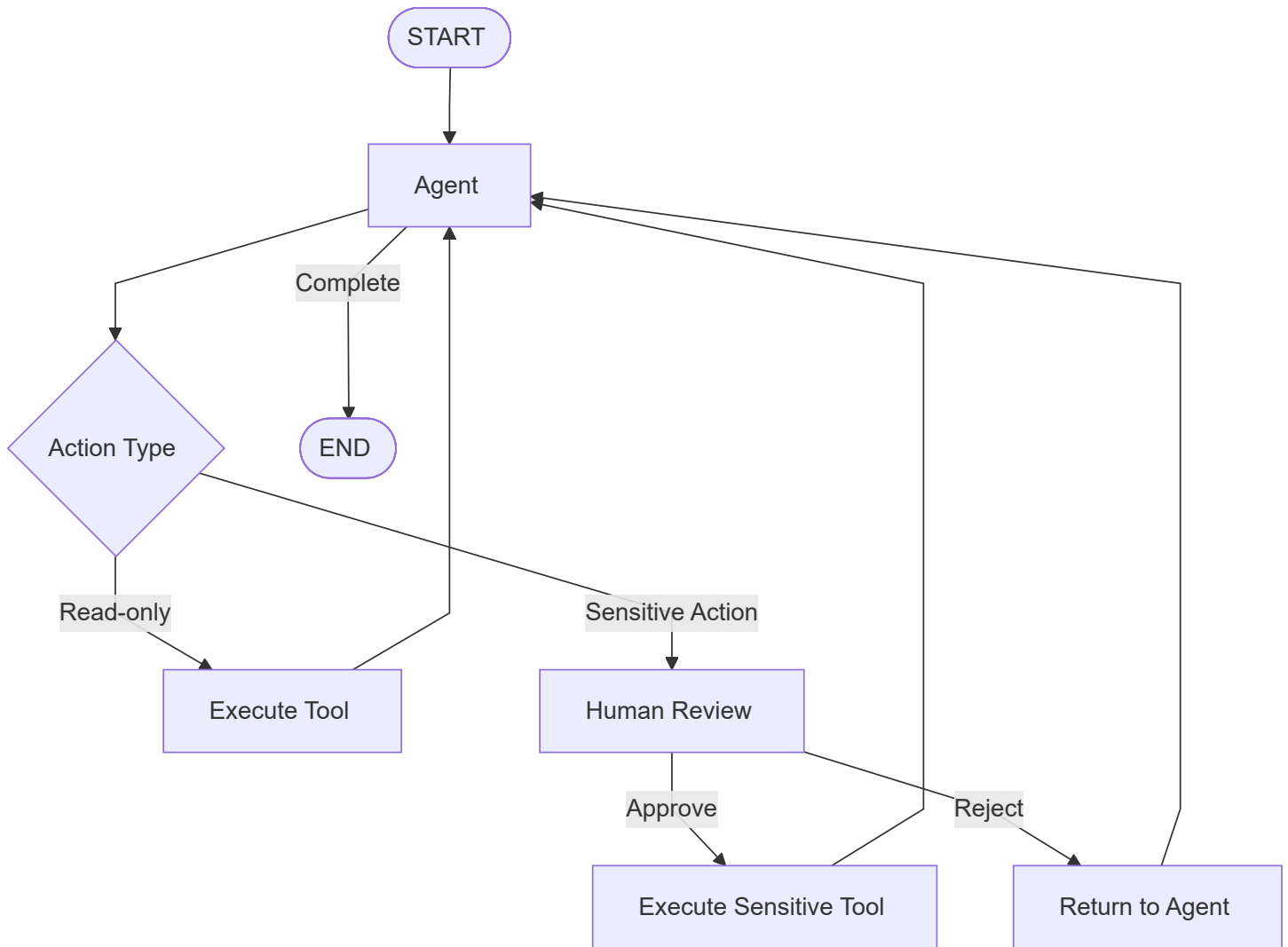
Publishing content

High-impact decisions

Sensitive external actions

The goal is to keep humans in control where appropriate.

25. Human Approval Architecture



26. Why Persistence is Important for Human Approval

Imagine:

Agent



Needs Approval



Workflow Pauses

The human may respond:

5 minutes later

1 hour later

1 day later

The system must remember:

What was the request?

What action was proposed?

What state existed?

Where should execution continue?

That is why:

Human-in-the-loop

+

Checkpointing

work naturally together.

27. Interrupts



LangGraph supports workflow interruption patterns.

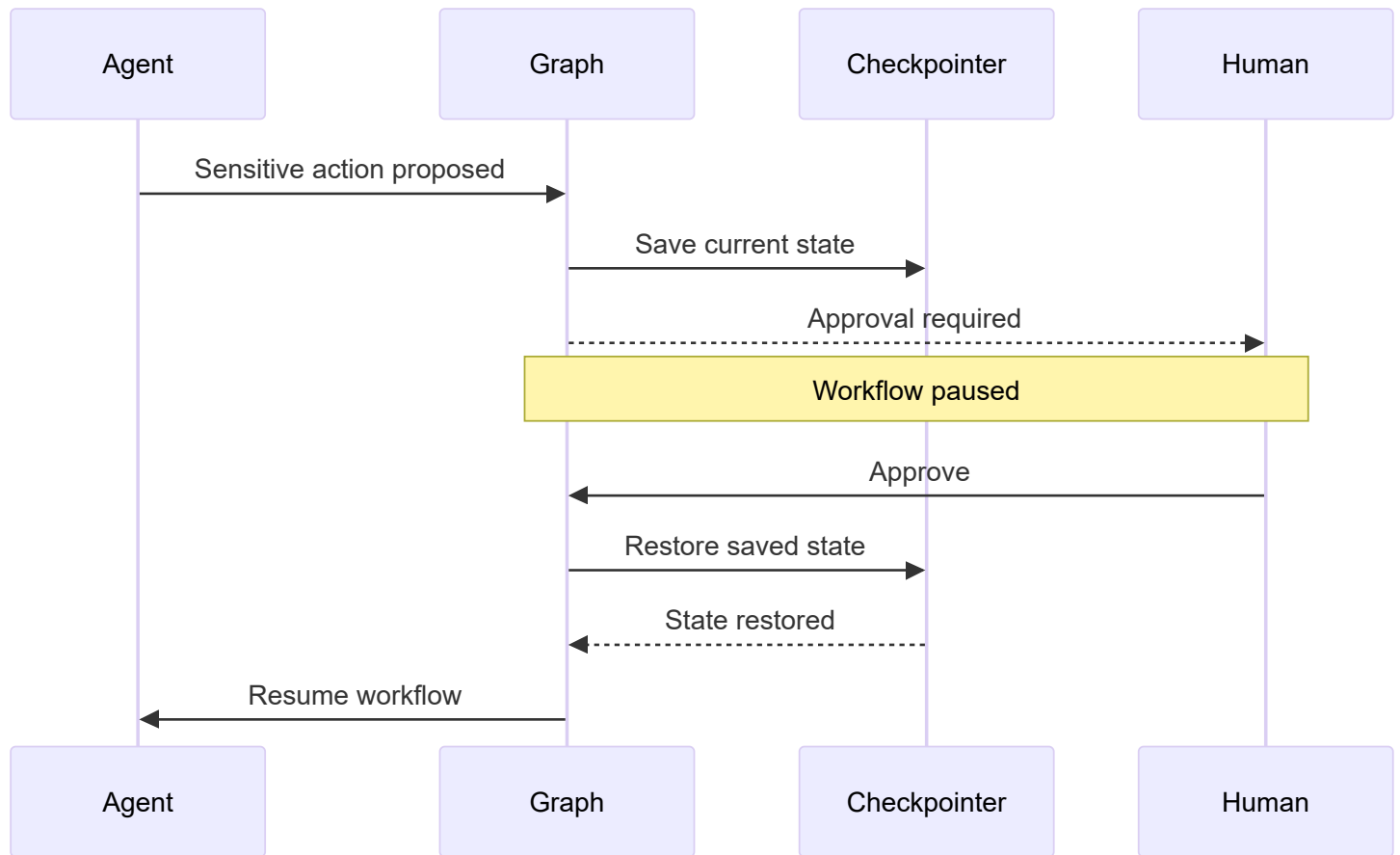
Conceptually, an interrupt means:

Pause graph execution here
and wait for external input.

Example:

```
graph TD
    Agent --> PrepareAction[Prepare Action]
    PrepareAction --> Interrupt
    Interrupt --> HumanInput[Human Input]
    HumanInput --> Resume
```

28. Interrupt Mental Model



29. Basic Interrupt Concept

A simplified LangGraph-style pattern may use:

```
from langgraph.types import (
    interrupt
)
```

Inside a node:

```
def approval_node(state):
```

```
decision = interrupt(  
    {  
        "question":  
            "Approve this action?",  
  
        "action":  
            state["pending_action"]  
    }  
)  
  
return {  
    "approval":  
        decision  
}
```

The exact resume mechanism depends on the LangGraph API/version, but the core concept is:

```
interrupt()  
→ Pause execution  
→ Persist state  
→ Wait for input  
→ Resume
```

30. Human Review Node

Suppose state contains:

```
class AgentState(  
    MessagesState  
):  
  
    pending_action: dict  
  
    approval: bool
```

The agent might prepare:

```
{  
    "pending_action": {  
        "tool":  
            "send_email",  
  
        "recipient":  
            "manager@example.com"  
    }  
}
```

Instead of executing directly:

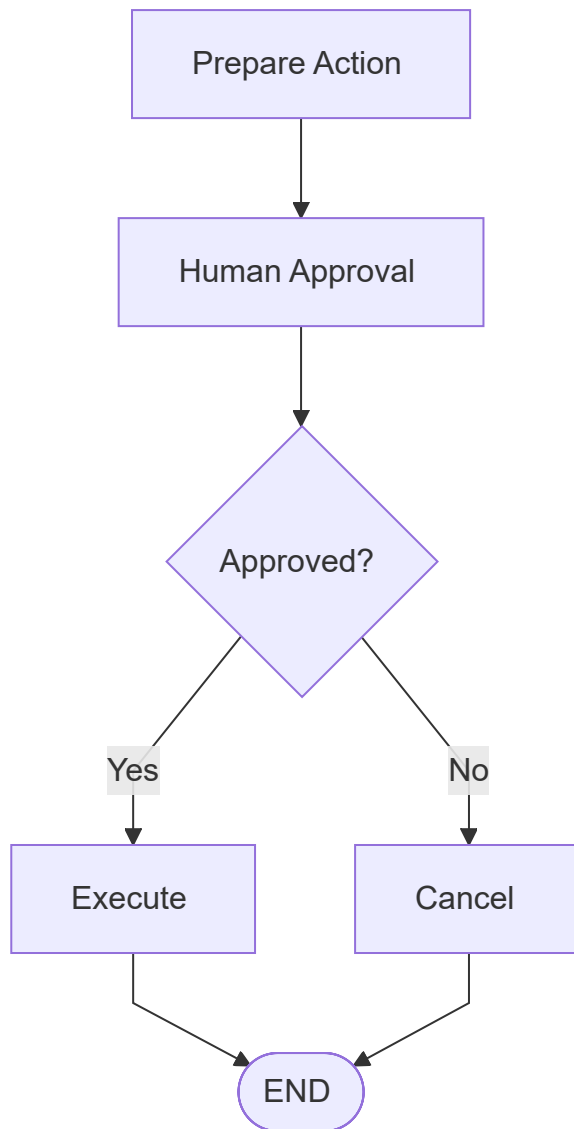
```
Pending Action  
↓  
Approval Node
```

31. Approval Routing

After human input:

```
def route_approval(state):  
  
    if state["approval"]:  
        return "execute"  
  
    return "cancel"
```

Graph:



32. Approval Should Show Useful Information

Bad approval:

Approve?

The human does not know what is happening.

Better:

Action:

Send Email

Recipient:

manager@example.com

Subject:

Weekly Report

Approve?

Humans need enough information to make an informed decision.

33. Human Can Also Edit

Human-in-the-loop does not have to mean only:

Approve

Reject

It can also allow:

Approve

Reject

Edit

Example:

Agent Draft

↓

Human Review

↓

Edit Recipient

↓

Approve

↓

Execute

34. Human-in-the-Loop vs Human-on-the-Loop

Useful conceptual distinction:

Human-in-the-Loop

Human approval is part of the execution path.

Agent

↓

Human

↓

Agent

Human-on-the-Loop

Agent operates automatically while humans monitor and intervene when necessary.

For your current level, focus mainly on:

Human-in-the-loop

because it directly relates to LangGraph interrupts.

35. Human Approval Does Not Replace Authorization



Suppose:

User approves deleting another user's account.

That does not mean the action should be allowed.

Backend must still check:

Is this user authorized?

Therefore:

Human Approval

≠

Authorization

Correct:

Agent Proposal

↓

Human Approval

↓

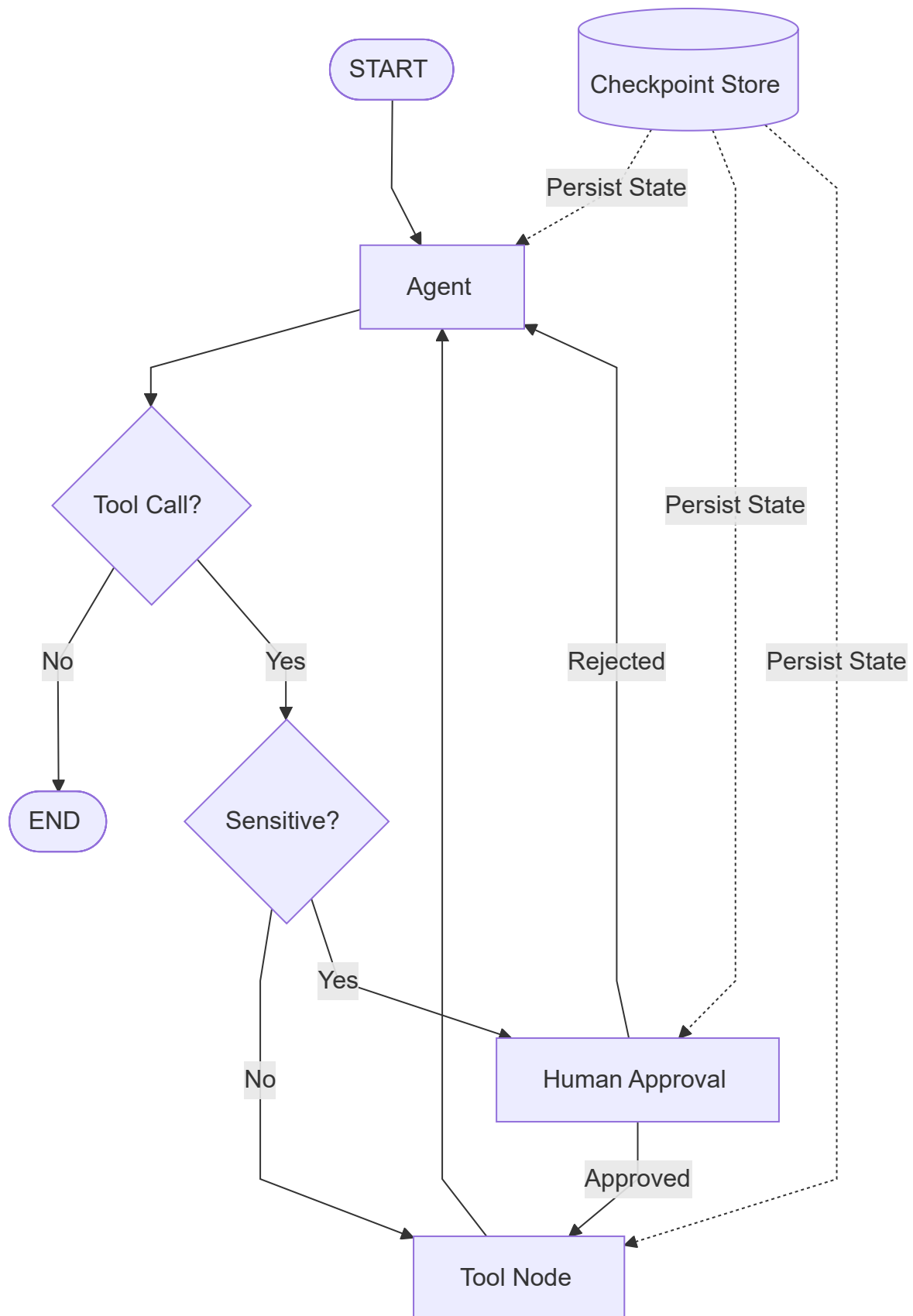
Authorization Check

↓

Execute

36. Persistence + Tool Agent

Let's extend our previous architecture:



Now our agent has:

Tool Calling

State

Persistence

Human Approval

37. Conversation Persistence Example

Let's extend the previous tool-calling agent.

Checkpointing:

```
from langgraph.checkpoint.memory import (  
    InMemorySaver  
)  
  
checkpointer = InMemorySaver()
```

Compile:

```
graph = builder.compile(  
    checkpointer=checkpointer  
)
```

Config:

```
config = {  
  "configurable": {  
    "thread_id":  
      "conversation_123"  
  }  
}
```

38. First Message

```
graph.invoke(  
  {  
    "messages": [  
      {  
        "role": "user",  
        "content":  
          "My city is Delhi."  
      }  
    ]  
  },  
  config=config  
)
```

The graph processes the message and saves the thread state.

39. Second Message

```
graph.invoke(  
  {  
    "messages": [  
      {  
        "role": "user",  
        "content":  
          "What is the weather there?"  
      }  
    ]  
  },  
  config=config  
)
```

Because the same:

```
thread_id
```

is used, the graph can continue from the persisted conversation state.

40. Different Conversation

```
another_config = {  
  "configurable": {  
    "thread_id":  
      "conversation_456"  
  }  
}
```

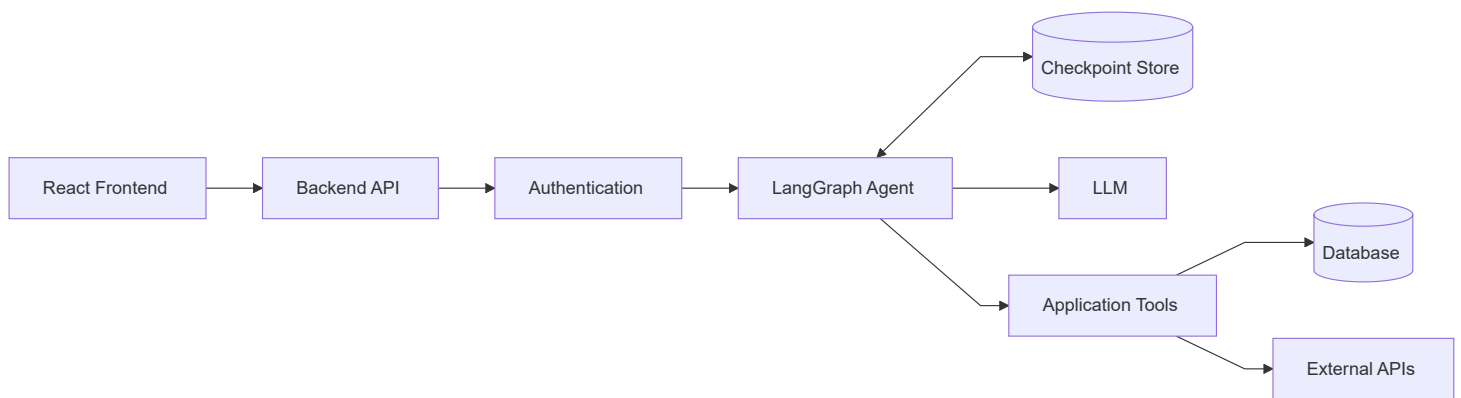
This creates/uses a separate state history.

Therefore:

conversation_123
≠
conversation_456

41. Practical Backend Architecture

For a MERN/FastAPI-style application, the overall architecture could be:



The backend remains responsible for:

Authentication
Authorization
Validation
Conversation ownership
Secrets
Rate limiting

The graph handles the agent workflow.

42. Where Should Thread ID Come From?

A good application can maintain:

- User ID
- Conversation ID

Then map:

- Conversation ID
→ LangGraph Thread ID

Example database:

- conversations
 - id
 - user_id
 - title
 - created_at

Then:

- `thread_id = conversation.id`

This provides clean separation between conversations.

43. Do Not Use Only User ID as Thread ID

Suppose one user has:

```
Chat 1:
Crop recommendation

Chat 2:
Market price research
```

If both use:

```
thread_id = user_id
```

their states may become one conversation.

Better:

```
User
├─ Conversation 1
│   └─ Thread 1
│
└─ Conversation 2
    └─ Thread 2
```

44. Persistence vs Database Chat History

You may already store chat messages in MongoDB/PostgreSQL.

So why use LangGraph persistence?

They serve related but different purposes.

Application Chat Database

Useful for:

Displaying conversations

Search

Analytics

User history

UI

LangGraph Checkpoints

Useful for:

Restoring graph execution state

Resuming workflows

Interrupts

Agent state

You may use both depending on the application.

45. Do Not Duplicate Everything Blindly

Avoid:

Messages in MongoDB

Same messages in another database

Same messages in vector DB

Same messages in checkpoints

Same messages in logs

without a reason.

Design clear responsibilities:

Chat DB

→ Product conversation records

Checkpoint Store

→ Workflow persistence

Vector Store

→ Semantic retrieval when needed

Logs/Tracing

→ Observability

46. State Inspection

Persistence can also help inspect workflow state.

Useful for debugging questions such as:

What messages existed?

What node was reached?

What action was pending?

What state caused the error?

This becomes especially valuable in complex agent workflows.

47. Failure Recovery

Consider:

Step 1

→ Search

Step 2

→ Analyze

Step 3

→ Generate Report

Step 4

→ Send

Application crashes after:

Step 3

With persisted checkpoints:

Restore State

↓

Continue From Appropriate Point

instead of blindly starting everything again.

48. Idempotency



Persistence introduces an important backend concept:

Idempotency

Suppose:

```
graph TD; A[Agent executes payment] --> B[↓]; B --> C[State save fails]; C --> D[↓]; D --> E[Workflow retries]; E --> F[↓]; F --> G[Payment executes again];
```

This is dangerous.

External actions should be designed so retries do not accidentally duplicate important operations.

Examples:

```
graph TD; A[Payment]; B[Sending email]; C[Creating order]; D[Deleting data];
```

Use techniques such as:

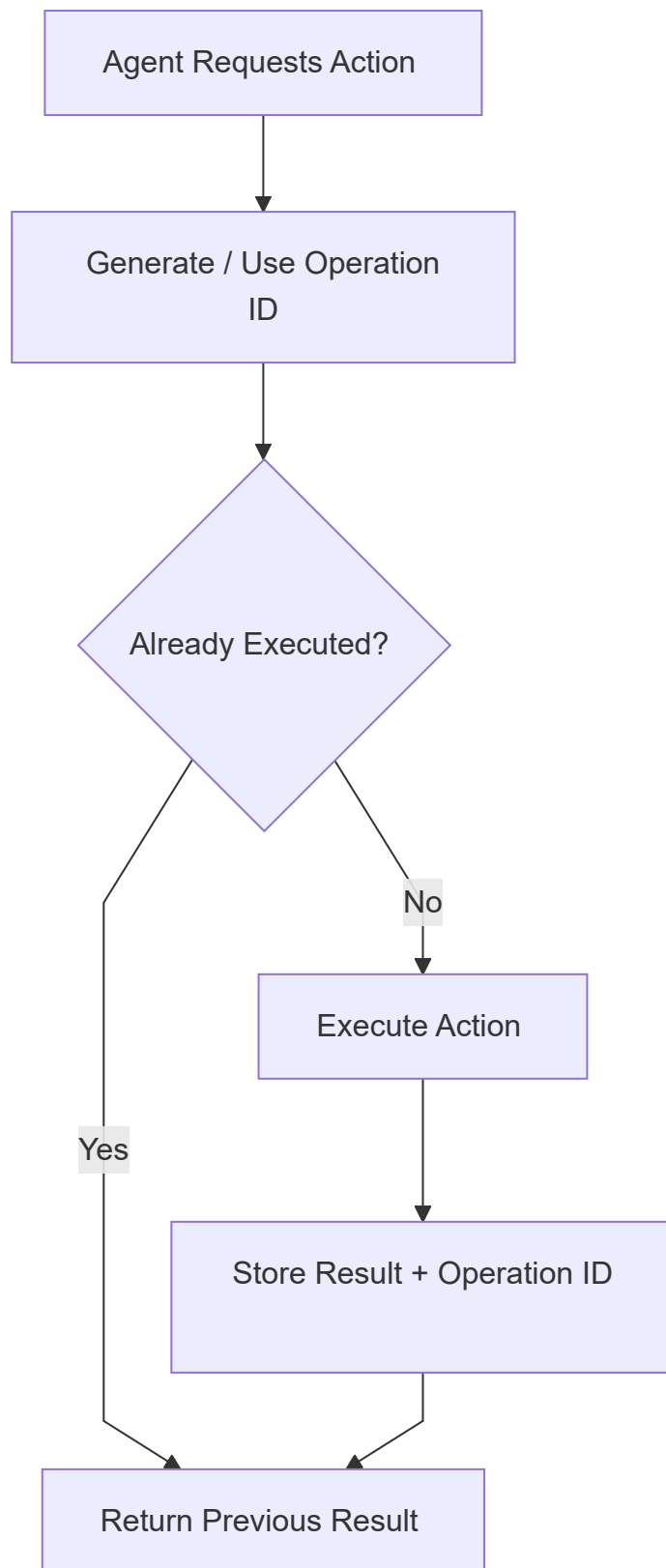
Idempotency keys

Unique request IDs

Database constraints

Execution records

49. Example Idempotency Flow



This prevents duplicate execution during retries or resume operations.

50. Persistence Security

Persisted state may contain:

- User messages
- Tool results
- Personal information
- Internal application data

Therefore production systems should consider:

- Access control
- Encryption
- Retention policies
- Deletion
- Data minimization
- Audit logging

Persistence should not mean:

- Store everything forever.

51. Common Mistakes

Mistake 1 – Thinking Persistence = Memory

Persistence

→ Save workflow state

Memory

→ Retain/retrieve useful information

Mistake 2 – Using Same Thread for Every User

This can mix state.

Mistake 3 – Treating Thread ID as Authentication

Thread identity does not prove ownership.

Mistake 4 – Using In-Memory Storage in Production

It disappears when the process restarts.

Mistake 5 – Sending Entire Persisted History to LLM Forever

Use context management when conversations grow.

Mistake 6 – Human Approval Without Checkpointing

Long approval waits require reliable state persistence.

Mistake 7 – Human Approval Without Authorization

Approval does not grant permissions.

Mistake 8 – Ignoring Duplicate Side Effects

Retries/resumes can accidentally execute actions twice.

Use idempotency.

52. Important Interview Questions

Q1. What is persistence in LangGraph?



Answer:

Persistence is the ability to save graph state using checkpoints so that a workflow or conversation can later retrieve and continue from previously stored state.

Q2. What is a checkpoint? ★ ★ ★ ★ ★

Answer:

A checkpoint is a saved snapshot of graph state at a particular point during workflow execution.

Q3. What is a checkpointer? ★ ★ ★ ★ ★

Answer:

A checkpointer is the component responsible for storing and retrieving graph checkpoints.

Q4. What is `thread_id`? ★ ★ ★ ★ ★

Answer:

A `thread_id` identifies a particular conversation or workflow thread so the checkpointer can associate execution with the correct persisted state.

Q5. Why shouldn't we use the same thread ID for every user?

Answer:

Different conversations require isolated state. Reusing one thread ID can cause unrelated conversation states to become mixed.

Q6. Is thread ID authentication?

Answer:

No. Authentication and authorization must be handled by the application. Thread IDs are used for workflow state identity.

Q7. Checkpointing vs memory? ★ ★ ★ ★ ★

Answer:

Checkpointing stores workflow state so execution can be restored or resumed.

Memory stores useful information that may be retrieved for future interactions.

Q8. Why is persistence important for human-in-the-loop?

Answer:

A human may respond much later, so the system needs to preserve the exact workflow state while execution is paused and restore it when the human provides input.

Q9. What is an interrupt? ★ ★ ★ ★ ★

Answer:

An interrupt pauses graph execution at a specific point so external input, such as human approval, can be obtained before the workflow continues.

Q10. What is human-in-the-loop?

Answer:

Human-in-the-loop is a workflow pattern where human input, review or approval becomes part of the agent execution path before certain decisions or actions proceed.

Q11. Should every tool require human approval?

Answer:

No. Approval should generally be used for actions where human control is valuable, especially consequential, sensitive or irreversible operations.

Q12. What is idempotency and why is it important for agents? ★ ★ ★ ★ ★

Answer:

Idempotency ensures that repeating an operation does not unintentionally create duplicate side effects. It is important because agent workflows may retry or resume operations after failures.

Q13. Can LangGraph persistence replace an application's chat database?

Answer:

Not necessarily. Checkpoints primarily support workflow execution and recovery, while an application database may serve product features such as conversation display, search, analytics and history management.

53. Final Mental Model

Remember:

CHECKPOINT

→ Saved graph state

CHECKPOINTER

→ Saves and retrieves checkpoints

THREAD ID

→ Identifies which workflow/conversation state belongs together

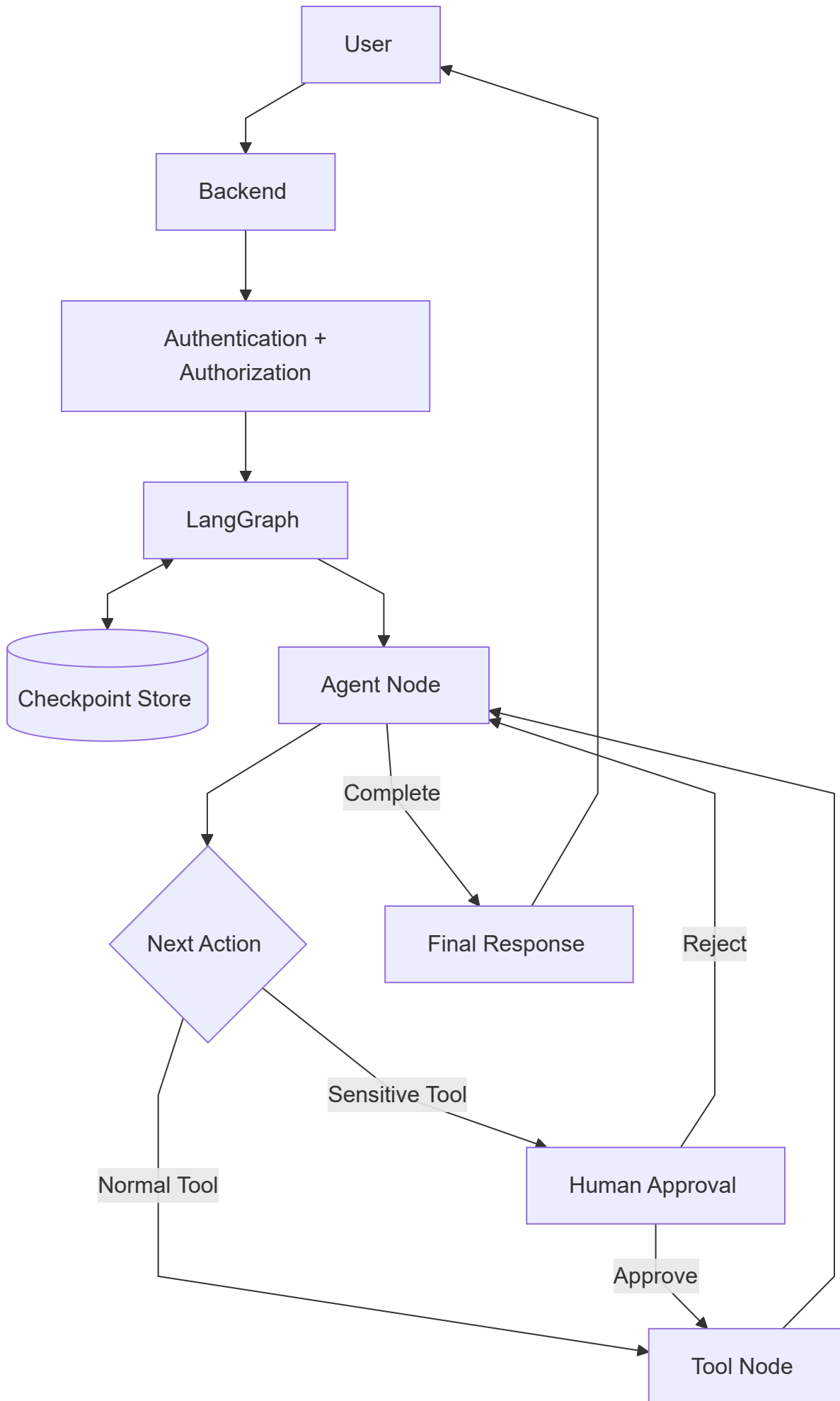
INTERRUPT

→ Pause workflow for external input

HUMAN-IN-THE-LOOP

→ Human becomes part of the workflow

Complete architecture:



The evolution of our LangGraph agent is now:

Part 1

State

+

Nodes

+

Edges

+

Conditional Routing



Part 2

LLM

+

Tools

+

Agent ↔ Tool Loop



Part 3

Persistence

+

Checkpoints

+

Threads

+

Interrupts

+

Human Approval

So our agent is no longer just:

LLM + Tool

It is becoming a real stateful workflow:

LLM

+

Tools

+

State

+

Graph

+

Persistence

+

Human Control

Checkpointing gives a LangGraph workflow continuity, thread IDs isolate individual executions, and interrupts allow humans to safely participate in long-running agent workflows.

Part 4 – Advanced LangGraph Patterns: Planning, Reflection, Subgraphs & Production Design

1. Where Are We Now?

So far, our LangGraph agent can:

Maintain State

Call Tools

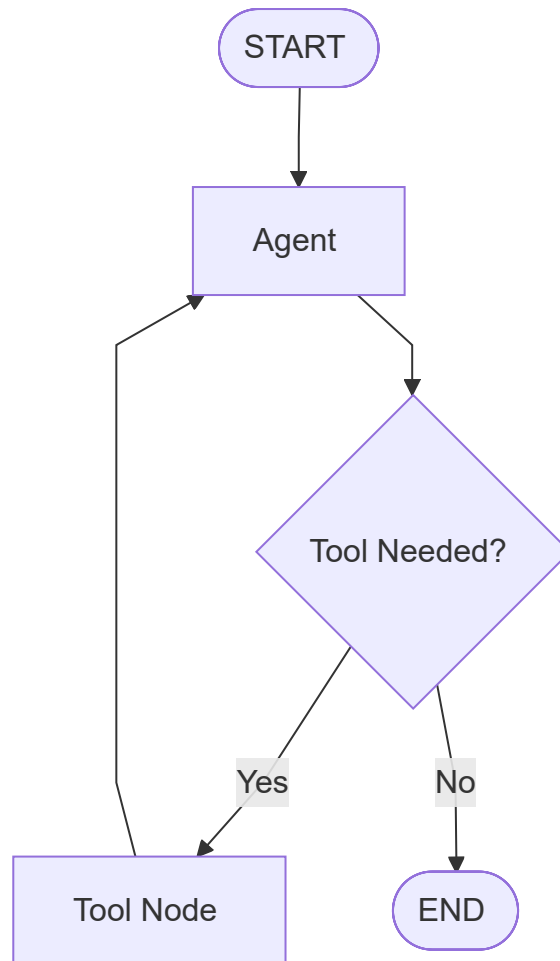
Loop Between Agent and Tools

Persist Conversations

Resume workflows

Pause for Human Approval

Basic architecture:



But some tasks require more structure.

Example:

User:
Research three crops, compare their profitability,
check weather suitability and recommend the best one.

Instead of allowing one agent to handle everything in one loop, we may explicitly separate:

Planning

Execution

Evaluation

Replanning

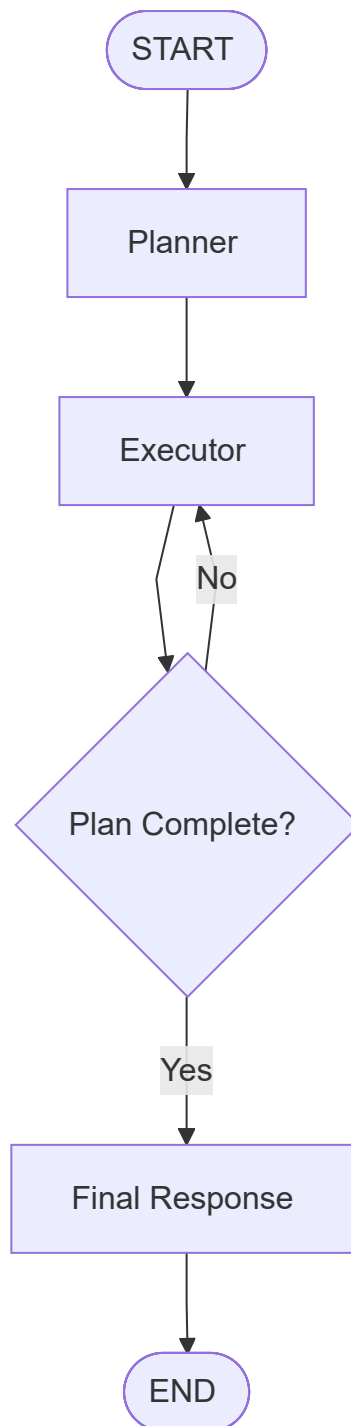
Final Response

2. Planning with LangGraph



We studied planning conceptually in Chapter 3.

LangGraph allows us to represent planning as an explicit node.



The planner creates the plan.

The executor performs the steps.

3. Planning State

Our state might contain:

```
from typing import TypedDict

class PlanningState(TypedDict):

    goal: str

    plan: list[str]

    current_step: int

    results: list[str]

    final_answer: str
```

Example state:

```
{
    "goal":
        "Compare wheat and rice",

    "plan": [
        "Get wheat information",
        "Get rice information",
        "Compare both crops"
    ],

    "current_step":
        0,

    "results":
        [],
```



```
"final_answer":  
    ""  
}
```

4. Planner Node

The planner converts:

```
Goal
```

into:

```
Structured Plan
```

Conceptually:

```
def planner_node(state):  
  
    goal = state["goal"]  
  
    plan = create_plan(goal)  
  
    return {  
        "plan": plan,  
        "current_step": 0  
    }
```

Example result:

```
[  
    "Get wheat information",  
    "Get rice information",  
    "Compare profitability"  
]
```

5. Prefer Structured Plans



Instead of asking the LLM to return:

```
First I think we should search wheat,  
then perhaps search rice...
```

prefer structured output.

Example:

```
from pydantic import BaseModel  
  
class Plan(BaseModel):  
  
    steps: list[str]
```

Then:

```
planner = llm.with_structured_output(  
    Plan  
)
```

Conceptually:

```
result = planner.invoke(  
    f"Create a plan for: {goal}"  
)
```

Now:

```
result.steps
```

contains predictable structured data.

6. Planner Node with Structured Output

```
def planner_node(state):  
  
    result = planner.invoke(  
        f"""  
        Create a concise plan  
        for this goal:  
  
        {state["goal"]}  
        """  
    )  
  
    return {  
        "plan":  
            result.steps,  
  
        "current_step":  
            0
```

```
}
```

This is much easier to execute than free-form text.

7. Executor Node

The executor works on the current step.

```
def executor_node(state):  
  
    step = state["plan"] [  
        state["current_step"]  
    ]  
  
    result = execute_step(step)  
  
    return {  
        "results":  
            state["results"] + [result],  
  
        "current_step":  
            state["current_step"] + 1  
    }
```

Conceptually:

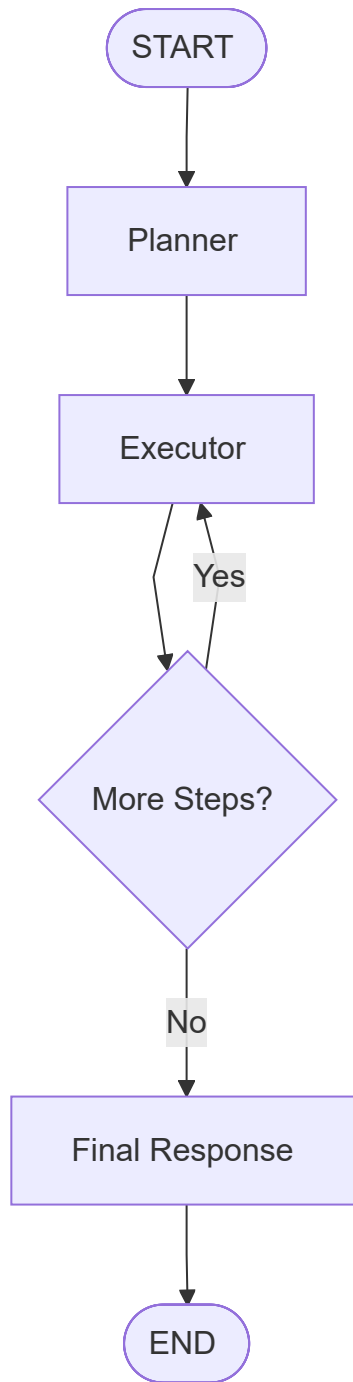
```
Plan  
↓  
Current Step  
↓  
Execute  
↓  
Observation  
↓  
Update State
```

8. Plan Completion Router

After each step:

```
def plan_complete(state):  
  
    if (  
        state["current_step"]  
        >= len(state["plan"])  
    ):  
        return "final"  
  
    return "executor"
```

Graph:



9. Final Response Node

```
def final_node(state):  
  
    response = llm.invoke(  
        f"  
        Goal:
```

```

    {state["goal"]}

    Results:
    {state["results"]}

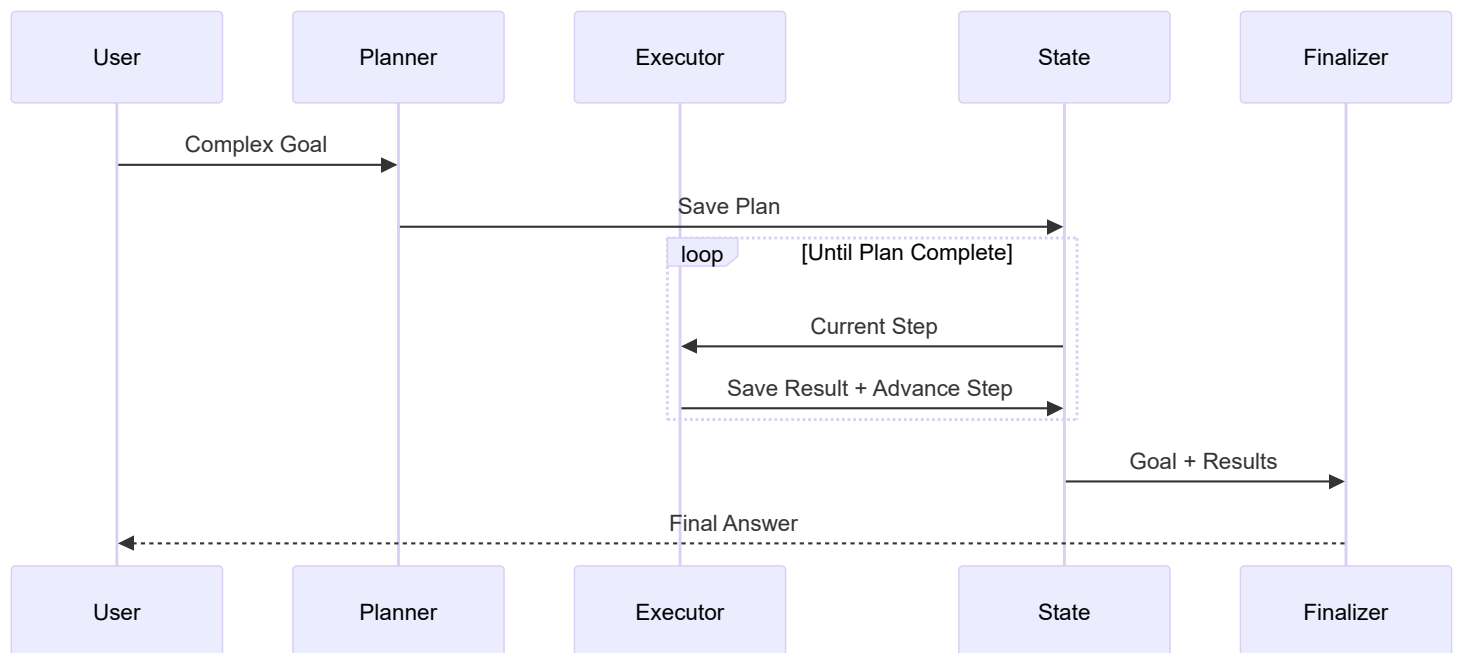
    Generate the final answer.
    ""
)

return {
    "final_answer":
        response.content
}

```

Now the final response is generated from the accumulated results.

10. Plan-and-Execute Flow



11. Replanning



The original plan may fail.

Example:

Plan:

1. Search local crop database
2. Compare results

But:

Database Result:
No information found.

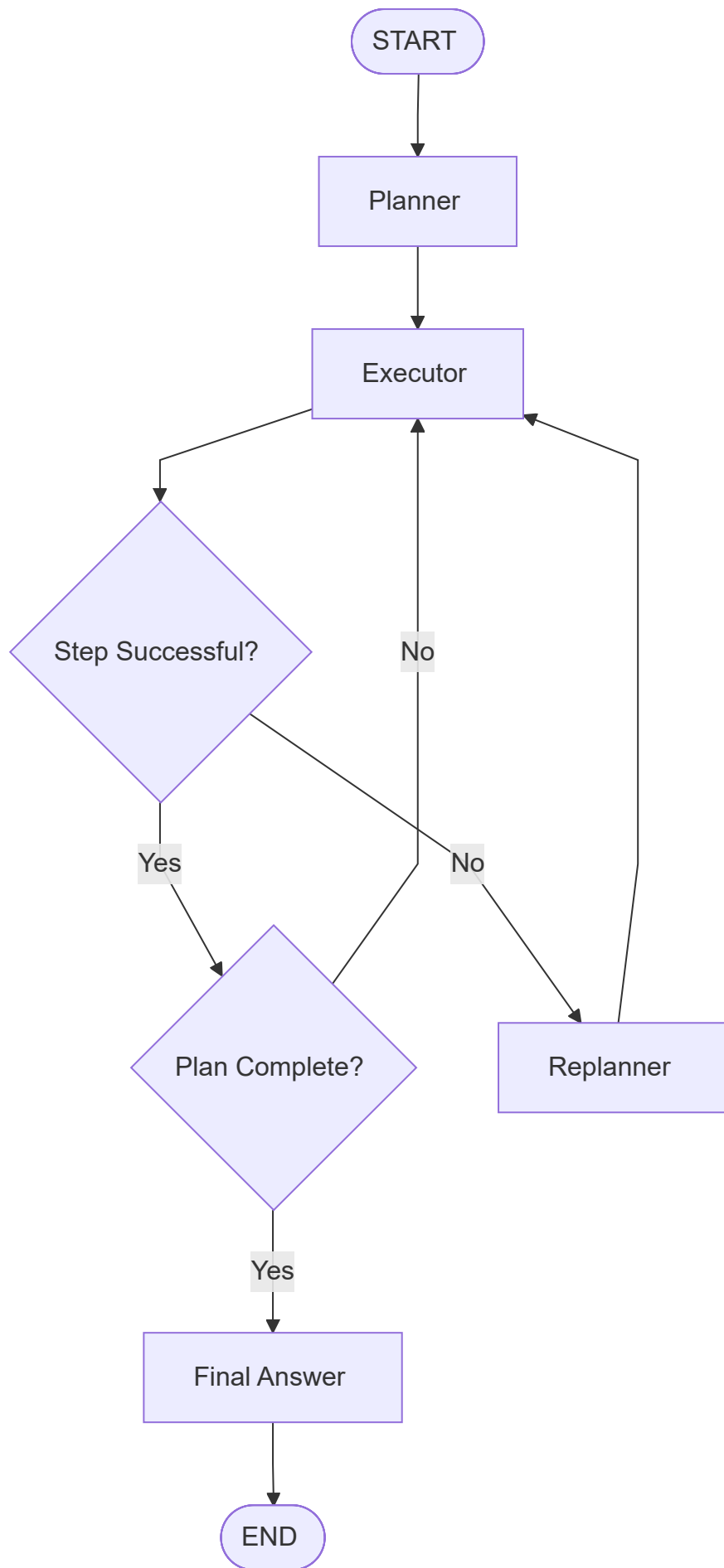
The workflow may need to:

Replan

New plan:

1. Search another knowledge source
 2. Compare results
-

12. Replanning Graph



13. Replanner Node

Conceptually:

```
def replanner_node(state):  
  
    new_plan = replan(  
        goal=state["goal"],  
        old_plan=state["plan"],  
        results=state["results"]  
    )  
  
    return {  
        "plan":  
            new_plan,  
  
        "current_step":  
            0  
    }
```

The replanner sees:

```
Original Goal  
  
Previous Plan  
  
Observations  
  
Failure
```

and generates a modified plan.

14. Limit Replanning



Never allow:

```
Plan
↓
Fail
↓
Replan
↓
Fail
↓
Replan
↓
Fail
↓
...
```

Track:

```
replan_count: int
```

Example:

```
MAX_REPLANS = 2
```

Routing:

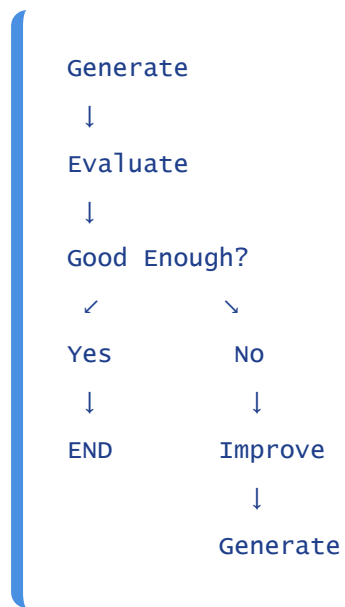
```
if state["replan_count"] >= MAX_REPLANS:

    return "fallback"
```

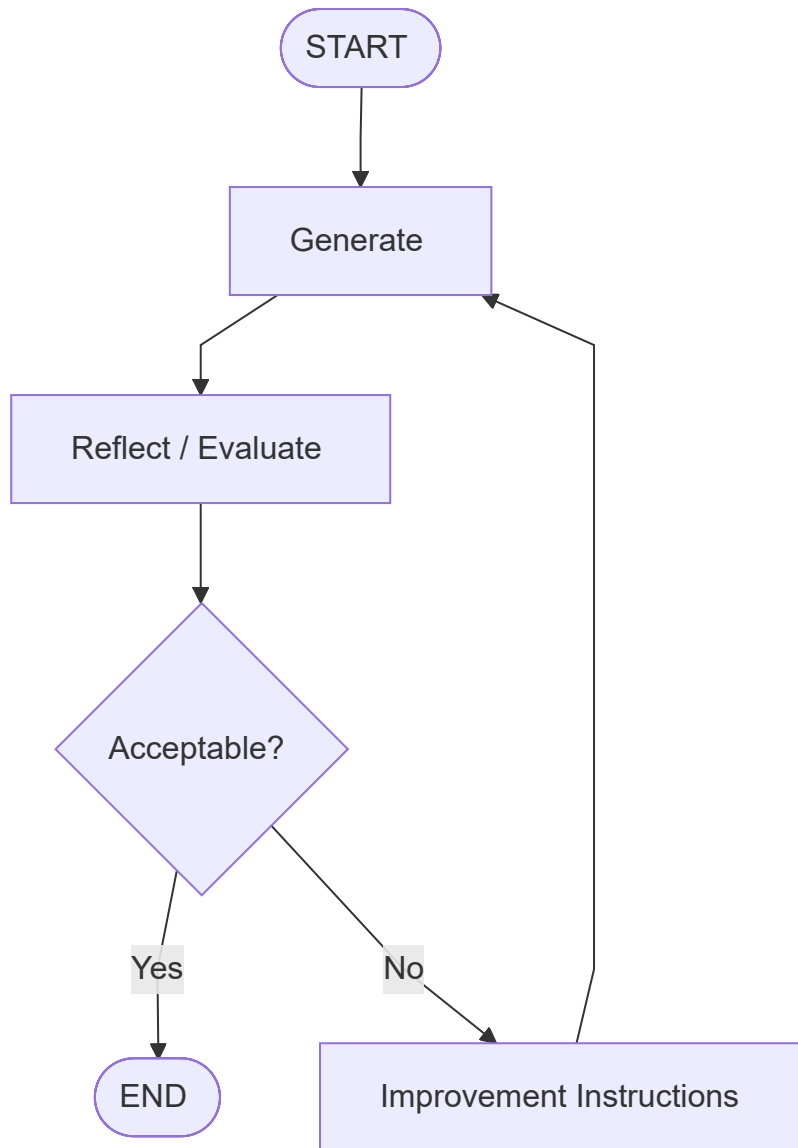
15. Reflection in LangGraph

Reflection can also become an explicit node.

Architecture:



Graph:



16. Reflection State

Example:

```
class ReflectionState(TypedDict):  
  
    query: str  
  
    answer: str  
  
    feedback: str  
  
    attempts: int
```

Execution:

```
Generate Answer  
↓  
Store answer  
↓  
Evaluate  
↓  
Store feedback  
↓  
Generate improved answer
```

17. Generate Node

```
def generate_node(state):  
  
    response = llm.invoke(  
        f"""  
        Question:  
        {state["query"]}  
  
        Previous feedback:  
        {state["feedback"]}  
  
        Generate an answer.
```

```

        """
    )

    return {
        "answer":
            response.content,

        "attempts":
            state["attempts"] + 1
    }

```

18. Reflection Node

Conceptually:

```

def reflection_node(state):

    evaluation = evaluator.invoke(
        f"""
        Question:
        {state["query"]}

        Answer:
        {state["answer"]}

        Evaluate whether the answer
        satisfies the request.
        """
    )

    return {
        "feedback":
            evaluation.content
    }

```

Then routing determines whether another attempt is needed.

19. Reflection Must Have a Limit



Without a limit:

```
Generate
↓
Critique
↓
Generate
↓
Critique
↓
...
```

can continue unnecessarily.

Example:

```
MAX_ATTEMPTS = 3
```

Router:

```
if state["attempts"] >= MAX_ATTEMPTS:

    return END
```

20. Reflection is Not Verification

Important:

LLM evaluates LLM output

does not guarantee correctness.

For factual or structured tasks, use deterministic checks where possible.

Example:

```
LLM Generated JSON
↓
Pydantic Validation
↓
Valid?
```

instead of asking another LLM:

"Does this look like valid JSON?"

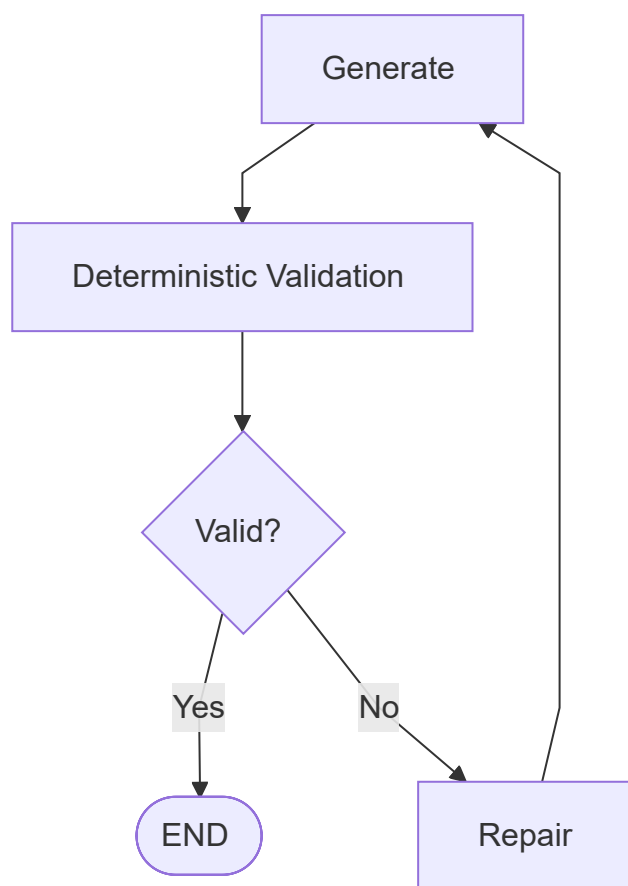
Use code when code can verify the condition reliably.

21. Deterministic Validation Node

Example:

```
def validation_node(state):  
  
    valid = validate_result(  
        state["answer"]  
    )  
  
    return {  
        "is_valid":  
            valid  
    }  
}
```

Graph:



This is often more reliable than pure LLM reflection.

22. Routing Node vs Work Node

A useful design distinction:

Work Node

Performs meaningful work.

Example:

```
Call LLM  
  
Retrieve Documents  
  
Execute Tool  
  
Validate Result
```

Routing Logic

Determines where execution should go.

Example:

```
Tool Needed?  
  
Approved?  
  
Valid?  
  
Plan Complete?
```

Try not to mix all routing and execution logic unnecessarily.

23. Subgraphs



As graphs become larger, putting everything into one graph can become difficult.

Example:

```
graph TD
    MainAgent[Main Agent] --> CropWorkflow[Crop Recommendation Workflow]
    MainAgent --> MarketWorkflow[Market Analysis Workflow]
    MainAgent --> ReportWorkflow[Report Generation Workflow]
```

Each workflow can itself be represented as a graph.

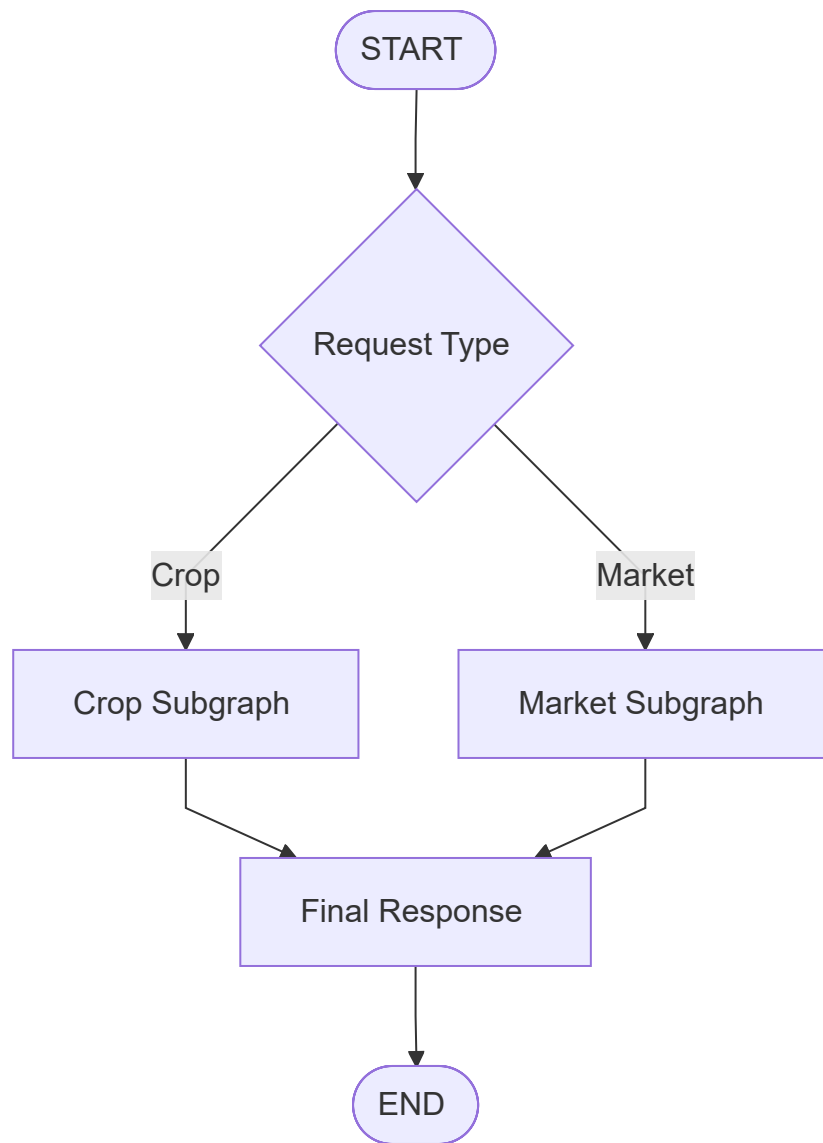
This leads to:

Subgraphs

24. What is a Subgraph?

A **subgraph** is a graph used as part of another graph.

Conceptually:

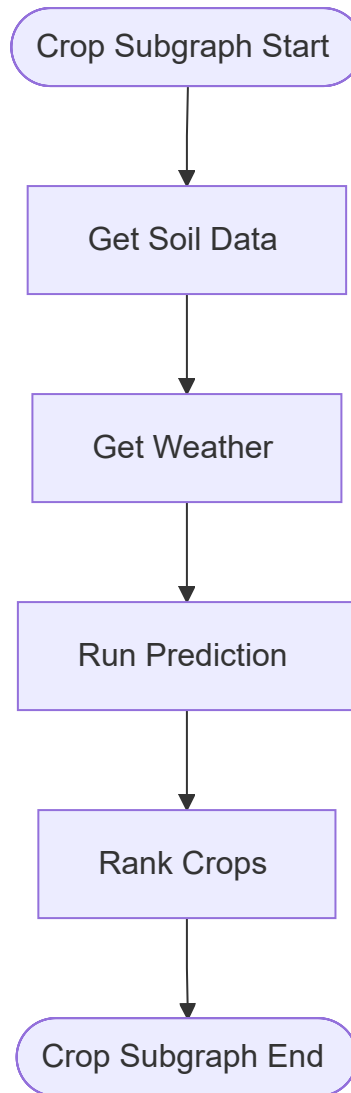


Inside:

Crop Subgraph

there may be another complete workflow.

25. Crop Subgraph Example



The parent graph does not need to manage every internal detail.

26. Why Use Subgraphs?

Subgraphs help with:

Modularity

Reuse

Testing

Separation of responsibilities

Managing large workflows

Instead of:

One giant graph with 30 nodes

we can organize:

Main Graph

├─ Crop Subgraph

├─ Market Subgraph

└─ Report Subgraph

27. Subgraph Mental Model

This is similar to normal programming:

Large Program

↓

Functions / Modules

In LangGraph:

Large workflow

↓

Subgraphs

Do not create subgraphs unless they improve organization.

28. Parallel Branches

Some tasks are independent.

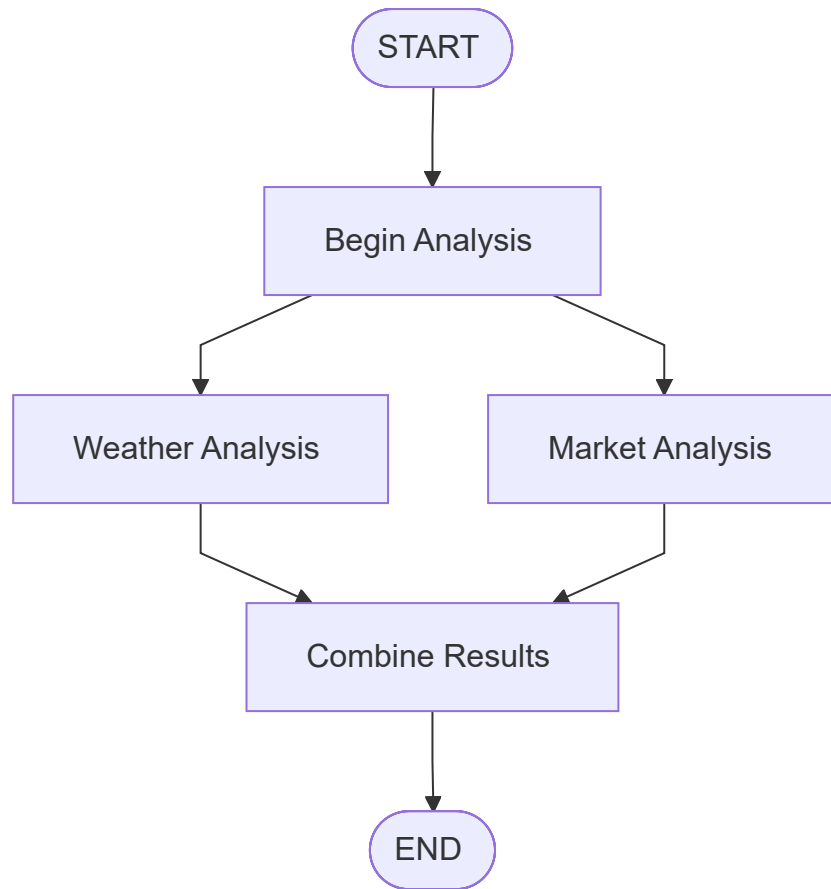
Example:

Get weather

Get Market Price

Neither necessarily depends on the other.

Conceptually:



Independent work may be executed concurrently where the framework and state design support it.

29. Why Parallel Execution Helps

Suppose:

Weather API = 2 seconds

Market API = 2 seconds

Sequential:

weather

↓

2 sec

Market

↓

2 sec

Total $\approx$ 4 sec

Parallel:

weather —┐

└─ Combine

Market —┐

Potential latency:

$\approx$ 2 sec

plus orchestration overhead.

30. Parallel State Updates Need Care

Suppose two parallel nodes both update:

`state["results"]`

The graph needs to know how these updates combine.

This connects back to:

Reducers

For example:

```
Weather Result
+
Market Result
↓
Reducer
↓
Combined Results
```

State design becomes especially important with parallel branches.

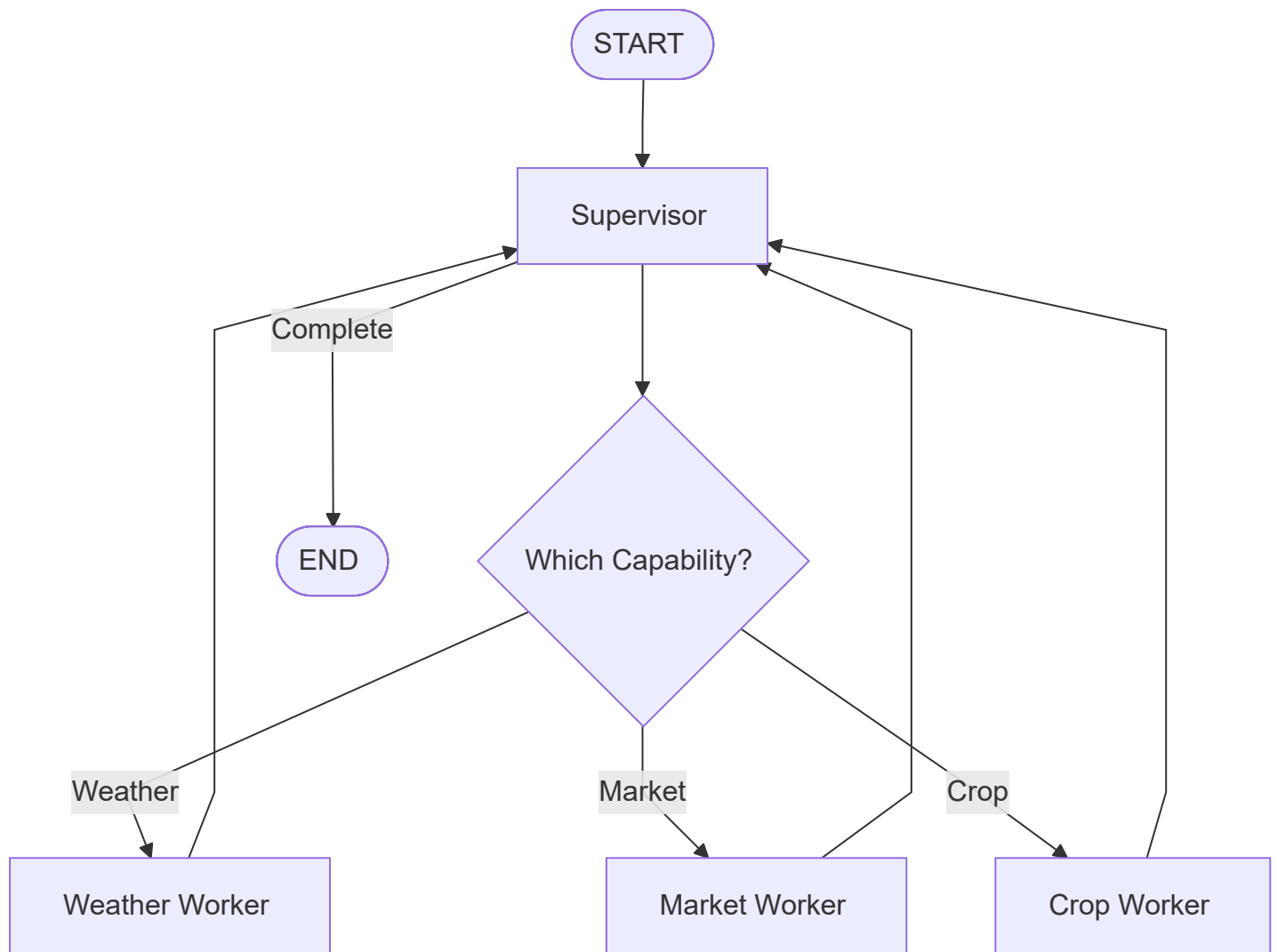
31. Supervisor Pattern

A more complex workflow may use a coordinator.

Conceptually:

```
Supervisor
↓
Choose specialized worker
```

Example:



The supervisor decides which specialized component should act.

32. Is Supervisor Always Multi-Agent?

Not necessarily.

A supervisor can route between:

Nodes

Tools

Subgraphs

Specialized agents

The important concept is:

Central coordination

We will keep detailed multi-agent architectures separate rather than mixing them deeply into basic LangGraph.

33. Agent Node vs Subgraph vs Tool

Understand the distinction.

Tool

specific capability

Example:

`get_weather(city)`

Node

workflow step

Example:

Analyze weather suitability

Subgraph

complete reusable workflow

Example:

Crop recommendation pipeline

34. Choosing the Right Abstraction

Example:

Get current weather

Use:

Tool

Example:

Determine whether weather information
is required.

Use:

Node / Routing Logic

Example:

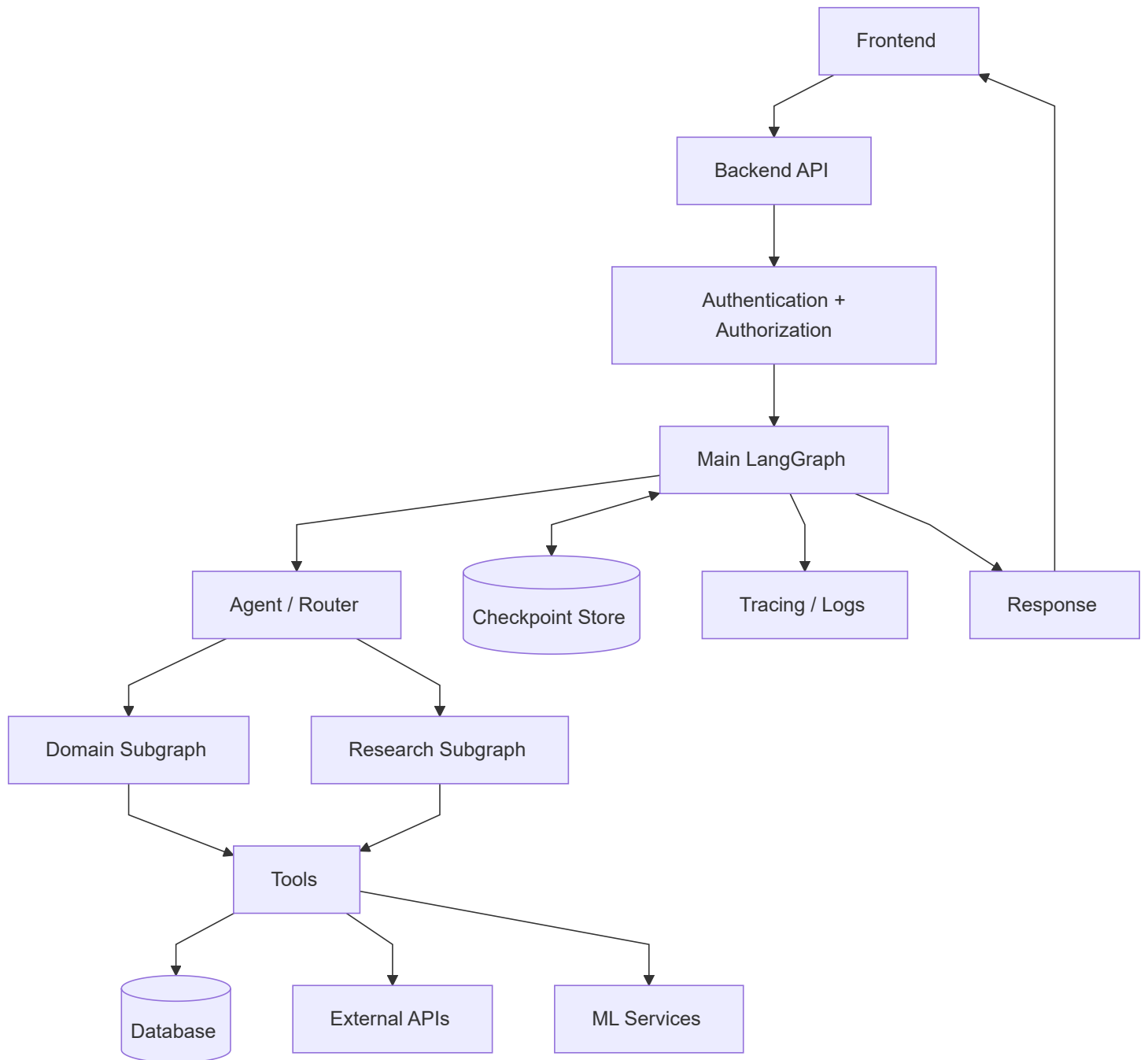
Complete crop recommendation workflow
with weather + soil + ML prediction.

Use:

Subgraph

35. Production LangGraph Architecture

A real application may look like:



Notice:

LangGraph is one part of the backend.

It does not replace the backend.

36. LangGraph Does Not Replace Your API Server



You may still have:

FastAPI

Express

Django

Flask

Example:

React



Express / FastAPI



LangGraph



LLM + Tools

LangGraph handles agent orchestration.

Your backend handles:

HTTP APIs

Authentication

Authorization

Validation

Rate limits

Database access

Business logic

37. LangGraph Does Not Replace Your Database

Graph state is not a replacement for all application data.

Keep normal application data in appropriate storage.

Example:

Users

→ PostgreSQL / MongoDB

Crop records

→ Database

Orders

→ Database

Agent checkpoints

→ Checkpoint storage

Semantic knowledge

→ Vector database when needed

38. LangGraph Does Not Replace ML Models

Suppose your application already has:

Random Forest Crop Predictor

Profit Prediction Model

Do not ask the LLM to replace them.

Instead expose meaningful capability:

```
@tool
def predict_crop(...):
    ...
```

Architecture:

Agent



Crop Prediction Tool



Existing ML Model



Prediction



Agent

This is a powerful pattern for combining:

Traditional ML

+

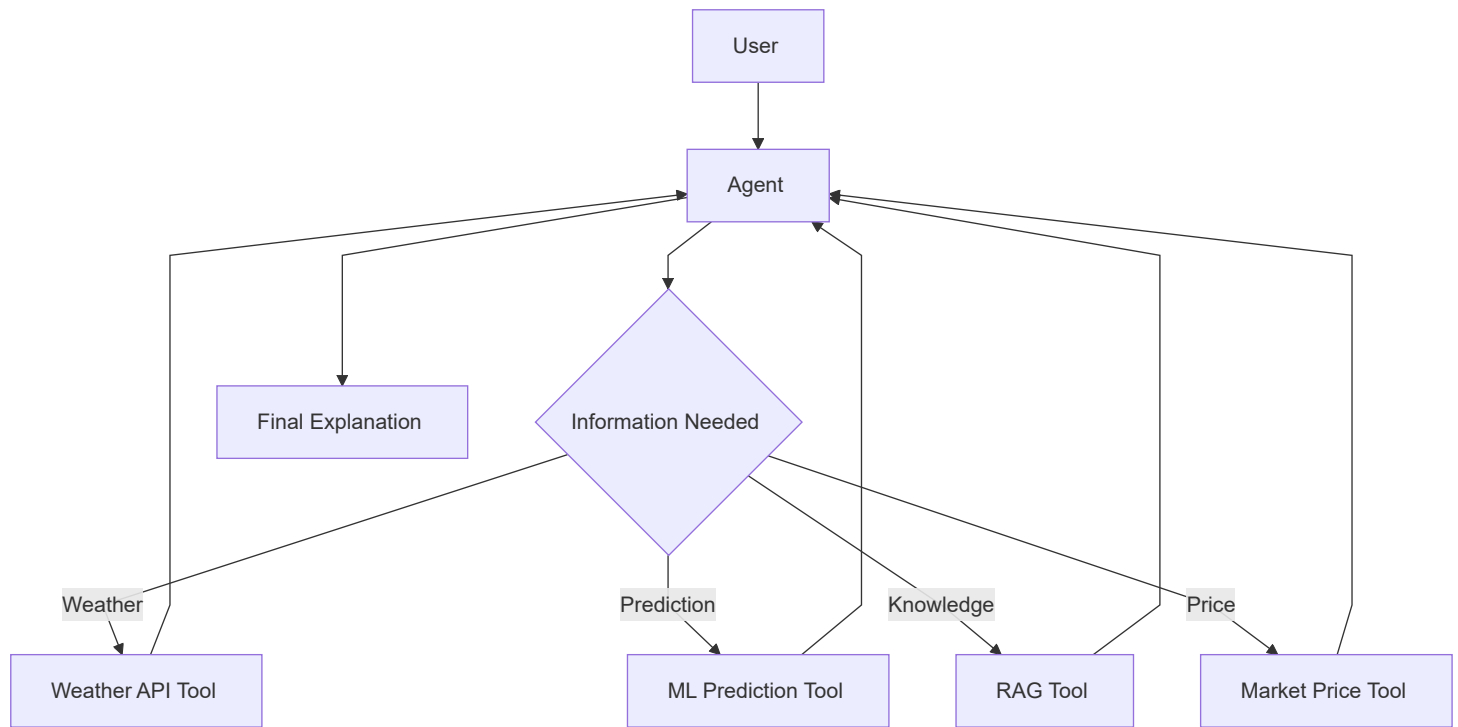
Generative AI

+

Agentic workflow



39. Example Hybrid AI Architecture



The LLM coordinates.

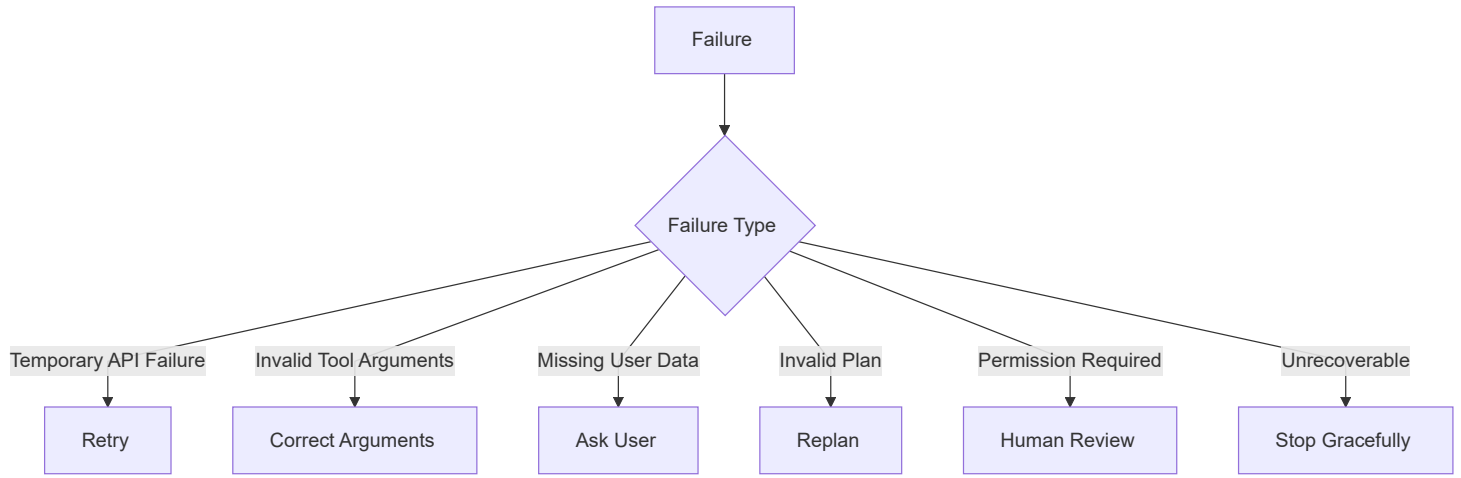
Specialized systems perform the actual domain work.

40. Error Handling Strategy



Do not use one recovery strategy for every failure.

Classify failures.



This makes agent behavior more predictable.

41. Retry Limits

Example state:

```
class AgentState(MessagesState):  
    retry_count: int
```

Then:

```
MAX_RETRIES = 3
```

Router:

```
def route_error(state):  
  
    if state["retry_count"] < MAX_RETRIES:  
  
        return "retry"  
  
    return "fallback"
```

42. Timeouts

External tools can hang.

Example:

```
Agent  
↓  
External API  
↓  
No Response
```

Your application should use:

```
HTTP timeouts  
  
Database timeouts  
  
Tool execution limits
```

Do not allow one tool to block the workflow indefinitely.

43. Tool Result Validation

Never assume every external result is correct.

Example API response:

```
{  
  "temperature": null  
}
```

Before giving it to downstream logic:

```
Tool Result  
↓  
validate  
↓  
valid?
```

You may use:

```
Pydantic  
  
JSON Schema  
  
Normal Python validation
```

44. Structured State is Better Than Random Text

Bad:

```
state["data"] = ""  
weather okay  
crop maybe wheat  
price around something  
""
```

Better:

```
state["weather"] = {  
  "temperature": 31,  
  "rainfall": 12  
}  
  
state["crop_prediction"] = {  
  "crop": "wheat",  
  "confidence": 0.91  
}
```

Structured state makes routing and validation easier.

45. Keep LLM Responsibilities Limited



Use the LLM for tasks it is good at:

Understanding language

Semantic classification

Tool selection

Planning

Summarization

Explanation

Flexible reasoning

Use normal code for:

Authentication

Authorization

Exact calculations

Validation

Database constraints

Security

Deterministic rules

46. Observability

Production agents should expose enough information for debugging.

Track:

Request ID

Thread ID

Executed Nodes

Tool Calls

Tool Latency

Errors

Retry Count

Token Usage

Overall Latency

Example:

Request
↓
Planner 400 ms
↓
Weather Tool 800 ms
↓
ML Tool 120 ms
↓
Agent 500 ms
↓
Response

This helps identify performance problems.

47. Cost Control

Agent loops can create multiple LLM calls.

Example:

Planner

→ 1 LLM call

Agent

→ 3 LLM calls

Reflection

→ 2 LLM calls

Finalizer

→ 1 LLM call

Total:

7 LLM calls

For a task that might have required only:

2 calls

Therefore:

More agentic

≠

Always better

48. Latency Control

Every additional:

LLM call

Tool call

Reflection step

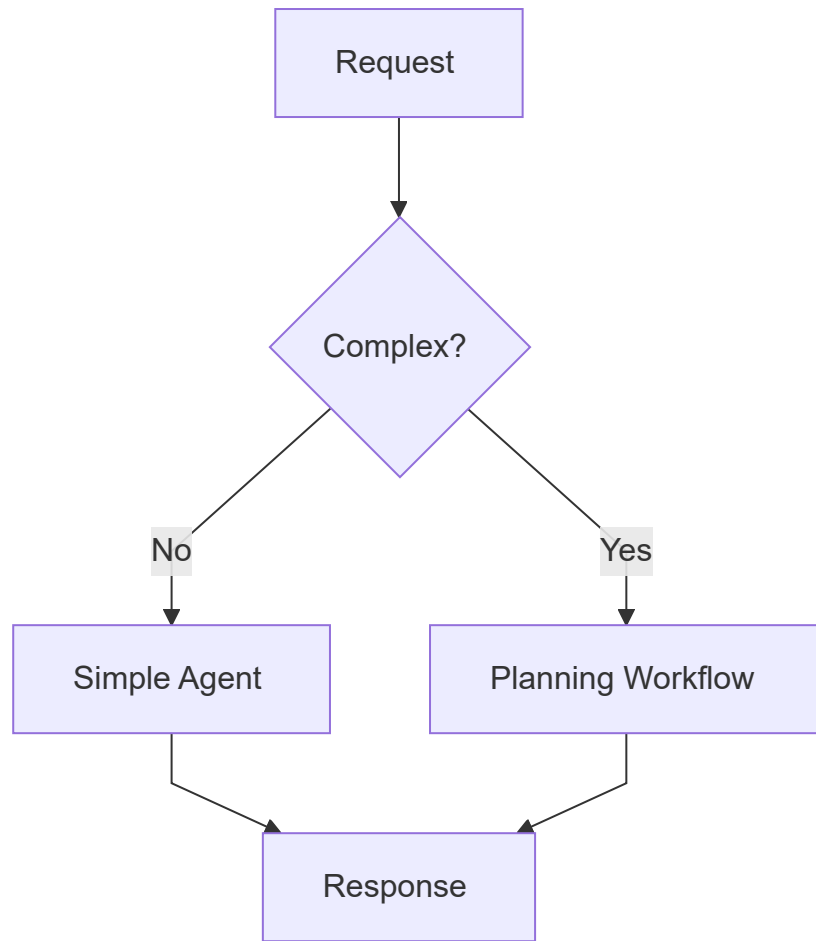
Planning step

adds latency.

For simple queries, prefer:

Direct Route

Example:



This keeps simple requests fast.

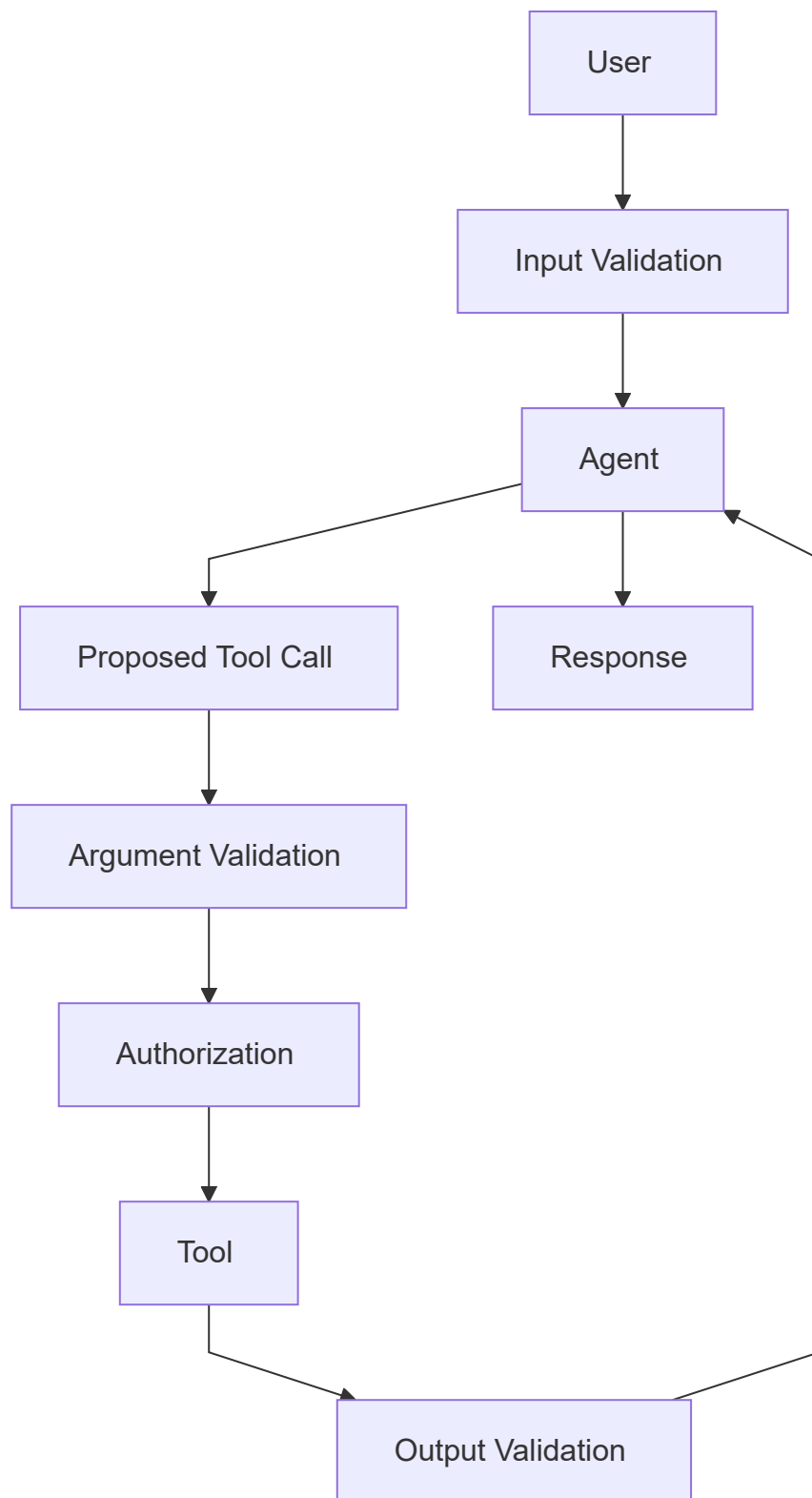
49. Production Safety Layers

A production agent should not be:

User
↓
LLM
↓
Database

with unrestricted access.

Better:



This creates deterministic boundaries around the model.

50. Tool Permissions

Different users may have different permissions.

Example:

```
Farmer
→ Read crop information

Admin
→ Update crop information
```

Do not rely on:

```
LLM deciding whether user is admin.
```

Instead:

```
Authenticated User
↓
Backend Role
↓
Permission Check
↓
Tool Execution
```

51. Testing LangGraph Workflows



Test individual nodes first.

Example:

```
def test_weather_node():  
  
    state = {  
        ...  
    }  
  
    result = weather_node(state)  
  
    assert result is not None
```

Then test routing:

```
def test_router():  
  
    state = {  
        "approved": True  
    }  
  
    assert route(state) == "execute"
```

Finally test the complete graph.

52. What Should Be Tested?

Node behavior

Routing

Tool validation

Retry limits

Failure paths

Human approval

Persistence

Authorization

Graph termination

Do not test only the happy path.

53. Graph Visualization

A major advantage of graph-based workflows is that the architecture can be visualized.

Example:

```
graph TD
    Planner --> Executor
    Executor --> Validator
    Validator --> Retry
    Validator --> Final
```

Visualization helps:

Development

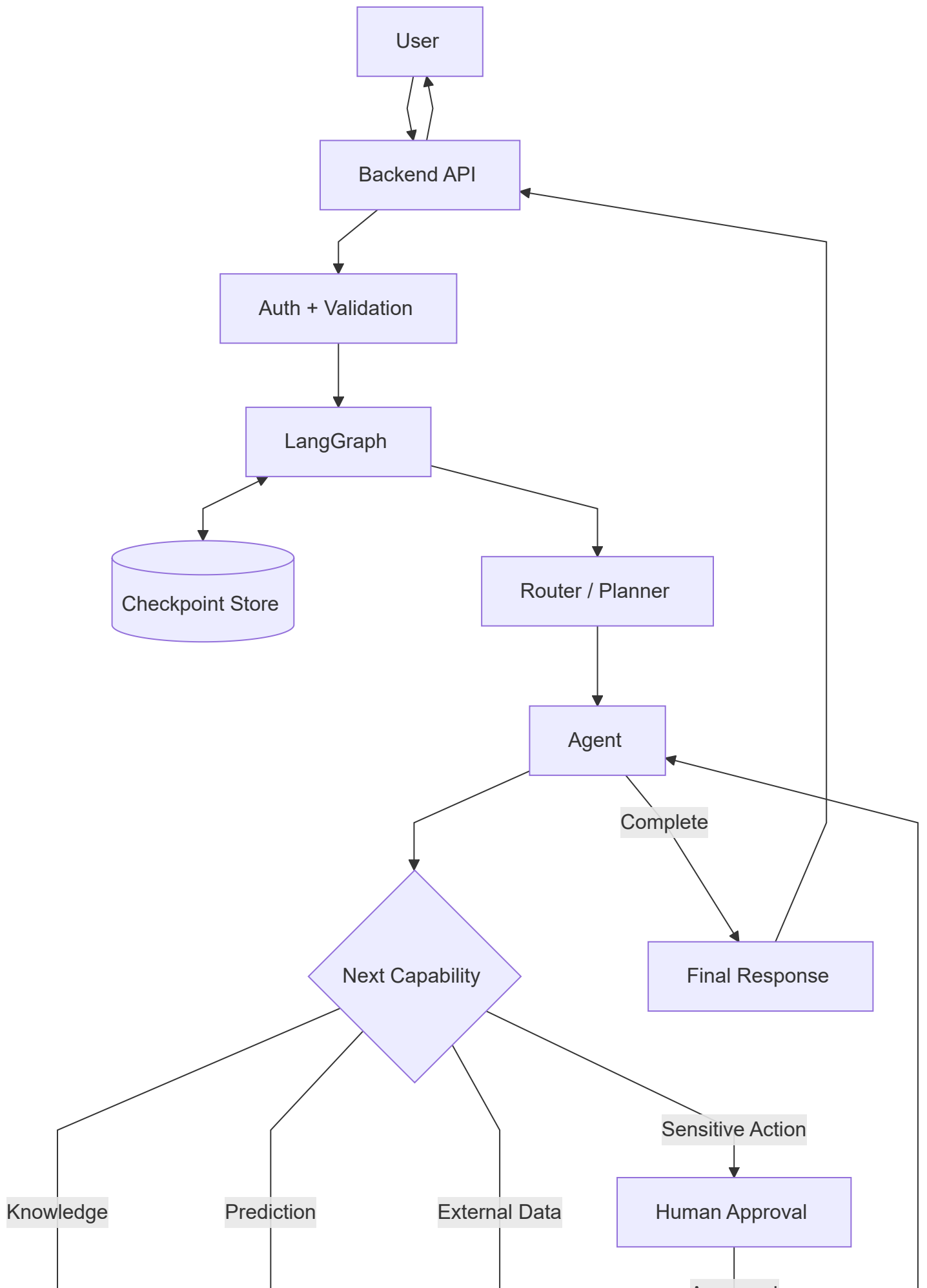
Debugging

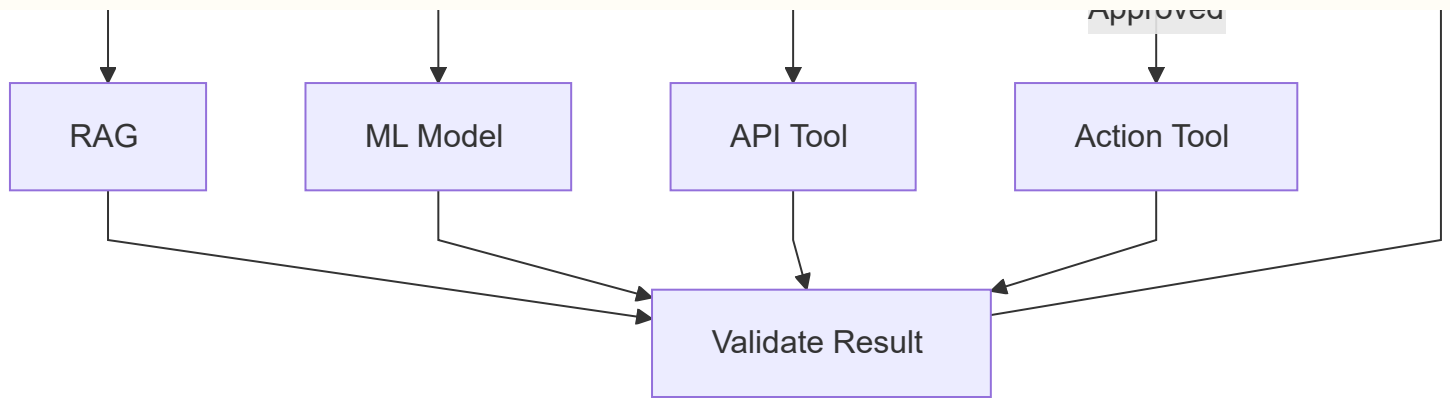
Documentation

Team communication

Your Mermaid diagrams can also mirror the real graph architecture.

54. Complete Production Mental Model





55. How to Decide Whether LangGraph is Needed

Ask:

- Does the workflow have multiple steps?
- Does execution branch dynamically?
- Can it loop?
- Does state need to persist?
- Do I need human approval?
- Can execution pause and resume?
- Do different tools/workflows interact?

If several answers are:

YES

LangGraph may be useful.

If the workflow is:

```
graph TD; Input --> LLM; LLM --> Output;
```

then probably not.

56. Common Overengineering

Bad:

```
graph TD; User[User: Explain photosynthesis.];
```

Architecture:

```
graph TD; Supervisor[Supervisor Agent] --> Planner[Planner Agent]; Planner --> Research[Research Agent]; Research --> Critic[Critic Agent]; Critic --> Reflection[Reflection Agent]; Reflection --> Writer[Writer Agent];
```


This is unnecessary.

Better:

```
User
↓
LLM
↓
Answer
```

Use complexity only when the problem requires it.

57. LangGraph Design Checklist

Before building a graph, decide:

1. what is the goal?
2. what state is required?
3. what are the meaningful nodes?
4. which transitions are deterministic?
5. which transitions require LLM decisions?
6. which tools are required?
7. where can loops occur?
8. what stops the loops?
9. what should persist?
10. where is human approval required?

11. What failures can occur?

12. How will they be handled?

This prevents random graph design.

58. Important Interview Questions

Q1. How can planning be implemented in LangGraph? ★ ★ ★ ★ ★

Answer:

Planning can be represented using a planner node that converts the user goal into a structured plan stored in graph state. Executor nodes then perform the plan steps, while conditional edges determine whether execution should continue, finish or replan.

Q2. What is replanning?

Answer:

Replanning modifies the existing plan when observations or failures show that the original plan can no longer achieve the goal effectively.

Q3. How can reflection be implemented in LangGraph? ★ ★ ★ ★

Answer:

A reflection or evaluator node can inspect an intermediate result and route execution either toward completion or back toward generation/correction based on the evaluation.

Q4. Why should reflection loops have limits?

Answer:

Without limits, the workflow may repeatedly generate and critique outputs, increasing cost and latency or creating an infinite loop.

Q5. What is a subgraph? ★ ★ ★ ★ ★

Answer:

A subgraph is a reusable graph workflow that is used as part of a larger graph, helping modularize complex applications.

Q6. Tool vs node vs subgraph?

Answer:

A tool represents a specific external capability, a node represents a workflow step, and a subgraph represents a larger reusable workflow containing multiple nodes and transitions.

Q7. Can LangGraph execute independent branches?

Answer:

Yes, graph workflows can represent independent branches that may execute concurrently when supported by the workflow and state design.

Q8. Why are reducers important with parallel branches?

Answer:

Multiple branches may update the same state field, so reducers define how those updates should be combined safely.

Q9. Does LangGraph replace FastAPI or Express? ★ ★ ★ ★ ★

Answer:

No. LangGraph orchestrates agent workflows, while FastAPI or Express typically handles HTTP APIs, authentication, authorization, validation and other backend responsibilities.

Q10. Does LangGraph replace databases?

Answer:

No. Application data should remain in appropriate databases. LangGraph state and checkpoints primarily support workflow execution.

Q11. Can traditional ML models be used with LangGraph? ★ ★ ★ ★ ★

Answer:

Yes. Existing ML models can be exposed through tools or nodes, allowing an LLM-driven workflow to decide when to use them while the ML model performs the actual prediction.

Q12. How do you make LangGraph applications production-ready?

Answer:

Use deterministic validation, authentication and authorization, tool permissions, persistent checkpoints, retry and timeout limits, idempotency, observability, structured state, error handling and explicit termination conditions.

Q13. Why shouldn't every task use planning and reflection?

Answer:

Planning and reflection add LLM calls, latency, cost and complexity. They should only be used when they materially improve the reliability of complex tasks.

59. Final LangGraph Mental Model

The progression is:

Simple Graph

START

↓

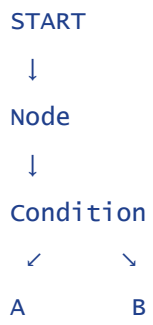
Node

↓

END

Then:

Conditional Graph



Then:

Tool Agent



Then:

Persistent Agent



Then:

Human-in-the-Loop

Agent



Interrupt



Human



Resume

Then:

Advanced workflow

Planner



Executor



Validator



Reflect / Replan



Final

And for large systems:

Main Graph

├ Tools

├ Subgraphs

├ ML Models

├ RAG

├ External APIs

└ Human Approval

The most important engineering rule:

Do not make a workflow more agentic than necessary. Use LLMs for flexible reasoning and semantic decisions, while keeping validation, security, permissions, calculations and deterministic business rules in normal application code.

End of Chapter 4 – LangGraph
